

Universidad Nacional del Centro de la Provincia de
Buenos Aires
Facultad de Ciencias Exactas



Tesis de grado

Un enfoque para la detección automática de fallas
en la construcción de argumentos

Alumnos: Arbiza Miris Julieta, Gareis Carla

Director: Monteserin Ariel

Co-Director: Armentano Marcelo

Agradecimientos

Quiero agradecer en primer lugar a mi familia por confiar en mi y apoyarme en este largo camino.

A mis amigas por alentarme y acompañarme en todo momento.

A Carla, mi compañera en este último tramo, por su dedicación, paciencia, y su gran esfuerzo. Como también agradecer a su familia por el apoyo en cada juntada de estudio.

A los que se fueron durante este camino, pero sin duda fueron un gran empuje para lograr este objetivo.

Por último, agradecer a nuestro director Ariel por guiarnos durante el desarrollo del trabajo final.

Julieta Arbiza Miris.

Quiero agradecerles a mis viejos, que con su esfuerzo y apoyo constante me permitieron completar esta etapa.

A los compañeros que caminaron conmigo este camino.

A Ariel, que con su dedicación y apoyo nos guió en este trabajo.

A Juli, por su paciencia y por empujarme siempre para adelante.

A Nico, porque sin su apoyo incondicional esto no hubiera sido posible.

A mis hijos, Uriel y Lorenzo, que fueron el motor que me llevó a la meta.

Carla Gareis.

Resumen

Definir un argumento como no válido suele ser más sencillo que explicar el por qué de su invalidez e incluso, cuando se refuta un argumento, el por qué no suele ser lo suficientemente sólido.

El principal objetivo de este trabajo, es lograr el desarrollo de un enfoque que permita determinar, dado una sentencia argumentativa expresada en lenguaje natural, las fallas por las cuales se considera inválida. Dicho enfoque brindará el soporte necesario para detectar los errores argumentativos en grandes volúmenes de datos de manera eficiente y con buen performance.

Con el objetivo de profundizar el análisis, se va a realizar un estudio semántico de las sentencias argumentativas. La tarea de integrar la semántica a la sintaxis no es trivial, ya que es necesario adecuar “sentido común” a las máquinas.

Para poder llevar a cabo este estudio se planteó la construcción de un proceso de extracción de conocimiento KDD, implementando cada una de las etapas que lo componen.

Finalmente se obtendrán dos herramientas cuya finalidad es clasificar sentencias mediante la detección de errores en la construcción de las mismas. Para lograr esta clasificación se hará uso de distintos algoritmos con la finalidad de determinar cuál de ellos posee la mayor tasa de acierto.

Índice General

Capítulo 1	9
1.1 Introducción a la problemática	9
1.2 Motivación	9
1.3 Solución propuesta	10
Capítulo 2	12
2.1 Argumentos: Definición y tipos	12
2.2 KDD: Proceso de extracción de conocimiento	14
2.3 Minería de datos	16
2.4 Minería de texto	16
2.5 Aprendizaje automático	17
2.6 Clasificación	18
2.6.1 Naive Bayes	20
2.6.2 Máquinas de Vectores de Soporte (SVM)	21
2.6.3 K-Vecinos Más Cercanos (KNN)	23
2.6.4 Árboles de Decisión ID3 y C4.5	23
2.7 Procesamiento de lenguaje natural (PLN)	24
2.8 Framework de procesamiento de lenguaje natural: Freeling	25
2.9 Framework de aprendizaje automático: Weka	28
2.10 Ontologías para el procesamiento de lenguaje natural	30
2.10.1 Wordnet	31
2.10.2 Multilingual Central Repository (MCR)	34
2.11 Resumen	38
Capítulo 3	41
3.1 Minería de argumentos en ambientes de aprendizaje colaborativo soportado por computadoras	41
3.1.1 Introducción	41
3.1.2 Enfoque de minería de argumentos	42
1. Pre-procesamiento	42
2. Procesamiento de lenguaje natural (PLN)	42
2.1 Enfoque basado en longitud de etiquetas	43
2.2 Enfoque basado en conectores argumentativos	43
3. Construcción del clasificador	44
4. Evaluación experimental	44
4.1 Resultados del enfoque sin etiquetar	45
4.2 Resultados del enfoque basado en longitud de etiquetas	45
4.3 Resultados del enfoque basado en conectores argumentativos	45
3.2 Detección de argumentos en textos legales	45
3.2.1 Introducción	45
3.2.2 Enfoque propuesto	45

3.3 Identificación de argumentos de editoriales chinas	47
3.3.1 Introducción	48
3.4 Procesamiento semántico automático, enfocado en la coherencia textual, para apoyar la producción escrita de noticias	51
3.4.1 Introducción	51
3.4.2 Concepto fundamental: Análisis Semántico Latente	51
3.4.3 Metodología de investigación	52
3.4.3.1 Recopilación automática del corpus	52
3.4.3.2 Preparación automática del corpus	52
3.4.3.3 Construcción del espacio semántico	53
3.4.3.4 Evaluador automático de coherencia textual	53
3.4.3.4.1 Lematización de los textos	53
3.4.3.4.2 Comparación de unidades textuales adyacentes	53
3.4.4 Resultados de las pruebas realizadas al prototipo	54
3.4.5 Conclusiones	54
3.5 Resumen	54
Capítulo 4	56
4.1 Selección de datos	56
4.2 Pre-procesamiento y transformación	57
4.2.1 Tratamiento preliminar del dataset	57
4.2.2 Procesamiento de lenguaje natural	59
4.2.3 Generación de archivo de procesamiento	60
4.3 Data mining	62
4.3.1 Enfoque basado en longitud de etiquetas	63
4.3.2 Enfoque basado en conectores argumentativos	64
4.3.2.1 Análisis sintáctico	65
4.3.2.2 Análisis semántico	65
4.3.2.2.1 Alternativa 1: Salida original	67
4.3.2.2.2 Alternativa 2: Salida ordenada	68
4.3.2.2.3 Alternativa 3: Salida ordenada y sin repeticiones	68
4.4 Implementación	68
4.4.1 Enfoque basado en la estructura sintáctica del argumento	69
4.4.2 Enfoque basado en el análisis semántico del argumento	71
4.5 Resumen	74
Capítulo 5	75
5.1 Enfoque basado en la estructura sintáctica del argumento	75
5.1.1 Métricas utilizadas	75
5.1.2 Dataset inicial sin etiquetar	76
5.1.3 Enfoque basado en conectores argumentativos	78
5.1.4 Enfoque basado en longitud de etiquetas	80

5.1.4.1 Longitud de etiqueta = 6	80
5.1.4.2 Longitud de etiqueta = 3	82
5.1.4.3 Longitud de etiqueta = 2	84
5.2 Enfoque basado en el análisis semántico del argumento	86
5.2.1 Alternativa 1: Salida original	87
5.2.2 Alternativa 2: Salida ordenada	87
5.2.3 Alternativa 3: Salida ordenada y sin repeticiones	88
5.2.4 Comparación de resultados	89
5.3 Resumen	89
Capítulo 6	92
6.1 Limitaciones	93
6.2 Trabajos futuros	94

Índice de figuras

Figura 1. Etapas del proceso KDD.	15
Figura 2. Carga de trabajo en las fases de un proyecto de datamining.....	16
Figura 3. Esquema de la clasificación automática de textos	19
Figura 4. Tabla de contingencia para la clase ci	20
Figura 5. Hiperplano de separación óptimo con margen de tamaño máximo	23
Figura 6. Arquitectura de un sistema de procesamiento de lenguaje natural	25
Figura 7. Clases de datos lingüísticos en Freeling 3.0	27
Figura 8. Estructura de archivo ARFF	30
Figura 9. Ejemplo de ontología	31
Figura 10. Ejemplo de relación entre synsets	34
Figura 11. Estructura de la base de datos para MCR y WEI	38
Figura 12. Enfoque de minería de argumentos propuesto por dicho trabajo	42
Figura 13. Ejemplos de etiquetas	43
Figura 14. Lista de indicadores de premisa	44
Figura 15. Lista de indicadores de conclusión	44
Figura 16. Clasificación de párrafos realizado por AMT	49
Figura 17. Resultados de la tres variantes propuestas por Chow	51
Figura 18. Pipeline de procesamiento de lenguaje natural	59
Figura 19. Fragmento de archivo de entrenamiento de Weka sin etiquetar	61
Figura 20. Fragmento de archivo de entrenamiento de Weka etiquetado	62
Figura 21. Etiqueta de Freeling	63
Figura 22. Ejemplos de configuración de indicadores de premisa	64
Figura 23. Ejemplos de configuración de indicadores de conclusión	65
Figura 24. Ejemplo de búsqueda de relaciones semánticas	67
Figura 25. Contenido de archivo Premisas.txt	68
Figura 26. Contenido de archivo Conclusiones.txt	68
Figura 27. Contenido de archivo Premisas.txt	68
Figura 28. Contenido de archivo Conclusiones.txt	68
Figura 29. Contenido de archivo Premisas.txt	69
Figura 30. Contenido de archivo Conclusiones.txt	69
Figura 31. Herramienta resultante para el enfoque sintáctico	69
Figura 32. Diagrama de secuencia	71
Figura 33. Estructura del dataset de entrada	72
Figura 34. Herramienta resultante para el enfoque semántico	73
Figura 35. Diagrama de secuencia	74
Figura 36. Matriz de confusión para dataset sin etiquetar	76
Figura 37. Gráficos de variables para el dataset sin etiquetar	77
Figura 38. Matriz de confusión el dataset con etiqueta diferenciada entre indicadores	78
Figura 39. Gráficos de variables para el dataset sin etiquetar	79
Figura 40. Matriz de confusión para longitud de etiqueta n=6	80
Figura 41. Gráficos de variables para el dataset de longitud de etiqueta n=6	81
Figura 42. Resultados para longitud de etiqueta n=3	82
Figura 43. Gráficos de variables para el dataset de longitud de etiqueta n=3	83
Figura 44. Matriz de confusión para longitud de etiqueta n=2	84
Figura 45. Gráficos de variables para el dataset de longitud de etiqueta n=2	85

Figura 46. Resultados del enfoque semántico	86
Figura 47. Gráfico de performance para la alternativa 1	87
Figura 48. Gráfico de performance para la alternativa 2	88
Figura 49. Gráfico de performance para la alternativa 3	88
Figura 50. Gráfico comparativo	89

Capítulo 1

1. Introducción

1.1 Introducción a la problemática

Considerando un contexto en el cual los alumnos deben realizar un análisis y argumentar con solidez sus resultados y conclusiones con el objetivo de persuadir a las otras personas para que compartan sus convicciones, el tener herramientas que garanticen argumentos bien logrados y sólidos es de gran importancia para lograr dicho objetivo.

En dicho contexto, al generarse una interacción entre distintos participantes, la negociación es fundamental para lograr un acuerdo entre ellos y, en esta situación, quien presente argumentos mejor armados y con más solidez, lleva una gran ventaja frente a los demás participantes. El objetivo de una negociación es el intercambio de propuestas o puntos de vista para lograr un punto en común donde todos los participantes estén de acuerdo. En una negociación, un participante puede hacer uso de la argumentación para respaldar su propuesta o para influir en las posiciones o fundamentos de los demás participantes [1].

En un contexto de aprendizaje colaborativo asistido por computadora (CSCL), la construcción de estos argumentos es aún más compleja que en las argumentaciones cara a cara, dado que existen factores que dificultan la construcción de argumentos. Por dicha razón, consideramos que es de gran ayuda tener una herramienta que determine cuándo un argumento está completo y es válido.

Adicionalmente, al contemplar la interacción de gran cantidad de usuarios en los sistemas CSCL, surge la necesidad de automatizar ciertos procesos, que por su volumen, dificulta el procesamiento manual que puede realizar un experto.

Lo primero que se plantea para poder asistir a los usuarios es la automatización del análisis de sus argumentos, los cuales están expresados en lenguaje natural. Para alcanzar dicho objetivo a lo largo de este plan, se llevará a cabo un enfoque que permita detectar automáticamente cuáles son las fallas en la construcción de argumentos.

1.2 Motivación

Definir un argumento como no válido suele ser más sencillo que explicar el por qué de su invalidez e incluso, cuando se refuta un argumento, el por qué no suele ser lo suficientemente sólido. De esta manera, lograr la implementación de una herramienta que determine el motivo por el cual una sentencia no es válida resultaría, a nuestro entender, de gran ayuda en el análisis del lenguaje natural.

Para lograr detectar que una sentencia no es válida se debe, en primer lugar, analizar los posibles formatos de un argumento. Tradicionalmente, un argumento es definido como un conjunto de premisas al cual sigue una conclusión [2]. Por ejemplo:

“Dado que estos últimos meses hubo muchas precipitaciones y coseche 100

hectáreas de trigo, entonces voy a perder dinero”.

El indicador de conclusión “entonces” indica que lo que aparece a continuación es la conclusión, y lo anterior son las premisas. Pero, cuando hablamos de argumentos expresados en lenguaje natural y específicamente en un contexto en el cual los alumnos deben realizar un análisis y argumentar con solidez sus resultados, esta estructura puede ser difusa.

Dado que las partes que conforman un argumento no siguen ningún orden, no siempre es válido presuponer que siempre las premisas se encontrarán al inicio y la conclusión al final del mismo. Un argumento puede estar bien definido y la estructura no ser la convencional, de manera tal que las partes se podrían encontrar invertidas o entrelazadas.

Otro desafío que se presenta al determinar la invalidez de un argumento, además de su estructura, es evaluar la coherencia entre las partes que conforman la misma. Actualmente, la mayoría de los analizadores que existen, trabajan de manera morfo-sintáctica, sin tomar en cuenta el aspecto semántico en el lenguaje. La tarea de integrar la semántica a la sintaxis no es trivial, ya que es necesario adecuar “sentido común” a las máquinas [3]. Por ejemplo:

“Dado que estos últimos meses hubo muchas precipitaciones, entonces voy a prender la radio”.

Si bien la estructura del argumento es correcta, semánticamente, el contexto de la premisa y de la conclusión es diferente, lo cual convierte al mismo en inválido.

Este desafío adicional permite obtener una gran mejora en cuanto a la calidad de los estudios realizados sobre las fallas en los argumentos, logrando abarcar una mayor amplitud en el dominio del problema.

1.3 Solución propuesta

El principal objetivo de esta tesis es lograr el desarrollo de un enfoque que permita determinar, dado una sentencia expresada en lenguaje natural, las fallas argumentativas por las cuales se considera inválida.

La primera tarea, consiste en analizar los posibles formatos estructurales de argumentos que se pueden hallar. Luego, se realizará un análisis en busca de las fallas que determinan la invalidez del argumento. Una vez identificadas dichas fallas, se trabajará a partir de una base de datos con un volumen suficiente de argumentos de cada uno de los tipos detectados.

A partir de esta estructura mencionada anteriormente, se hará uso de un módulo de procesamiento de lenguaje natural, el cual permitirá etiquetar los textos de entrada con el objetivo de abstraer el concepto de cada una de las sentencias de su contexto general con el fin de obtener una salida mejorada y así lograr un análisis más certero [4][5]. Con esto se intentará determinar la función de cada palabra o frase dentro de dichas sentencias (verbo, sustantivo, adverbio, etc.).

Con el objetivo de comparar los resultados obtenidos y determinar cuál de ellos maximiza la tasa de aciertos, se hará uso de un clasificador que determinará, dado una

sentencia de entrada, el motivo por el cual no se considera válida aplicando distintos algoritmos de análisis, tales como: Naive Bayes, SMO, IBk y J48.

Como última tarea, para lograr determinar otra de las fallas argumentativas, se realizará el estudio semántico de las sentencias. Para esto, se utilizará el módulo anterior que nos proporciona el etiquetado de los mismos con el objetivo de separar la premisa de la conclusión. Luego, se extraerán las palabras más relevantes (sustantivos y/o verbos) de cada una de las partes. Serán comparadas dichas palabras con el fin de determinar si existe una relación entre ellas, obteniendo como resultado final la clasificación de cada sentencia en “válida” o “inválida” respecto a su semántica.

Para lograr lo mencionado en el apartado anterior, se hará uso de una base de datos léxica multilingüe llamada MultiWordNet, la cual agrupa palabras en conjuntos de sinónimos llamados synset, almacenando las relaciones semánticas entre ellas.

Capítulo 2

2. Marco teórico

En el presente capítulo se tratan diversos conceptos que se abordaron en esta tesis, los cuales brindan un marco para facilitar el seguimiento y entendimiento del trabajo realizado.

Como paso inicial, se explica el concepto de “argumento” y los diferentes tipos que se pueden encontrar en los textos argumentativos. Este concepto y su importancia es fundamental para la comprensión del estudio realizado y es el punto de partida del trabajo de tesis ya que el mismo consiste en la detección automática de fallas en la construcción de argumentos, lo que incluye errores estructurales y semánticos.

Seguidamente se definen el proceso de extracción de conocimiento (KDD), minería de datos y de texto. Dicho proceso se utiliza para la detección e identificación de patrones válidos, útiles y entendibles por el usuario a partir de los datos. Este proceso es de suma importancia para la primera etapa de la tesis aquí abordada. Los dos siguientes conceptos están fuertemente relacionados con KDD, incluso forman parte de dicho proceso.

Luego, se define el aprendizaje automático haciendo énfasis en la técnica de clasificación y los diferentes algoritmos utilizados para este fin. Este tipo de aprendizaje es fundamental para lograr entrenar la herramienta de clasificación implementada, con el objetivo de obtener buenos resultados. El concepto de clasificación y los algoritmos se explican con el objetivo de comprender el funcionamiento de la herramienta desarrollada.

Seguidamente, se explica el concepto de “procesamiento de lenguaje natural” (PLN) y las etapas que lo componen. Se mencionan los diferentes framework que se utilizaron para el procesamiento, uno de ellos es Freeling, el cual realiza un procesamiento sobre los datos de entrada con el objetivo de generar datos etiquetados, los cuales serán posteriormente interpretados y analizados con Weka, el cual nos permitirá, además, obtener el resultado que maximice el análisis propuesto.

Por último, se explican las ontologías utilizadas para el análisis semántico, Wordnet y MCR, las cuales son la base de la cual se obtienen los resultados de dicho análisis dado que permiten realizar consultas directamente a bases de datos.

2.1 Argumentos: Definición y tipos

Un argumento puede definirse como una serie de aseveraciones (oraciones, proposiciones), sea dentro de una oración más amplia o dentro de un grupo de oraciones gramaticalmente diferenciadas, que individualmente o en su conjunto dicen "apoyar", "demostrar" o "dar prueba de" otra aseveración. La aseveración que resulta apoyada, demostrada o probada se denomina conclusión.[34]

Hay una sola conclusión para cada argumento. Pero puede haber toda una serie de aseveraciones de apoyo. Las aseveraciones que brindan apoyo, demostración o prueba se denominan premisas.

La clave para reconocer un argumento está en la conclusión: a una conclusión, corresponde un argumento. Cabe advertir, asimismo, que los términos "premisa" y "conclusión" son relativos: o sea que una misma aseveración puede ser tanto premisa en determinado argumento, como conclusión de otro. Ello se debe a que puede haber toda una serie de argumentos entrelazados.

Un argumento puede ser formalmente correcto o válido, pero, aun así, carecer de premisas ciertas. Cuando un argumento es válido pero al menos una de sus premisas es falsa, entonces se dice que el argumento no es sólido. Cuando el argumento es formalmente válido y sus premisas son todas ciertas, entonces dícese que dicho argumento es sólido. El argumento ideal es, claro está, sólido.

Como se mencionó anteriormente, la estructura de los argumentos se compone de premisas y de una conclusión: las premisas deben apoyar, demostrar o brindar pruebas que sostengan a la conclusión. Pero antes de poder analizar la estructura de un argumento y evaluar las pruebas presentadas en las premisas, debemos estar capacitados para identificar a un argumento y sus partes. Para realizar esto existen las siguientes reglas:

1. No se debe identificar premisas y conclusiones por su contenido. Como se dijo en el apartado anterior, una aseveración puede ser premisa en un determinado argumento y conclusión en otro.
2. No se debe identificar premisas y conclusiones por su posición o ubicación dentro del párrafo. Generalmente, los argumentos comienzan con las premisas seguidas de la conclusión, pero puede suceder que, por razones de estilo o de alguna otra índole, el argumento comience con la conclusión seguida de las premisas.
3. La aparición de la siguiente palabra o palabras al comienzo de una oración o proposición significa que dicha oración o proposición podría ser el comienzo de una conclusión: por lo tanto, por ende, así, de ahí que, en consecuencia, por consiguiente, se desprende que, como resultado, llegamos a la conclusión, y otros términos sinónimos.
4. La aparición de la siguiente palabra o palabras al comienzo de una oración o proposición significa que dicha oración o proposición constituye una premisa: puesto que, ya que, como, en tanto que, dado que, por cuanto, viendo que, y otros términos sinónimos.
5. Las premisas y conclusiones pueden ser implícitas o faltantes. Muchas veces un argumento queda planteado con tanta claridad dentro del contexto, que el autor sólo lo enuncia parcialmente. En estos casos queda librado al lector el que agregue las premisas faltantes o presuntas, o aun la conclusión. Este tipo de argumento se denomina *entimema*.

De acuerdo a la información que respalda un argumento, estos se pueden clasificar de la siguiente manera:

- Argumentos basados en datos: Se presenta información irrefutable, datos estadísticos o demostrables, así como también informaciones que se presumen como ciertas o válidas. Sin embargo, no necesariamente la inferencia argumentos-conclusión será cierta, a pesar de que lo sean los datos que la sustentan.
- Argumentos basados en valores: Se apela al sentido emotivo o afectivo del receptor, que al ser convencido de que algo es bueno o malo en sí mismo, seguramente se inclinara por la aceptación o el rechazo natural a eso.
- Argumentos basados en descripciones: Es una forma de acumular varias argumentaciones en una. Si se trata de persuadir sobre la importancia de algo en virtud de muchas causas, lo más lógico será describir ese fenómeno, añadiendo detalles que terminen por concluir en la ventaja o desventaja de su aplicación.
- Argumentos basados en comparaciones: Como las personas suelen verse tan reflejadas en lo que le ocurre a los demás, suelen tener mucho efecto este tipo de argumentos. Se presume que como otro estaba en la misma situación, tomó cierto camino y tuvo ciertos resultados, cuando uno esté en esa situación tomando el mismo camino tendrá los mismos resultados.
- Argumentos por generalización: En algún punto similar a las comparaciones, pero con algo más de validez científica: si muchos casos iguales en una variable cumplen con cierta condición, probablemente otros con la misma variable también la cumplen.
- Falacias: En todos los casos anteriores se ha mencionado la posibilidad de que un argumento parezca cierto, pero no lo sea. Estos casos de engaños han tenido distintas divisiones respecto a donde esté el engaño (afirmación de consecuentes, generalizaciones demasiado amplias, por autoridad, número de adeptos o antigüedad, entre otros tantos), pero comparten la cuestión de una falsa conexión entre las premisas y la conclusión.

2.2 KDD: Proceso de extracción de conocimiento

El proceso de extraer conocimiento a partir de grandes volúmenes de datos ha sido reconocido por muchos investigadores como un tópico de investigación clave en los sistemas de bases de datos como una importante área y una oportunidad para obtener mayores ganancias [6]. Se define como un proceso no trivial de identificación de patrones válidos, novedosos, potencialmente útiles y fundamentalmente entendibles al usuario a partir de los datos.

No es un proceso automático, es un proceso iterativo que se puede aplicar en diferentes dominios. Consiste en extraer patrones en forma de reglas o funciones, a partir de los datos, para que el usuario los analice. Esta tarea implica generalmente preprocesar los datos, hacer minería de datos (data mining) y presentar resultados [7][8][9][10]. La siguiente figura ilustra las etapas del proceso KDD:

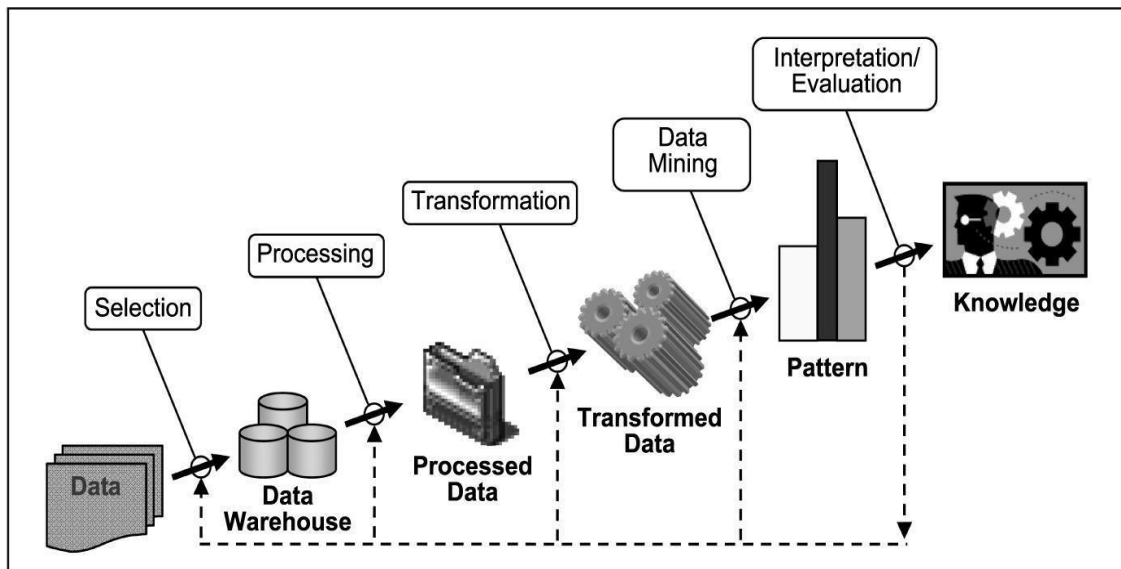


Figura 1. Etapas del proceso KDD

Como muestra la figura 1, las etapas del proceso KDD se dividen en 5 fases:

1. Selección de datos. En esta etapa se determinan las fuentes de datos y el tipo de información a utilizar. Es la etapa donde los datos relevantes para el análisis son extraídos desde la/s fuente/s de datos.
2. Preprocesamiento. Esta etapa consiste en la preparación y limpieza de los datos extraídos desde las distintas fuentes de datos en una forma manejable, necesaria para las fases posteriores. En esta etapa se utilizan diversas estrategias para manejar datos faltantes o en blanco, datos inconsistentes o fuera de rango, obteniéndose al final una estructura de datos adecuada para su posterior transformación.
3. Transformación. Consiste en el tratamiento preliminar de los datos, transformación y generación de nuevas variables a partir de las ya existentes con una estructura de datos apropiada. Aquí se realizan operaciones de agregación o normalización, consolidando los datos de una forma necesaria para la fase siguiente.
4. Data Mining. Es la fase de modelamiento propiamente dicho, en la cual los métodos inteligentes son aplicados con el objetivo de extraer patrones previamente desconocidos, válidos, nuevos, potencialmente útiles y comprensibles y que están contenidos u “ocultos” en los datos.
5. Interpretación y evaluación. Se identifican los patrones obtenidos y realmente interesantes, basándose en algunas medidas y se realiza una evaluación de los resultados obtenidos.

Además de las fases descritas, frecuentemente se incluye una fase previa de análisis de las necesidades de la organización y definición del problema, en la que se establecen los objetivos de la minería de datos. También, es usual incluir una etapa final, donde los resultados obtenidos se integran al negocio para la realización de acciones comerciales.

2.3 Minería de datos

La minería de datos se puede definir como una etapa dentro del proceso de KDD. Consiste en un conjunto de técnicas y tecnologías que permiten explorar grandes bases de datos, de manera automática o semiautomática, con el objetivo de encontrar patrones repetitivos, tendencias o reglas que expliquen el comportamiento de los datos en un determinado contexto.

Suele componerse de cuatro etapas principales:

1. Determinación de los objetivos. Trata de la delimitación de los objetivos que el cliente desea bajo la orientación del especialista en data mining.
2. Preprocesamiento de los datos. Se refiere a la selección, la limpieza, el enriquecimiento, la reducción y la transformación de las bases de datos. Esta etapa consume generalmente cerca del setenta por ciento del tiempo total de un proyecto de data mining.
3. Determinación del modelo. Se comienza realizando análisis estadísticos de los datos, para luego llevar a cabo una visualización gráfica de los mismos con el objetivo de lograr obtener una primera aproximación. Según los objetivos planteados y la tarea que debe llevarse a cabo, pueden utilizarse algoritmos desarrollados en diferentes áreas de la Inteligencia Artificial.
4. Análisis de los resultados. Verifica si los resultados obtenidos son coherentes y los coteja con los obtenidos por los análisis estadísticos y de visualización gráfica. El cliente determina si son novedosos y si le aportan un nuevo conocimiento que le permita considerar sus decisiones.

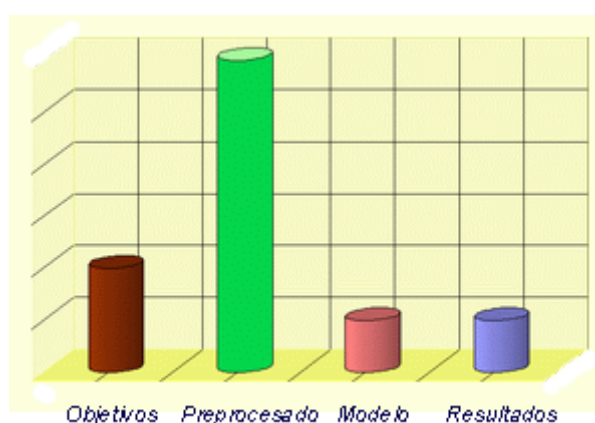


Figura 2. Carga de trabajo en las fases de un proyecto de datamining

2.4 Minería de texto

La minería de texto se suele confundir con la minería de datos, de la cual se diferencia porque en esta última, como se dijo en la sección anterior, la información se obtiene normalmente de bases de datos, en las cuales la información está estructurada. Por tal motivo, es más sencilla la extracción de la información de una base de datos, la cual está diseñada para que se pueda tratar su información de manera automática, al contrario a lo

que ocurre en la minería de textos. De tal manera, se puede decir que la minería de textos puede definirse como una forma específica de minería de datos que trata de obtener información y conocimiento a partir de un conjunto de datos que, en principio, no tienen una estructura determinada.[11]

Su principal objetivo es capturar los temas y conceptos clave y descubrir las relaciones ocultas y las tendencias existentes sin necesidad de conocer las palabras o los términos exactos que se han utilizado para expresar dichos conceptos.

Para lograr esto, la minería de textos comprende tres actividades fundamentales:

1. Recuperación de información, es decir, selección de los textos pertinentes.
2. Extracción de la información incluida en esos textos: hechos, acontecimientos, datos clave, relaciones entre ellos, etc.
3. Realización de lo que antes se definió como minería de datos para encontrar asociaciones entre esos datos claves previamente extraídos de los textos.

2.5 Aprendizaje automático

El proceso de aprendizaje automático es similar al de la minería de datos. Ambos sistemas buscan entre los datos para encontrar patrones. Sin embargo, en lugar de extraer los datos para la comprensión humana, como es el caso de las aplicaciones de minería de datos, el aprendizaje automático utiliza los datos para detectar patrones y ajustar las acciones del programa en consecuencia.

En otras palabras, el Aprendizaje Automático (AA) es la rama de la Inteligencia Artificial que tiene como objetivo desarrollar técnicas que permitan a las computadoras aprender, utilizando evidencia conocida para poder crear una hipótesis y poder dar una respuesta a nuevas situaciones no conocidas [11]. Es decir, predecir comportamiento futuro a partir de lo que ha ocurrido en el pasado.

El aprendizaje automático tiene una amplia gama de aplicaciones, incluyendo motores de búsqueda, diagnósticos médicos, detección de fraude en el uso de tarjetas de crédito, análisis del mercado de valores, clasificación de secuencias de ADN, reconocimiento del habla y del lenguaje escrito, juegos y robótica.

Si bien el AA está diseñado para repetir procesos hasta poder prácticamente tomar decisiones por sí mismo, si sus datos de origen están sesgados o mal documentados, la máquina corre el riesgo de cometer errores o reflejar vicios no deseados, propios de los humanos que alimentaron dichos procesos.

El aprendizaje automático puede clasificarse en aprendizaje supervisado o no supervisado [12], dependiendo del tipo de salida que se produzca y de cómo se aborde el tratamiento de los ejemplos.

- Aprendizaje supervisado: se genera una función que establece una correspondencia entre las entradas y las salidas deseadas del sistema, donde la base de conocimientos está formada por ejemplos etiquetados a priori. Es decir, ejemplos de los que sabemos su clasificación correcta. Para este tipo de aprendizaje, existen

(básicamente) dos problemas a solucionar: Clasificación donde $|y| \ll \infty$. Es decir, que el número de posibles respuestas es finito. Y Regresión, en el cual $Y \in \mathbb{R}$. Es decir, la respuesta es una variable numérica con una infinidad de posibles resultados.

- Aprendizaje no supervisado: donde el proceso de modelado se lleva a cabo sobre un conjunto de ejemplos formados únicamente por entradas al sistema, sin conocer su clasificación correcta. Se busca que el sistema sea capaz de reconocer patrones para poder etiquetar las nuevas entradas.

En este trabajo de tesis, todos los algoritmos utilizados pertenecen al paradigma de aprendizaje supervisado, los cuales necesitan una etapa de aprendizaje para construir un modelo a partir de datos de entrenamiento. En particular, se utilizaron algoritmos de clasificación.

2.6 Clasificación

La clasificación automática de textos es, por lo general, un proceso supervisado [13]. Esto significa que requiere de un conjunto de documentos previamente clasificados por expertos humanos que funcionan como entrenamiento para el sistema. Así, un conjunto de documentos clasificado por un humano en cierta categoría sirven para que el clasificador automático genere una clasificación propia frente a un documento desconocido. El desempeño del clasificador automático dependerá de qué tan similar sea esta clasificación respecto a la humana, lo que se evalúa con matrices de confusión y otras métricas [14].

Bajo el marco formal de Máquinas de Aprendizaje se define como D el espacio de entrada que representa los documentos de textos y C el espacio de salida de un proceso de clasificación realizado por un humano, donde cada par $\{d_i, c_i\}$ corresponden al documento y la categoría en la que este fue clasificado, respectivamente. Asimismo, se define Z como el conjunto de todos estos pares de documentos y sus clasificaciones, $z_i = (d_i, c_i)$. Los valores de Z corresponden a los datos que sirven de ejemplo para que la máquina de aprendizaje utilice en su proceso de entrenamiento y prueba.

El principal desafío del proceso de aprendizaje automático es obtener una máquina que posea buena capacidad de generalización, es decir, que no solo aprenda a clasificar los ejemplos Z , conocidos utilizados en su proceso de entrenamiento, sino también, que sea capaz de construir un modelo general que permita clasificar bien nuevos ejemplos desconocidos. Para asegurar una buena capacidad de generalización, la Teoría del Aprendizaje Estadístico (Vapnik, 1989) define una Función Riesgo $R(g(z))$ como aquella que agrupa funciones $g(z)$ del clasificador automático representando la distancia de estas funciones respecto a las reales. Cada conjunto de funciones g opera como clasificador con otro conjunto de parámetros. Al conjunto de funciones g y sus parámetros se les denomina Funciones Admisibles, de forma que $R(g(z)) = R(\alpha)$, donde α es el conjunto de parámetros del clasificador automático. R evalúa la distancia acumulada entre las funciones del clasificador respecto a la función real subyacente, por lo que $R(\alpha) = \int Q(z, \alpha) dz$, entonces $Q(z, \alpha) = L(z, g(z, \alpha))$, siendo L la Función de Pérdida que relaciona los datos ya clasificados con la clasificación automática.

En problemas de clasificación automática, cuando $L \approx 1$ se dice que el resultado de esta clasificación no es bueno, ya que es distinto a la realidad z , usando una determinada función con ciertos parámetros, ambos pertenecientes a $g(z, \alpha)$. Por el contrario, cuando $L \approx 0$ se dice que la clasificación automática es similar a la real y , por lo tanto, el resultado es bueno [15]. Entonces, los algoritmos para entrenar las máquinas de aprendizaje buscan encontrar el conjunto de parámetros α para la función g que minimice L . Estos algoritmos pueden estar basados en procesos de optimización, o también, en heurísticas.

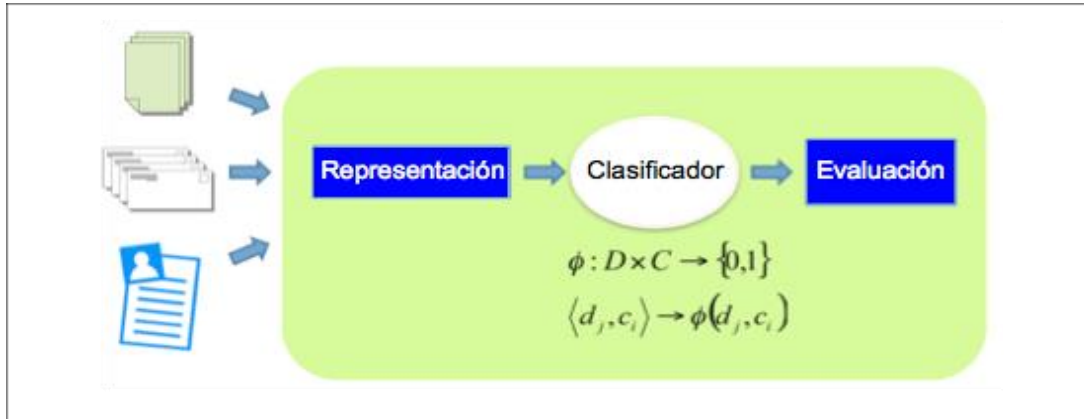


Figura 3. Esquema de la clasificación automática de textos

Como se aprecia en la figura 3, la representación del documento, que no es otra cosa que una estructuración particular de este, es la entrada que recibe el clasificador definido formalmente como una función binaria del tipo $\phi: D \times C \rightarrow \{0,1\}$. En esta función, D corresponde al conjunto de documentos y C al conjunto de categorías, de forma que si la función $\phi=1$ el documento (texto) pertenece a la categoría, mientras que si $\phi=0$, no pertenece [14].

La forma usual de medir la efectividad de un clasificador es por medio de su exactitud (α), i.e. el porcentaje de decisiones correctas. Un cálculo para la exactitud sobre la clase puede darse en términos de su tabla de contingencia de la siguiente manera:

$$\alpha_i = \frac{TP_i + TN_i}{TP_i + FP_i + TN_i + FN_i}$$

donde FP_i (falsos positivos) es el número de instancias de prueba incorrectamente clasificados bajo c_i , TN_i (verdaderos negativos) es el número de instancias de prueba correctamente clasificados bajo c_i , TP_i (verdaderos positivos) y FN_i (falsos negativos) son definidos consecuentemente.

Categoría c_i		Juicio del experto	
		SI	NO
Juicio del clasificador	SI	TP_i	FP_i
	NO	FN_i	TN_i

Figura 4. Tabla de contingencia para la clase c_i

Es importante destacar que existen otras medidas para evaluar la exactitud del clasificador. Dentro de las técnicas de clasificación podemos encontrar árboles de clasificación, máquinas de soporte vectorial (SVM), redes de bayes, k-vecinos entre otros.

2.6.1 Naive Bayes

Naive Bayes es uno de los clasificadores más utilizados por su simplicidad, aprendizaje rápido, desempeño razonable con grande cantidad de datos y capacidad de aprender en forma incremental [16].

Se trata de una técnica de clasificación y predicción supervisada que construye modelos que predicen la probabilidad de posibles resultados. Constituye una técnica supervisada porque necesita tener ejemplos clasificados para que funcione. Donde cada uno de estos ejemplo observado, va a modificar la probabilidad de que la hipótesis formulada sea correcta (aumentándola o disminuyéndola). Es decir, una hipótesis que no concuerda con un conjunto de ejemplos más o menos grande no es desechada por completo sino que lo que harán será disminuir esa probabilidad estimada para la hipótesis.

Estos métodos son robustos al posible ruido presente en los ejemplos de entrenamiento y a la posibilidad de tener entre dichos ejemplos de entrenamiento datos incompletos o posiblemente erróneos.

Los métodos bayesianos permiten tener en cuenta en la predicción de la hipótesis el conocimiento a priori o conocimiento del dominio en forma de probabilidades. El problema puede surgir al tener que estimar ese conocimiento estadístico sin disponer de datos suficientes.

Dado un conjunto de datos (divididos en dos conjuntos: de entrenamiento y de test) representados por pares $\langle \text{atributo}, \text{valor} \rangle$, el problema consiste en encontrar una función $f(x)$ (llamada hipótesis) que clasifique dichos ejemplos.

La idea de usar el teorema de Bayes en cualquier problema de aprendizaje automático, es la posibilidad de estimar las probabilidades a posteriori de cualquier hipótesis consistente con el conjunto de datos de entrenamiento para poder escoger la hipótesis más probable.

Dado un ejemplo x representado por k valores, el clasificador Naïve Bayes se basa en encontrar la hipótesis más probable que describa a ese ejemplo.

Si la descripción del ejemplo viene dada por los valores $\langle a_1, a_2, \dots, a_n \rangle$, la hipótesis más probable será aquella que cumpla:

$$V_{MAP} = \underset{v \in V}{\operatorname{argmax}} P(v_j | a_1, \dots, a_n)$$

Es decir, la probabilidad que conocidos los valores que describen a ese ejemplo, esté pertenezcan a la clase v_j (donde v_j es el valor de la función de clasificación $f(x)$ en el conjunto finito V).

Por el teorema de Bayes:

$$V_{MAP} = \underset{v \in V}{\operatorname{argmax}} \frac{P(a_1, \dots, a_n | v_j) p(v_j)}{P(a_1, \dots, a_n)}$$

$$= \underset{v \in V}{\operatorname{argmax}} P(a_1, \dots, a_n | v_j) p(v_j)$$

Se puede estimar $P(v_j)$ contando las veces que aparece el ejemplo v_j en el conjunto de entrenamiento y dividiéndolo por el número total de ejemplos que forman ese conjunto.

Para estimar el término $P(a_1, \dots, a_n | v_j)$, es decir, las veces en que para cada categoría aparecen los valores del ejemplo x , se debe recorrer todo el conjunto de entrenamiento. Siendo esto imposible.

Por lo que se recurre a la hipótesis de independencia condicional con el objeto de poder factorizar la probabilidad. Esta hipótesis dice lo siguiente:

Los valores a_j que describen un atributo de un ejemplo cualquiera x son independientes entre sí conocido el valor de la categoría a la que pertenecen. Así, la probabilidad de observar la conjunción de atributos a_j dada una categoría a la que pertenecen es el producto de las probabilidades de cada valor separado:

$$P(a_1, \dots, a_n | v_j) = \prod_i P(a_i | v_j)$$

El término $P(a_i | v_j)$ se calcula a partir del número de apariciones de cada término a_i en una clase v_j pero para evitar el problema de las probabilidades 0, se usa la estimación de Laplace:

$$P(h|D) = \frac{P(D|h) P(h)}{P(D)}$$

Donde:

- $P(h)$ es la probabilidad a priori de la hipótesis h .
- $P(D)$ es la probabilidad de observar el conjunto de entrenamiento D .
- $P(D|h)$ es la probabilidad de observar el conjunto de entrenamiento D en un universo donde se verifica la hipótesis h .
- $P(h|D)$ es la probabilidad a posteriori de h , cuando se ha observado el conjunto de entrenamiento D .

2.6.2 Máquinas de Vectores de Soporte (SVM)

Una máquina de vectores de soporte (SVM) es un algoritmo de aprendizaje supervisado que se puede emplear para clasificación binaria o regresión. Estas máquinas, pertenecen a una clase de algoritmos de Machine Learning denominados métodos kernel y también se conocen como máquinas kernel.

SVM construye un hiperplano óptimo en forma de superficie de decisión, de modo que el margen de separación entre las dos clases en los datos se amplía al máximo. Los vectores de soporte hacen referencia a un pequeño subconjunto de las observaciones de entrenamiento que se utilizan como soporte para la ubicación óptima de la superficie de decisión.

Dado un conjunto separable de ejemplos $S = \{(x_1, y_1), \dots, (x_n, y_n)\}$ donde $x_i \in R^d$ e $y_i \in \{+1, -1\}$, se puede definir un hiperplano de separación como una función lineal que es capaz de separar dicho conjunto sin error:

$$D(x) = (w_1 x_1 + \dots + w_d x_d) + b = \langle w, x \rangle + b$$

donde w y b son coeficientes reales. El hiperplano de separación cumplirá las siguientes restricciones para todo x_i del conjunto de ejemplos:

$$\begin{aligned} \langle w, x_i \rangle + b &\geq 0 & \text{si } y_i = +1 \\ \langle w, x_i \rangle + b &\leq 0 & \text{si } y_i = -1, i = 1, \dots, n \end{aligned} \quad (1)$$

o también,

$$y_i (\langle w, x_i \rangle + b) \geq 0, i = 1, \dots, n \quad (2)$$

o de forma más compacta

$$y_i D(x_i) \geq 0, i = 1, \dots, n \quad (3)$$

El hiperplano que permite separar los ejemplos no es único, es decir, existen infinitos hiperplanos separables, representados por todos aquellos hiperplanos que son capaces de cumplir las restricciones impuestas por cualquiera de las expresiones equivalentes (3-4). Surge entonces la pregunta sobre si es posible establecer algún criterio adicional que permita definir un hiperplano de separación óptimo. Para ello, primero, se define el concepto de margen de un hiperplano de separación, denotado por τ , como la mínima distancia entre dicho hiperplano y el ejemplos más cercano de cualquiera de las dos clases. A partir de esta definición, un hiperplano de separación se denominará óptimo si su margen es de tamaño máximo.

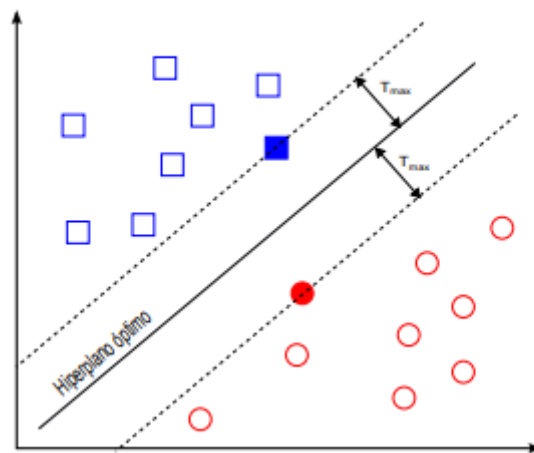


Figura 5. Hiperplano de separación óptimo con margen de tamaño máximo

Una propiedad inmediata de la definición de hiperplano de separación óptimo es que éste equidista del ejemplo más cercano de cada clase. La demostración de esta propiedad se puede hacer fácilmente por reducción al absurdo. Si suponemos que la distancia del

hiperplano óptimo al ejemplo más cercano de la clase +1 fuese menor que la correspondiente al ejemplo más cercano de la clase -1, esto significaría que se puede alejar el hiperplano del ejemplo de la clase +1 una distancia tal que la distancia del hiperplano a dicho ejemplo sea mayor que antes y, a su vez, siga siendo menor que la distancia al ejemplo más cercano de la clase -1. Se llega así al absurdo de poder aumentar el tamaño del margen cuando, de partida, habíamos supuesto que éste era máximo (hiperplano óptimo). Se aplica un razonamiento similar en el caso de suponer que la distancia del hiperplano óptimo al ejemplo más cercano de la clase -1 fuese menor que la correspondiente al ejemplo más cercano de la clase +1.

2.6.3 K-Vecinos Más Cercanos (KNN)

El clasificador k, vecinos más próximos (KNN), es un método de aprendizaje perezoso basado en ejemplos. En la clasificación basada en ejemplos cuando una nueva instancia necesita ser clasificada, se la compara con los ejemplos existentes usando una métrica de distancia y los ejemplos más próximos son utilizados para asignar la clase a la nueva instancia [17].

Para la clasificación de texto representa los documentos en forma de vector con la frecuencia de los términos existentes en el documento. Una vez generado tal vector de características para un determinado texto desconocido, el algoritmo KNN procura por todos los ejemplos de entrenamiento comparar la similitud entre sus vectores de características, para encontrar los k ejemplos de entrenamiento más próximos y el documento desconocido es designado a los k vecinos más próximos con mayor valor de clasificación [18].

La principal ventaja del algoritmo KNN es su facilidad de implementación, no necesita una etapa de aprendizaje, pero su costo computacional es alto cuando el tamaño de los documentos usados en el entrenamiento crece.

2.6.4 Árboles de Decisión ID3 y C4.5

El árbol de decisión (DT) es una estructura en árbol, donde cada nodo representa un atributo a ser probado; las ramas representan la salida de la prueba y los nodos finales (hojas) representan la clasificación.

El algoritmo de DT posee dos fases principales: en la primera llamada fase de crecimiento del árbol, el algoritmo inicia con todo el conjunto de datos como nodos raíz. Los datos son divididos en subconjuntos utilizando algún criterio de división. En la segunda fase, etapa de poda del árbol, el árbol total formado se poda para prevenir el exceso de ajuste (over-fitting) del árbol a los datos de entrenamiento [19].

Existen diversos algoritmos para construir árboles de decisión entre ellos ID3, C4.5, entre otros. El algoritmo ID3 utiliza esencialmente el criterio de ganancia de información como medida de pureza para cada nodo. El algoritmo C4.5 incluye diversos métodos para trabajar con atributos numéricos, valores ausentes, datos con ruidos y para generar reglas a partir de árboles de decisión. El algoritmo J48 es la implementación del algoritmo C4.

Una de las ventajas de los árboles de decisión es que representan el modelo aprendido con una estructura que puede ser interpretada fácilmente por los usuarios, esto facilita la realización de ajustes al árbol de decisión en caso de necesidad [20].

2.7 Procesamiento de lenguaje natural (PLN)

Una de las tareas fundamentales de la Inteligencia Artificial (IA) es la manipulación de lenguajes naturales usando herramientas de computación, en la cual los lenguajes de programación juegan un papel importante, ya que forman el enlace necesario entre los lenguajes naturales y su manipulación mediante una máquina.

El procesamiento de lenguaje natural (PLN), consiste en la utilización de un lenguaje natural para la comunicación hombre-computadora, debiendo ésta entender las oraciones que le sean proporcionadas, el uso de dichos lenguajes naturales, facilitar el desarrollo de programas que realicen tareas relacionadas con el lenguaje o bien, desarrollar modelos que ayuden a comprender los mecanismos humanos relacionados con el lenguaje. El uso del lenguaje natural (LN) en la comunicación hombre-máquina presenta una ventaja y un obstáculo con respecto a otros medios de comunicación.

Es posible dividir el estudio del lenguaje natural en cuatro niveles de análisis: morfológico, sintáctico, semántico y pragmático, pudiendo ser incluido el análisis de la fonética y fonología. Los cuatro niveles están ordenados de la forma descrita, por lo que comprenden una secuencia en la cual cada etapa toma como información de entrada la salida de la etapa anterior (con la obvia excepción de la primera etapa) [21].

- **Análisis morfológico:** Consiste en detectar la relación entre las unidades mínimas que forman una palabra (morfemas), reconocerla y construir una representación estructurada. Una palabra es la combinación de una raíz y un conjunto de morfemas, los cuales son aplicados mediante flexión, derivación y composición. La flexión es un mecanismo de producción de palabras dentro de una misma clase, la derivación es la combinación de una raíz con un afixo para generar una palabra de otra clase, o con otro significado y la composición es la unión de dos raíces.
- **Análisis sintáctico:** Tiene como cometido el etiquetado de cada componente sintáctico (sintagma) que aparece en una oración, construyendo en el proceso un árbol de análisis sintáctico, el cual tiene como nodo raíz el símbolo de oración, y tiene como hojas a las palabras.
- **Análisis semántico:** Cumple la tarea de procesar el lenguaje natural y comprender su significado. Esto permite saber si una oración es una correcta representación de la realidad, responder preguntas o saber qué se requerirá para responderlas en caso de faltar información. Permitiría que una máquina pudiera entender que "Pablo le vendió un libro a Juan" y "Juan compró un libro a Pablo" son semánticamente equivalentes, y que si una es verdadera también lo es la otra.
- **Análisis pragmático:** Se refiere al uso del lenguaje en el contexto. En general la pragmática incluye aspectos del conocimiento conceptual del mundo que van más allá de las condiciones reales literales de cada oración. Este conocimiento lo tienen en cuenta los hablantes cuando se comunican mediante una lengua. Les sirve para comprender mucha información sobreentendida pero no expresada explícitamente en las oraciones. Mientras la sintaxis y semántica estudia las oraciones, la pragmática estudia "las acciones del discurso" y las situaciones en las cuales el lenguaje es usado.

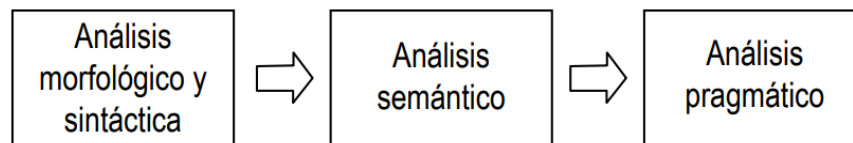


Figura 6. Arquitectura de un sistema de procesamiento de lenguaje natural

La arquitectura del sistema de procesamiento del lenguaje natural muestra el modo en que la computadora interpreta y analiza las oraciones que le son proporcionadas.

La explicación de este sistema es sencilla. En primer lugar, el usuario le indica a la computadora lo que desea hacer. Luego, la computadora analiza las oraciones proporcionadas en el sentido morfológico y sintáctico, es decir, estudia si las frases contienen palabras compuestas por morfemas y si la estructura de las oraciones es correcta. A continuación, se analizan las oraciones semánticamente, es decir, se determina el significado de cada oración, y se asigna el significado de las mismas a expresiones lógicas (cierto o falso). Una vez realizado el paso anterior, se lleva a cabo el análisis pragmático de la instrucción, es decir, una vez analizadas las oraciones individualmente, se analizan todas juntas, considerando la situación de cada oración y analizando las oraciones anteriores. Posteriormente, la computadora es capaz de determinar lo que va a hacer, es decir, ya tiene la expresión final. Una vez obtenida dicha expresión, el siguiente paso es su ejecución, para obtener finalmente un resultado y brindárselo al usuario.

2.8 Framework de procesamiento de lenguaje natural: Freeling

Freeling es una librería de código abierto para el procesamiento multilingüe, que proporciona una amplia gama de funcionalidades de análisis para varios idiomas [22][23].

El proyecto se estructura como una librería que puede ser utilizada en cualquier aplicación de usuario que requiera servicios de análisis del lenguaje. El software se distribuye como código abierto bajo una licencia GNU General Public License y bajo licencia dual a empresas que deseen incluirlo en sus productos comerciales.

Freeling está concebido como una librería sobre la cual se puedan desarrollar potentes aplicaciones de PLN, y está orientado a facilitar la integración con las aplicaciones de niveles superiores de los servicios lingüísticos que ofrece.

La arquitectura de la librería se basa en un enfoque de dos capas cliente-servidor: La primera es una capa básica de servicios de análisis lingüístico (morfológico, morfosintáctico, sintáctico, ...). La segunda es una capa de aplicación que, actuando como cliente, realiza las peticiones deseadas a los analizadores y usa su respuesta según la finalidad de la aplicación.

La arquitectura interna de la librería se estructura en dos tipos de objetos: los que almacenan datos lingüísticos con los análisis obtenidos y los que realizan el procesamiento en sí.

- Clases de almacenamiento de datos lingüísticos: Las clases básicas de la librería tienen la finalidad de contener los datos lingüísticos (palabras, etiquetas morfológicas, frases, árboles sintácticos, párrafos, etc.) resultado de los análisis realizados. Cualquier aplicación cliente debe usar dichas clases para poder proporcionar a los módulos de análisis los datos en el formato oportuno, y para poder recuperar el resultado de los analizadores. Las clases de datos lingüísticos en la versión actual son las siguientes:
 - Analysis: Una tupla <lema, etiqueta, probabilidad, lista de sentidos>.
 - Word: Forma de una palabra, con una lista de posibles objetos Analysis.
 - Sentence: Una lista de objetos Word marcada como una frase completa. Puede contener también un árbol de constituyentes o de dependencias.
 - Paragraph: Una lista de objetos Sentence marcada como un párrafo independiente.
 - Document: Una lista de objetos Paragraph que forman un documento completo. Puede contener también información sobre la correferencia entre las menciones a entidades del documento.

La figura 7 presenta un diagrama UML con las clases de datos lingüísticos.

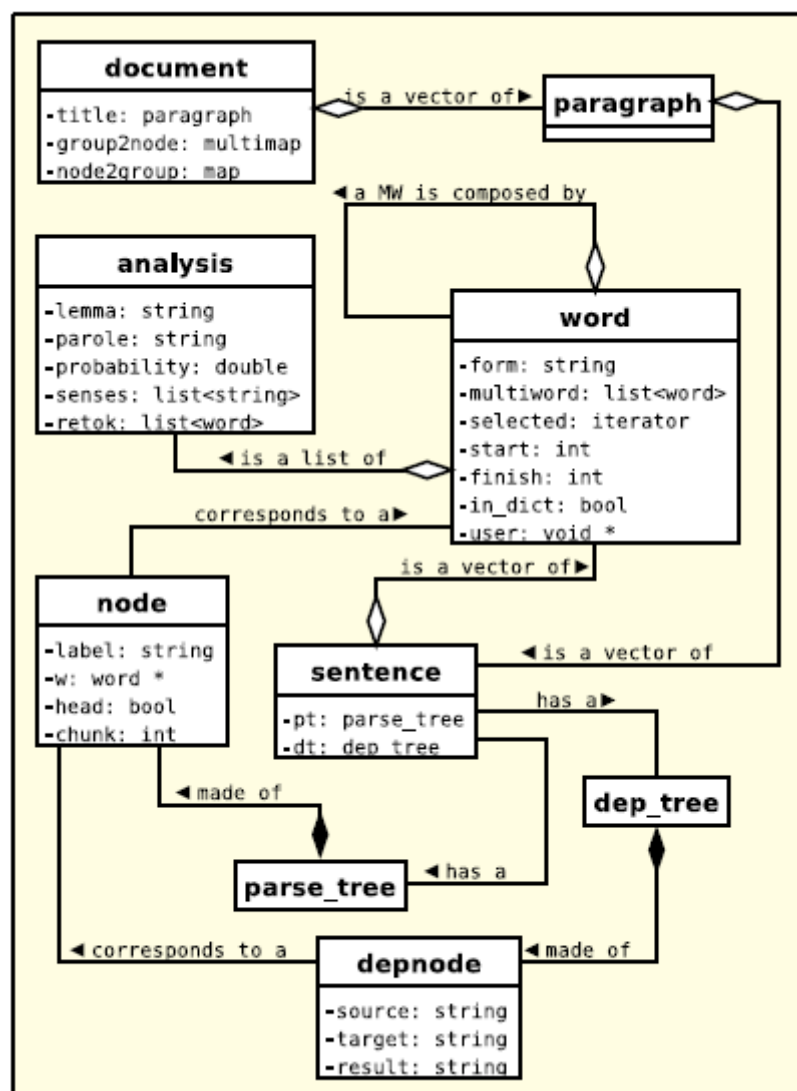


Figura 7. Clases de datos lingüísticos en Freeling 3.0

- Clases de procesamiento: Además de las clases para contener datos lingüísticos descritos anteriormente, la librería proporciona clases para transformarlos, usualmente enriqueciéndose con información adicional. Las clases de procesamiento son las siguientes:
 - Lang_ident: Identificador de idioma. Recibe texto plano y devuelve una lista de pares <idioma,probabilidad>.
 - Tokenizer: Recibe texto plano y devuelve una lista de objetos Word.
 - Splitter: Recibe una lista de objetos Word y devuelve una lista de objetos Sentence.
 - Morfo: Recibe una lista de objetos Sentence y analiza morfológicamente cada Word de cada Sentence de la lista. Esta clase es un meta-analizador que simplemente aplica una cascada de analizadores especializados (detección de números, fechas, locuciones y multi-palabras, etc.), cada uno

de los cuales es a su vez una clase de procesamiento que puede ser llamada independientemente si fuese necesario.

- **Tagger:** Recibe una lista de objetos Sentence y desambigua la categoría morfosintáctica de cada palabra en las frases de la lista. Si el análisis seleccionado incorpora información de retokenización, la palabra puede separarse en varias. Freeling ofrece dos taggers con una precisión de aproximadamente 98%: Uno basado en modelos ocultos de Markov, y otro basado en relaxation labelling que permite la combinación de información estadística con reglas manuales.
- **NEclassifier:** Recibe una lista de objetos Sentence y clasifica cada Word etiquetada como nombre propio que aparezca en las frases dadas.
- **Sense annotator:** Recibe una lista de Sentence y añade información sobre los sentidos posibles (según WordNet) a los objetos Analysis de cada palabra.
- **Word sense disambiguator:** Recibe una lista de objetos Sentence y ordena por relevancia en el contexto los posibles sentidos de cada palabra.
- **Chunkparser:** Recibe una lista de Sentence y enriquece cada una con un árbol de análisis. Este módulo consiste en un chart parser.
- **Dependency parser:** Recibe una lista de Sentence analizadas sintácticamente y las enriquece con un árbol de dependencias. Este módulo usa un conjunto de reglas escritas manualmente que operan en tres etapas: primero completan el árbol sintáctico superficial construido por el chart parser, a continuación transforman el árbol de constituyentes a dependencias, y finalmente etiquetan la función de cada dependencia.
- **Coreference solver:** Recibe un documento formado por objetos Sentence analizados sintácticamente y lo enriquece con información de correferencia.

2.9 Framework de aprendizaje automático: Weka

Weka es un conjunto de librerías JAVA para la extracción de conocimiento desde bases de datos.[35] Es un software que ha sido desarrollado en la universidad de Waikato (Nueva Zelanda) bajo licencia GPL, lo cual ha impulsado que sea una de las suites más utilizadas en el área en los últimos años. Además, contiene las herramientas necesarias para la realización de transformaciones sobre datos, tareas de clasificación, regresión, clustering, asociación y visualización. Weka está diseñado como una herramienta orientada a la extensibilidad por lo que una de las propiedades más interesantes de este software, es su facilidad para añadir extensiones, modificar métodos, etc.

Una de las ventajas que nos brinda Weka es que proporciona interfaces para la comunicación con el usuario, tal es el caso de CLI (*Simple Client*), la cual proporciona una consola para poder introducir mandatos, posee una apariencia muy simple pero brinda la posibilidad de ejecutar tareas complejas ya que permite realizar cualquier operación

soportada por Weka de forma directa; no obstante, es muy complicada de manejar dado que es necesario un conocimiento completo de la aplicación.

Adicionalmente, permite ubicar patrones de comportamiento de la información a procesar de tal manera que es de gran ayuda en la toma de decisiones.

- Está disponible libremente bajo la licencia pública general de GNU.
- Es portable, porque está completamente implementado en Java y puede correr en casi cualquier plataforma.
- Contiene una extensa colección de técnicas para pre-procesamiento de datos y modelado.

Nativamente, Weka trabaja con un formato denominado arff, acrónimo de Attribute-Relation File Format, el cual está compuesto por una estructura claramente diferenciada en tres partes:

1. Cabecera. Se define el nombre de la relación. Su formato es el siguiente:

`@relation <nombre-de-la-relación>`

Donde <nombre-de-la-relación> es de tipo String. Si dicho nombre contiene algún espacio será necesario expresarlo entrecomillado.

2. Declaraciones de atributos. En esta sección se declaran los atributos que compondrán nuestro archivo junto a su tipo. La sintaxis es la siguiente:

`@attribute <nombre-del-atributo> <tipo>`

Donde <nombre-del-atributo> es de tipo String teniendo las mismas restricciones que el caso anterior. Weka acepta diversos tipos, estos son:

- a. NUMERIC: Expresa números reales.
- b. INTEGER: Expresa números enteros.
- c. DATE: Expresa fechas. Este tipo debe ir precedido de una etiqueta de formato entrecomillada. La etiqueta de formato está compuesta por caracteres separadores (guiones y/o espacios) y unidades de tiempo:

- dd: Día.
- MM: Mes.
- yyyy: Año.
- HH: Horas.
- mm: Minutos.
- ss: Segundos.

- d. STRING: Expresa cadenas de texto, con las restricciones del tipo String comentadas anteriormente.
- e. ENUMERADO: El identificador de este tipo consiste en expresar entre llaves y separados por comas los posibles valores (caracteres o cadenas de caracteres) que puede tomar el atributo. Por ejemplo, si tenemos un atributo que indica el tiempo podría definirse:

`@attribute tiempo {soleado,lluvioso,nublado}`

3. Sección de datos. Aquí se declaran los datos que componen la relación separando entre comas los atributos y con saltos de línea las relaciones.
@data 4,3,2

Aunque éste es el modo "completo", es posible definir los datos de una forma abreviada (sparse data). Si tenemos una muestra en la que hay muchos datos que sean 0 podemos expresar los datos prescindiendo de los elementos que son nulos, rodeando cada una de las filas entre llaves y situando delante de cada uno de los datos el número de atributo.

En el caso de que algún dato sea desconocido, se expresa con un símbolo de cerrar interrogación ("?"). Es posible añadir comentarios con el símbolo porcentual ("%"), el cual indica que a partir de ese símbolo hasta el final de la línea es todo un comentario. Los comentarios pueden situarse en cualquier lugar del fichero. Un ejemplo de un archivo de prueba del fichero arff puede observarse en la figura 8.

```
1  % Archivo de prueba para Weka.
2  @relation prueba
3
4  @attribute nombre STRING
5  @attribute ojo_izquierdo {Bien,Mal}
6  @attribute dimension NUMERIC
7  @attribute fecha_analisis DATE "dd-MM-yyyy HH:mm"
8
9  @data
10 Antonio,Bien,38.43,"12-04-2003 12:23"
11 'Maria Jose',?,34.53,"14-05-2003 13:45"
12 Juan,Bien,43,"01-01-2004 08:04"
13 Maria,?,?, "03-04-2003 11:03"
```

Figura 8. Estructura de archivo ARFF

2.10 Ontologías para el procesamiento de lenguaje natural

Dentro del campo de análisis de las ciencias de la computación, una ontología puede verse como un vocabulario de representación para un dominio específico, que representa elementos conceptuales y relaciones entre ellos; sin embargo, la ontología no es el vocabulario en sí mismo, sino lo que él representa ya que, por ejemplo, si se traduce cada vocablo a otro idioma no significa que cambie la ontología [24].

Algunos autores postulan la necesidad de que una ontología sea una "teoría", es decir, un vocabulario formal con un conjunto de axiomas definido para ese vocabulario, pues los axiomas expresan nuevas relaciones entre conceptos y restringen las posibles interpretaciones [25][26].

Las ontologías están formadas por componentes que sirven para representar el conocimiento de algún dominio. Estos se definen de la siguiente manera:

- Conceptos: son las ideas básicas que se intentan formalizar. Pueden ser clases de objetos, métodos, planes, estrategias, procesos de razonamiento, etc.

- Relaciones: representan la interacción y enlace entre los conceptos del dominio. Suelen formar la taxonomía del dominio. Por ejemplo: subclase-de, parte-de, parte-exhaustiva-de, conectado-a, etc.
- Funciones: son un tipo concreto de relación donde se identifica un elemento mediante el cálculo de una función que considera varios elementos de la ontología. Por ejemplo, pueden aparecer funciones como categorizar-clase, asignar fecha, etc.
- Instancias: se utilizan para representar objetos determinados de un concepto.
- Axiomas: son teoremas que se declaran sobre relaciones que deben cumplir los elementos de la ontología. Por ejemplo: "Si A y B son de la clase C, entonces A no es subclase de B", "Para todo A que cumpla la condición C1, A es B", etc.

En la figura 9 se puede observar un ejemplo de cómo se relacionan los elementos de un dominio dado.

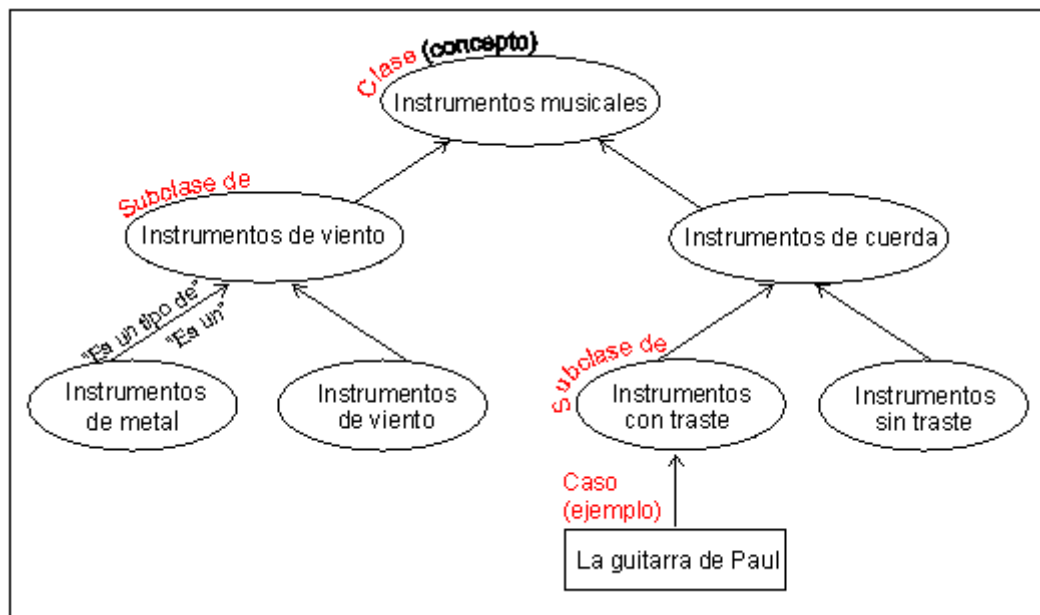


Figura 9. Ejemplo de ontología

2.10.1 Wordnet

La ontología Wordnet fue creada por el Cognitive Science Laboratory de la Universidad de Princeton, Wordnet es una base de datos conceptual de lengua inglesa estructurada en forma de red semántica y se la clasifica dentro del grupo de las ontologías léxico-conceptuales. Su diseño se fundamenta en teorías psicolingüísticas sobre la memoria léxica humana basada en redes semánticas [27]. Wordnet constituye un intento de reflejar este modelo de memoria léxica en un modelo lexicográfico de organización léxica.

Wordnet contiene 126.000 entradas divididas en cinco categorías: nombres, verbos, adjetivos, adverbios y elementos funcionales. A pesar de no incluir definiciones semánticas formales, engloba 100.000 significados (conceptos) organizados en una taxonomía.

La ontología está organizada en base a relaciones semánticas. Puesto que las mismas son relaciones de significados, y los significados están representados por medio de

"synsets", Wordnet expresa las relaciones semánticas como punteros (*pointers*) entre "synsets". Un synset es un conjunto de palabras de la misma categoría morfosintáctica que pueden ser intercambiados en un contexto dado. Por ejemplo, {student, pupil, educatee} forman un synset porque pueden ser utilizados para referirse al mismo concepto. Un synset es comúnmente descrito por una gloss o definición que, en el caso del synset anterior, es "a learner who is enrolled in an educational institution", y además, por un conjunto explícito de relaciones semánticas con otros synsets. Cada synset representa un concepto que está relacionado con otros conceptos mediante una gran variedad de relaciones semánticas, incluyendo hiperonimia/hiponimia, meronimia/holonimia, antonimia, etc. Wordnet codifica 26 tipos diferentes de relaciones semánticas.

La estructura se compone de las siguientes categorías sintácticas:

1. Sustantivos: No están estructurados en torno a una jerarquía única que contenga un término superordinado general del tipo {*entity*} que englobe a todos los demás. Al contrario que en otras jerarquías, los sustantivos se han agrupado en torno a un conjunto de "primitivos semánticos", un grupo de conceptos genéricos, de forma que cada uno de ellos es el término superior de una jerarquía separada. Estas jerarquías se corresponden, según sus autores, con campos léxicos relativamente bien definidos, cada uno de los cuales cuenta con su propio vocabulario. Gran parte de la estructuración de los sustantivos de Wordnet se ha generado por medio de las relaciones de hiponimia, aunque también se incluyen detalles sobre los rasgos que distinguen un concepto de otro.
2. Adjetivos: Se dividen en dos clases principales: descriptivos y relacionales. Wordnet contiene aproximadamente 19.000 formas adjetivales, organizadas en unos 10.000 synsets o significados léxicos. Además de adjetivos relacionales y descriptivos, también contiene un grupo cerrado de adjetivos como *former* o *alleged*, que se consideran como adjetivos de modificación de referencia (*reference-modifying adjectives*).
 - a. Los adjetivos descriptivos, son aquellos que adscriben a los sustantivos valores de atributos bipolares, y por tanto están organizados en base a oposiciones binarias (antonimia) o similitud de significado (sinonimia) (Fellbaum et al. 1993). Se considera que aquellos adjetivos que no poseen antónimos directos tienen antónimos indirectos en virtud a su similitud semántica con otros adjetivos que sí poseen antónimos directos. Wordnet contiene punteros entre los adjetivos que expresan el valor de un atributo y el synset de sustantivos que hace referencia al atributo en cuestión.
 - b. Los adjetivos relacionales, son aquéllos que significan algo parecido a "relativo a/asociado con" el sustantivo al que modifican. Se consideran como variantes estilísticas de modificadores nominales, como por ejemplo en el caso de *dental hygiene*, donde el adjetivo *dental* está asociado al sustantivo *tooth* o en casos donde tanto un adjetivo como un modificador nominal es aceptable como en *atomic bomb* y *atom bomb*. En Wordnet, estos adjetivos relacionales tienen adscritos punteros que hacen referencia a los sustantivos con los que están relacionados. Por ejemplo, la entrada {*stellar*, *astral*,

sidereal, noun.object:star} indica que *stellar, astral, sidereal* están relacionados con el nombre *star*.

La antonimia es la relación semántica básica de los adjetivos descriptivos. Esta relación está estructurada de forma que la función de estos adjetivos es expresar los valores opuestos de atributos que en la mayoría de los casos son bipolares.

3. Verbos: Wordnet contiene más de 21.000 verbos (formas verbales) y aproximadamente 8.400 significados léxicos ("synsets"). Las relaciones semánticas que se habían usado en la construcción de las redes de sustantivos y adjetivos no eran válidas a la hora de organizar los verbos ingleses, ya que la naturaleza de las relaciones semánticas existentes entre verbos difieren las que han observado entre las otras categorías. Los verbos están divididos en 15 archivos diferentes, en base a criterios semánticos. Esos ficheros se corresponden con dominios semánticos, como por ejemplo *verbs of bodily care and functions, change, cognition, communication, competition, consumption, contact, creation, emotion, etc.* Todos estos verbos denotan acciones o eventos, a excepción de un archivo que contiene verbos que se refieren a estados como por ejemplo *suffice, belong* o *resemble*, aunque éstos no forman un dominio semántico ni comparten otra propiedad semántica que no sea la de referirse a estados. La relación semántica de hiponimia considerada entre sustantivos, se denomina en el caso de los verbos "troponimia" (*troponymy*), ya que se considera que las distinciones de "modo" son las más importantes a la hora de diferenciar un hipónimo verbal de su hiperónimo. La relación de troponimia entre dos verbos se expresa mediante la fórmula *To V₁ is to V₂ in some particular manner*. La troponimia se considera como un tipo de implicación léxica, ya que cada topónimo V₁ de un verbo más general V₂, también implica V₂.

Wordnet puede ser interpretado y usado como una ontología dado que la relación hiperónimo/hipónimo entre los *synsets* puede ser interpretada como una relación de especialización entre categorías conceptuales.

En la figura 10 se puede observar un ejemplo de cómo se relacionan diferentes synset.

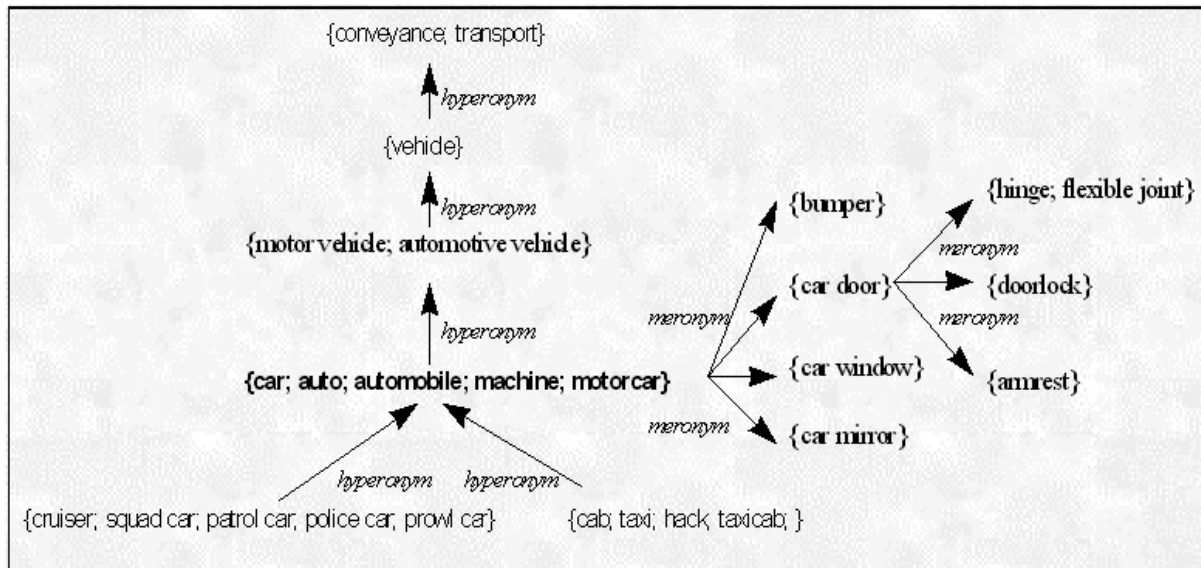


Figura 10. Ejemplo de relación entre synsets

2.10.2 Multilingual Central Repository (MCR)

Multilingual Central Repository, es una expansión multilingüe de la ontología Wordnet. Wordnet se caracteriza por ser la base de conocimiento léxica más grande del mundo, sin embargo las definiciones, tags y relaciones semánticas están estructuradas para el idioma inglés, es por esto que se considera el uso de Multilingual Central Repository, ya que contiene la traducción al español de gran cantidad de definiciones y relaciones semánticas de Wordnet (entre otros idiomas).

Esta arquitectura hace posible desarrollar wordnets locales de forma relativamente independiente, garantizando al mismo tiempo un alto nivel de compatibilidad. MCR permite transportar el conocimiento de un wordnet al resto de wordnets a través del ILI (Inter-Lingual Index), manteniendo la compatibilidad entre todos ellos.

Este repositorio multilingüe está estructurado con los siguientes componentes:

- Inter-Lingual Index (abreviado como ILI). Es el medio por el cual los wordnets de los distintos idiomas se conectan. Estas conexiones permiten acceder a palabras similares en cualquiera de los idiomas integrados en el arquitectura Eurowordnet. Además, el ILI da acceso a una ontología lingüística compuesta por 63 relaciones semánticas distintas. Esta ontología proporciona una categorización común para todos los idiomas, mientras que las distinciones específicas de cada idioma están en cada uno de los wordnets locales.
- Top Ontology. Para maximizar un desarrollo uniforme y consistente de los wordnets, el proyecto Eurowordnet categorizó los Base Concepts (bloques fundamentales para el establecimiento de relaciones en un wordnet y dar información acerca de los patrones dominantes de lexicalización en los idiomas) usando la top ontology, que fue diseñado específicamente para este propósito. Dicha top ontology [28] está basada en clasificaciones lingüísticas ya existentes y adaptada para representar la

diversidad de los base concepts. El primer nivel de la top ontology está dividido en tres tipos:

- 1st Order Entity, corresponde a objetos y sustancias concretas y perceptibles.
- 2nd Order Entity, estados, situaciones y eventos.
- 3rd Order Entity, entidades mentales como las ideas, conceptos y conocimientos.

Así, la top ontology de Eurowordnet está organizada mediante 63 características que pueden ser combinadas.

- Wordnet Domains. Es un recurso léxico que se incorporó debido al nivel de granularidad que tenía WordNet [29]. Fue desarrollado en el ITC-IRST [30] donde los synsets han sido anotados de un modo semi-automático con una o más etiquetas de dominio, escogidas de un conjunto de 165 etiquetas organizadas jerárquicamente. Los usos de WND incluyen el poder de reducir el nivel de polisemia de las palabras y agrupar aquellos sentidos que pertenecen al mismo dominio.
- SUMO y AdimenSUMO. [31] fue creado por el IEEE Standard Upper Ontology Working Group. Su objetivo fue desarrollar una ontología estándar de alto nivel para promover el intercambio de datos, la búsqueda y extracción de información, la inferencia automática y el procesamiento del lenguaje natural. SUMO provee definiciones para términos de propósito general resultantes de fusionar diferentes ontologías libres de alto nivel. Está organizada en tres niveles; la parte superior y la parte central consisten en aproximadamente 1.000 términos y 4.000 axiomas, dependiendo de la versión. El tercer nivel contiene ontologías de dominio. En total, cuando todas las ontologías de dominio son combinadas, SUMO consiste en aproximadamente 20.000 términos y cerca de 70.000 axiomas. AdimenSUMO [32] es una reconversión de SUMO a una ontología de primera orden operativa. Así, AdimenSUMO puede ser utilizada para el razonamiento formal por demostradores de teoremas de lógicas de primer orden (como E-prover o Vampire). Al estar también enlazado a Wordnet, AdimenSUMO se convierte en una herramienta muy potente para realizar razonamiento avanzado.

De esta forma, la estructura del ILI (incluyendo la top ontology, wordnet domains y la ontología SUMO) actúa como la columna vertebral que permite transferir el conocimiento adquirido de cada uno de los wordnets locales al resto. Del mismo modo, los diferentes recursos (ej. las diferentes ontologías) están relacionados mediante el ILI y, en consecuencia, también pueden ser validados entre ellos.

MCR sólo incluye conocimiento conceptual. Esto significa que tan solo las relaciones semánticas entre synsets pueden ser integradas y transportadas entre los diferentes wordnets. Aun así, cuando sea necesario, las relaciones adquiridas pueden mantenerse sub-especificadas. En ese sentido, pueden integrarse y transportarse a otros idiomas o procesos. Por ejemplo, la relación <gain> involved <money> capturada como un objeto-directo típico, más tarde puede detallarse como <gain> involved-patient <money> y ser portada al wordnet en castellano como <ganar> involved-patient <dinero>.

Además MCR utiliza la interfaz Web EuroWordnet Interface (WEI) [33], la cual permite navegar, consultar y editar la información asociada a un ítem (que puede ser un synset, una palabra, un variant o un ILI) de uno de los wordnets integrados en el MCR. La aplicación WEI consulta la información correspondiente a un ítem y la información ontológica asociada a los ILIs correspondientes.

Actualmente, el MCR está almacenado en una base de datos relacional consistente de 40 tablas. Este apartado describe cada una de las tablas necesarias para que el MCR funcione correctamente.

La tabla principal del MCR es la tabla que contiene el Inter-Lingual Index (ILI):

- **wei_ili_record:** Contiene los identificadores de los ILI, en formato ili-30-xxxxxxx-y (las x indican el offset del synset, y las y representan la categoría gramatical o Part-Of-Speech (PoS). Cada registro también almacena el origen del ILI (el WordNet del que proviene) si es un base concept o no, el fichero lexicográfico, y si es una instancia o no.

Cada uno de los idiomas incluidos en el MCR (incluyendo el inglés) está ligado al ILI, y compuesto por cinco tablas. Cada idioma tiene su propio código de 3 letras, que está indicado por xxx en la lista siguiente:

- **wei_xxx-30_to_ili:** Esta tabla conecta el ILI (por ejemplo, ili-30-00001740-a) con el número de synset (por ejemplo, eng-30-00001740-a).
- **wei_xxx-30_relation:** Esta tabla contiene todas las relaciones de wordnet. Cada registro (que es la instancia de una relación) guarda un código que indica el tipo de relación (que se almacena de la tabla wei relations), la dirección de la relación (synset origen y synset destino), el valor de confianza y el wordnet del que proviene.
- **wei_xxx-30_synset:** Aquí se almacena la información acerca del synset: el identificador, el número de descendientes, la glosa, el nivel en el que se encuentre (contando desde arriba), y finalmente una marca (opcional) y un comentario del synset (opcional).
- **wei_xxx-30_variant:** Aquí se almacenan todos los variant del wordnet. Cada registro representa un único variant y contiene la siguiente información: el variant, el sentido de la palabra, el identificador de synset, un valor de confianza, el experimento del que proviene (opcional), y finalmente la marca (opcional) y el comentario del variant (opcional).
- **wei xxx-30 examples:** En esta tabla se listan todos los ejemplos del wordnet. Cada ejemplo está identificado por el número de synset, la palabra y el sentido.

Además de los wordnets, el MCR integra otros recursos (dominios, ontologías, marcas, etc.). Las tablas que contienen esta información son las siguientes:

- **Dominios:**
 - **wei_domains:** Esta tabla representa la jerarquía de wordnet domains usando tuplas origen-destino.
 - **wei_ili_to_domains:** Cada registro enlaza un dominio a un ILI. También se indica el wordnet del que proviene. Esta tabla es única, haciendo que la información de dominios esté compartida entre todos los wordnets.

- AdimenSUMO:
 - `wei_sumo_relations`: Esta tabla representa la jerarquía de AdimenSUMO usando tuplas origen-destino. También incluye un campo que indica si se trata de una sub-clase o no.
 - `wei_ili_to_sumo`: Cada registro enlaza una etiqueta de AdimenSUMO a un ILI. También se indica el wordnet del que proviene. Al igual que la tabla de dominios, esta tabla es única, haciendo que la información de dominios esté compartida entre todos los wordnets.
- Top Ontology:
 - `wei_to_relations`: Esta tabla representa la jerarquía de top ontology usando tuplas origen-destino. También incluye un campo que indica el tipo de la relación.
 - `wei_ili_to_to`: Cada registro enlaza una etiqueta de top ontology a un ILI. También se indica el wordnet del que proviene. Al igual que la tabla de dominios y la de AdimenSUMO, esta tabla es única, haciendo que la información de dominios esté compartida entre todos los idiomas.
 - `wei_to_record`: Esta tabla almacena, para cada etiqueta de top ontology, la glosa asociada a ella.
- Marcas:
 - `mark_values_synset`: Valores permitidos para las marcas de synset, así como su descripción.
 - `mark_values_variant`: Valores permitidos para las marcas de variant, así como su descripción.

El resto de tablas incluidas en el MCR son las siguientes:

- `wei_relations`: Esta tabla contiene todas las relaciones posibles en el MCR. Cada relación tiene un identificador, un nombre, sus propiedades y una nota (opcional). También se indica si es inversa (en caso de que sea posible) y a qué grupo de relaciones pertenece. El código ID que aparece en esta tabla es el que está reflejado en la tabla `wei_xxx-30_relation`. Esta tabla es la que permite realizar búsquedas mediante el WEI.
- `wei_relations_group`: Aquí se almacenan los super-grupos de relaciones (sinónimos, hiperónimos, merónimos, causa, etc.). El código ID que aparece en esta tabla es el que está reflejado en la tabla `wei_relations`.
- `wei_languages`: Los wordnets disponibles en el MCR. Para cada wordnet se indica el código, el nombre y el color con el que debe de aparecer en el WEI.
- `wei_lexnames`: Aquí se almacenan los ficheros lexicográficos de Wordnet. Cada entrada tiene un código (el indicado en la tabla `wei_ili_record`) y un nombre descriptivo.
- `wei_wn_counters`: La interfaz del WEI permite la creación de nuevos synset. Para evitar solapamientos y problemas futuros, cada PoS tiene su propio número de offset, empezando desde el 800.000. Esta tabla guarda los números que deben adoptar los nuevos synsets que se creen en cada categoría gramatical.

En la figura 11 se puede observar la estructura completa del MCR y WEI.

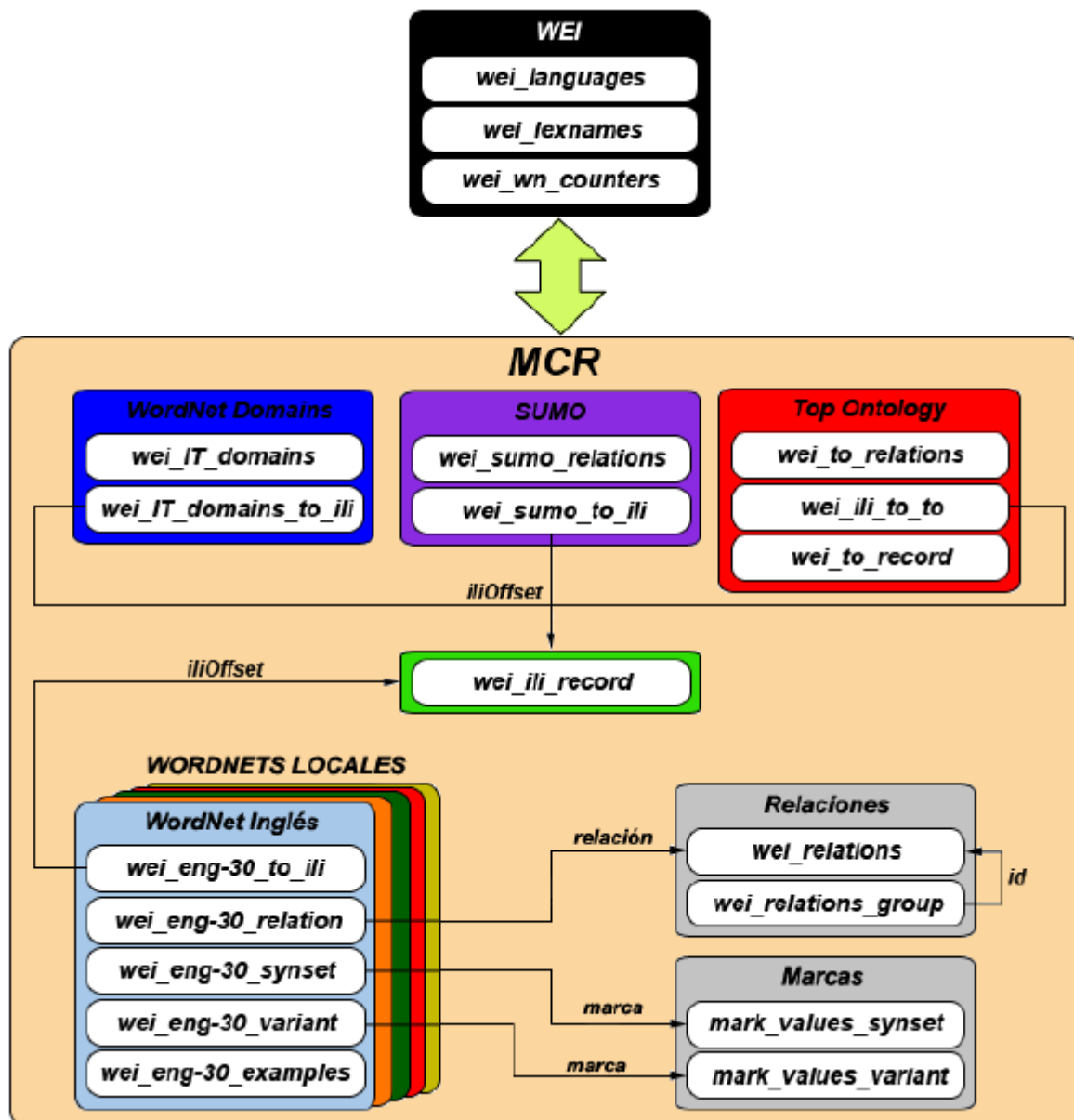


Figura 11. Estructura de la base de datos para MCR y WEI

2.11 Resumen

En este capítulo se explicaron los conceptos teóricos básicos para poder comprender el desarrollo del presente trabajo.

En primer lugar se definió el argumento, estructura en la cual está centrado el trabajo de tesis. Este puede definirse como una serie de aseveraciones seguidas de una conclusión, la cual tiene como objetivo apoyar, demostrar o probar una idea.

Luego, se analizaron diferentes mecanismos a utilizar para seguir un línea evolutiva en el desarrollo del trabajo actual, los mismos se mencionan a continuación.

KDD es un proceso iterativo que consiste en extraer conocimiento a partir de grandes volúmenes de datos para luego ser analizados. Puede ser aplicado en diferentes dominios.

La minería de datos, es una etapa dentro del proceso de KDD. Consiste en un conjunto de técnicas y tecnologías que permiten explorar grandes bases de datos, de manera automática o semiautomática, con el objetivo de encontrar patrones repetitivos, tendencias o reglas que expliquen el comportamiento de los datos en un determinado contexto.

La minería de texto, puede definirse como una forma específica de minería de datos debido a que en esta última se obtiene la información de base de datos en la que la información está estructurada. En cambio, la minería de texto obtiene la información y conocimiento a partir de un conjunto de datos que, en principio, no tienen una estructura determinada.

El aprendizaje automático es la rama de la inteligencia artificial que tiene como objetivo desarrollar técnicas que permitan a las computadoras aprender utilizando evidencia conocida para poder crear una hipótesis y poder dar una respuesta a nuevas situaciones no conocidas, es decir, analizar datos con el objetivo de encontrar y determinar patrones de manera de que se pueda predecir comportamiento futuro a partir de lo que ha ocurrido en el pasado.

De acuerdo al tipo de datos de entrada con los que se desarrolla el trabajo de tesis, se optó por utilizar el aprendizaje supervisado, el cual necesita una etapa de aprendizaje para construir un modelo a partir de datos de entrenamiento donde la base de conocimientos está formada por ejemplos etiquetados a priori. En particular, se utilizan algoritmos de clasificación.

La clasificación automática de textos es, por lo general, un proceso supervisado. Esto significa que requiere de un conjunto de documentos previamente clasificados por expertos humanos que funcionan como entrenamiento para el sistema.

Dentro de las técnicas de clasificación se pueden mencionar redes de bayes, máquinas de soporte vectorial (SVM), k-vecinos, árboles de clasificación ID3 y C4.5, entre otros. Weka contiene las herramientas necesarias para realizar transformaciones sobre los datos, tareas de clasificación, regresión, clustering, asociación y visualización. Todas estas técnicas son utilizadas por el framework de aprendizaje automático Weka para realizar tareas de clasificación, entre otras. Dicho framework contiene un conjunto de librerías JAVA que utiliza para la extracción de conocimientos desde bases de datos.

El procesamiento de lenguaje natural (PLN) consiste en la utilización de un lenguaje natural para la comunicación hombre-computadora, debiendo ésta entender las oraciones que le sean proporcionadas. PLN está dividido en cuatro niveles de análisis: morfológico, sintáctico, semántico y pragmático. Los cuatro niveles están ordenados de la forma descrita, por lo que comprenden una secuencia en la cual cada etapa toma como información de entrada la salida de la etapa anterior.

Para el presente trabajo se utiliza la herramienta de procesamiento de lenguaje natural Freeling. La misma es una librería de código abierto para el procesamiento

multilingüe que proporciona una amplia gama de funcionalidades de análisis para varios idiomas con el objetivo de brindar soporte a los análisis mencionados anteriormente.

Con el objetivo de realizar un análisis sobre el dominio semántico de los argumentos se hace uso de ontologías para el procesamiento de lenguaje natural. La misma puede verse como un vocabulario de representación para un dominio específico, que representa elementos conceptuales y relaciones entre ellos.

La ontología Wordnet es una base de datos conceptual de lengua inglesa estructurada en forma de red semántica y se la clasifica dentro del grupo de las ontologías léxico-conceptuales. Como se explicó anteriormente, Wordnet está compuesta por sustantivos, adjetivos y verbos.

Multilingual Central Repository (MCR) es una expansión multilingüe de la ontología Wordnet la cual contiene la traducción al español de gran cantidad de definiciones y relaciones semánticas de Wordnet (entre otros idiomas). Esta arquitectura hace posible desarrollar wordnets locales de forma relativamente independiente, garantizando al mismo tiempo un alto nivel de compatibilidad.

En el capítulo siguiente se hace mención de los trabajos relacionados a nuestro estudio, los cuales fueron considerados como punto de partida con el objetivo de ampliar y profundizar el análisis para lograr optimizar los resultados.

Capítulo 3

3. Trabajos Relacionados

En este capítulo se hará referencia a diferentes trabajos relacionados a nuestro estudio, los cuales fueron considerados y analizados con el objetivo de afianzar conceptos y evaluar diferentes alternativas a la hora de tomar decisiones.

El primer trabajo relacionado fue el punto de partida para el análisis de sentencias. Dicho trabajo se enfoca en la estructura de un argumento, y en la determinación de si un texto es considerado argumento o no. En el presente trabajo se profundizó el análisis con el objetivo de detectar las causas por las cuales un texto no es un argumento, considerando fallas estructurales y extendiendo dicho análisis en la detección de fallas semánticas.

Los primeros tres trabajos citados están relacionados a la detección de errores estructurales en sentencias, mientras que el último se vincula al análisis semántico, considerando un enfoque diferente al estudiado en este trabajo de tesis. Este enfoque podría ser considerado como una nueva alternativa de análisis en nuestro desarrollo ya que implementa una forma diferente de encontrar relaciones entre dos palabras basándose en procesos estadísticos para extraer y representar los significados de las palabras.

3.1 Minería de argumentos en ambientes de aprendizaje colaborativo soportado por computadoras

Determinar automáticamente si una sentencia expresada en lenguaje natural es un argumento bien conformado es una tarea clave en sistemas de aprendizaje colaborativo, especialmente cuando se pretende analizar la calidad de los argumentos elaborados por los estudiantes.

Este trabajo presenta un enfoque de minería de argumentos que se basa en el procesamiento del lenguaje de una sentencia, particularmente mediante un análisis de su estructura sintáctica en donde se aplica un enfoque basado en conectores argumentativos con etiquetas diferenciadas para indicadores de premisas y conclusiones, y el posterior entrenamiento de un modelo de clasificación.

3.1.1 Introducción

La primera tarea de dicho estudio es llevada a cabo por un módulo de procesamiento de lenguaje natural el cual permite etiquetar los textos de entrada con el fin de centrar el análisis en la estructura sintáctica de los argumentos, sin que el aspecto semántico sea influyente. Con esto se determina la función de cada palabra o frase dentro de las sentencias (verbo, sustantivo, adverbio, etc.) y se logra independizar el modelo del dominio en el cual se está trabajando. En este punto, se proponen dos tipos de procesamiento: enfoque basado en longitud de etiqueta y enfoque basado en conectores

argumentativos. La primera trata a todas las palabras o frases por igual y varía la cantidad de información sintáctica almacenada en cada etiqueta. La segunda, en cambio, procesa de forma particular aquellas palabras o frases fuertemente identificadas con la estructura de los argumentos. Una vez realizado este procesamiento, un clasificador es utilizado para determinar si el texto procesado es un argumento o no.

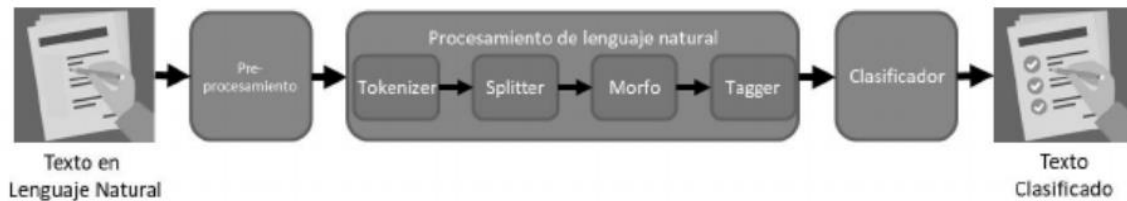


Figura 12. Enfoque de minería de argumentos propuesto por dicho trabajo

3.1.2 Enfoque de minería de argumentos

Como se observa en la figura 11, el enfoque toma como punto de partida texto en lenguaje natural y determina si dicho texto representa o no un argumento. Para realizar esta clasificación, el enfoque está compuesto por tres componentes principales: un módulo de pre-procesamiento, un módulo de procesamiento de lenguaje natural, y un módulo de clasificación.

1. Pre-procesamiento

El análisis de argumentos procederá de foros de discusión como de chats o documentos online, y en estos ámbitos predomina el lenguaje informal, en donde aparecen abreviación y estructuras lingüísticas poco claras. Es por esto que se decidió aplicar un pre-procesamiento de los datos de entrada para evitar el “ruido” generado por estas variantes.

Este pre-procesamiento consta de dos pasos principales:

- a. Cada argumento debe estar contenido en un único párrafo de manera que, si un argumento candidato contiene un salto de línea, este es eliminado.
- b. Módulo de corrección ortográfica. Tiene como objetivo reducir el ruido que pudiera generar la presencia de errores ortográficos.

2. Procesamiento de lenguaje natural (PLN)

Es un proceso modo pipeline, compuesto por diversos módulos con funciones específicas, que tienen una entrada y una salida previamente definidas. Cada módulo tiene como entrada la información procesada por un módulo anterior, y como resultado se obtiene información enriquecida útil para los módulos que continúan en el pipeline. La herramienta seleccionada para llevar a cabo esta tarea fue Freeling. En la figura 11 se detallan los sub-módulos que componen el pipeline comprendido en el módulo de procesamiento de lenguaje natural: tokenizer, splitter, morfo y tagger.

2.1 Enfoque basado en longitud de etiquetas

Este enfoque utiliza las etiquetas definidas por EAGLES y adoptadas por Freeling, las cuales comienzan con una clasificación genérica en el primer carácter que define qué tipo de palabra es (por ejemplo, adjetivo, adverbio, verbo, sustantivo, etc.), y las restantes denotan características tales como género, número, entre otros.

FORMA	LEMA	ETIQUETA	FORMA	LEMA	ETIQUETA
chico	chico	NCMS000	oyente	oyente	NCCS000
chicas	chico	NCFP000	oyentes	oyente	NCCP000
gatito	gato	NCMS00D	Barcelona	barcelona	NP000G0
tesis	tesis	NCFN000	Pedro	pedro	NP000P0

Figura 13. Ejemplos de etiquetas

En la figura 13 se puede visualizar que para las palabras “chico” y “chicas”, la etiqueta difiere aun cuando el lema es el mismo. Se puede ver que los primeros dos caracteres coinciden y se diferencian al momento de indicar género y número en la tercera y cuarta posición respectivamente. Por este motivo se pensó que un sustantivo sea singular o plural es irrelevante al momento de realizar la clasificación.

2.2 Enfoque basado en conectores argumentativos

El objetivo de este enfoque consiste en diferenciar aquellas palabras o frases que indican la presencia de componentes argumentativos. De esta forma algunas palabras o frases normalmente sirven para introducir la conclusión de un argumento y son, por lo tanto, indicadores de conclusión. Sucede lo mismo con los indicadores de premisas, de manera que existen palabras o frases que sirven para señalar las premisas de un argumento y, por lo tanto, se llaman indicadores de premisas.

Los argumentos con indicadores de premisas tienen, por ejemplo, la siguiente estructura: [conclusión], [indicador de premisa], [premisa]. Algunos posibles indicadores de premisas se muestran en la figura 14.

FORMA	LEMA	ETIQUETA	FORMA	LEMA	ETIQUETA
chico	chico	NCMS000	oyente	oyente	NCCS000
chicas	chico	NCFP000	oyentes	oyente	NCCP000
gatito	gato	NCMS00D	Barcelona	barcelona	NP000G0
tesis	tesis	NCFN000	Pedro	pedro	NP000P0

Figura 14. Lista de indicadores de premisa

Los argumentos con indicadores de conclusión tienen, por ejemplo, la siguiente estructura: [premisas], [indicador de conclusión], [conclusión]. En la figura 15 se muestra una lista parcial de los posibles indicadores de conclusión.

por lo tanto	por estas razones	por consiguiente	lo que quiere decir que	consecuentemente	por esta razón	de este modo
se sigue que	podemos inferir que	concluyo que	lo que lleva a la conclusión	lo que implica que	como resultado	así que
de ahí que	lo que muestra que	en consecuencia	lo que permite inferir que	lo que conlleva a	prueba que	luego

Figura 15. Lista de indicadores de conclusión

Definidos los posibles conectores argumentativos tanto para premisas como para conclusiones, se presentan a continuación diversas transformaciones aplicadas a los mismos:

- Conectores argumentativos sin etiquetar: Se toma la lista de conectores y se exceptúa su etiquetado.
- Conectores argumentativos con etiqueta unificada: como en el ítem anterior, se tienen en cuenta la totalidad de los conectores, sin discriminar entre los orientados a preceder premisas o conclusiones. Estos conectores fueron etiquetados con una única etiqueta especial.
- Conectores argumentativos con etiqueta diferenciada: en esta transformación se etiqueta los conectores distinguiendo entre los indicadores de premisas y los indicadores de conclusión. Siempre tomando como base la lista de conectores mencionada previamente, se rotuló con las etiquetas “tag-pre” y “tag-con” aquellos indicadores de premisa y conclusión, respectivamente.

3. Construcción del clasificador

El objetivo de esta etapa consiste en encontrar el modelo que maximice la tasa de detección de argumentos. Los modelos de clasificación evaluados son: Naive Bayes, SVM, árboles de decisión, y k-vecinos más cercanos. En este punto, se utilizó la herramienta Weka 3 para entrenar y evaluar los distintos modelos generados.

4. Evaluación experimental

La evaluación experimental se realizó con alumnos de la materia Inteligencia Artificial de la carrera de Ingeniería de Sistemas de la UNCPBA, los cuales fueron un total de 37 estudiantes, que generaron un total de 264 sentencias. A continuación, se evaluó cuáles sentencias eran efectivamente un argumento y cuáles no, conformando de esta forma un total de 202 sentencias de entrenamiento.

Luego se evaluaron diversos escenarios de acuerdo a las distintas alternativas planteadas en la sección 2. Para cada uno de los escenarios, se aplicó el procesamiento correspondiente y a continuación se construyeron los modelos de clasificación mencionados en la sección 3.

Finalmente, distintas métricas fueron computadas y comparadas para determinar cuál de los enfoques obtuvo un mejor desempeño durante la clasificación de los argumentos. Las métricas utilizadas fueron: Efectividad, Precisión, Recall y F-Measure

4.1 Resultados del enfoque sin etiquetar

Para este caso el mejor resultado se obtuvo con SVM teniendo una efectividad del 70,79 % y un F-Measure de 0,7.

4.2 Resultados del enfoque basado en longitud de etiquetas

Para esta alternativa se incluyó el módulo de procesamiento de lenguaje natural variando las longitudes de las etiquetas. Ninguna variante proporcionó una mejora con respecto a los resultados obtenidos por el enfoque base.

4.3 Resultados del enfoque basado en conectores argumentativos

Este enfoque fue basado en la diferenciación de conectores argumentativos y las diferentes transformaciones que se le pueden aplicar al conjunto de datos previos a la generación del modelo. Los resultados obtenidos evidencian una mejora en cada una de las métricas analizadas al momento de compararlas con el enfoque base.

3.2 Detección de argumentos en textos legales

Este trabajo hace hincapié en la detección de argumentos en textos de carácter legal con el objetivo de expandir el dominio en el cual se pueden aplicar diversas técnicas de análisis y detección de argumentos.

3.2.1 Introducción

En el presente trabajo se considera que la argumentación tiene un papel muy importante en diversas áreas. Muchos profesionales (científicos, abogados, periodistas o gerentes) implícita o explícitamente manejan argumentos sistemáticamente. Dichas personas consumen gran cantidad de tiempo y esfuerzo interpretando grandes volúmenes de datos, ya sea para detectar argumentos, validar o refutar alguna idea, entre otros.

Considerando que los textos argumentativos aparecen frecuentemente en el ámbito legal, Mochales Palau y Moens centraron su trabajo en este tipo de textos, con el objetivo de ofrecer a abogados una herramienta que facilite el procesamiento de dichos textos.

3.2.2 Enfoque propuesto

El enfoque está centrado en la clasificación de textos en argumentos y no argumentos, teniendo como entrada textos en idioma anglosajón con contenido referenciado a hechos, antecedentes y explicaciones. El desafío planteado se basó en la descomposición de los argumentos en los componentes que lo conforman: premisas y conclusiones (Moens, Boiy, Palau, & Reed, 2007) (Palau & Moens, 2009).

Los primeros trabajos de Mochales Palau y Moens relacionados a esta temática, se centraron en la detección automática de argumentos en textos legales a un nivel de sentencias, determinando cuáles de ellas eran argumentativas y cuáles no. Cada sentencia se representó como un vector de características, entre las que se pueden mencionar:

- Unigramas: Cada palabra dentro de una sentencia. Se consideraron dos variantes, las cuales se diferencian en el tratamiento que realizan sobre los signos de puntuación.
- Bigramas: Pares de palabras sucesivas dentro de cada sentencia.
- Trigramas: Tercetos de palabras sucesivas dentro de cada oración.
- Adverbios: Pueden dar indicios de información argumentativa.
- Verbos: Sólo se consideraron los verbos principales.
- Auxiliares modales: Mediante un binario se determina la presencia de un modal auxiliar, el cual indica un nivel de necesidad. Por ejemplo:
 - must/need to/have to = obligación, requerimiento sin alternativa.
 - should/ought to/had better = recomendación.
 - can/could = condicional.
 - may/might = con opción o alternativa.
 - will/shall = intención.
 - would = condicional.

Estos verbos podrían indicar la presencia de un argumento.

- Tuplas de palabras: Todas las combinaciones de pares de palabras (todas las sentencias fueron consideradas). Con esta característica se intentó nutrir de contexto al análisis, a diferencia de los bigramas mencionados anteriormente. Como desventaja se debe mencionar que el vector resultante crece en gran medida.
- Estadísticas del texto: Se consideraron los siguientes detalles:
 - Longitud de sentencia.
 - Longitud de palabra promedio.
 - Cantidad de signos de puntuación.
- Puntuación: Se tuvieron en cuenta las secuencias de signos de puntuación.
- Palabras claves: Consideraron 286 secuencias de palabra/s obtenidas a partir de una lista de términos indicativos de argumentos. Por ejemplo "but", "consequently", and "because of".
- Árbol de características: Se analizaron las profundidades de los árboles obtenidos de cada sentencia.

Teniendo en cuenta las características previamente detalladas se entrenaron dos clasificadores, Naive Bayes Multinomial y Modelo de Máxima Entropía, usando como dataset el Corpus Araucaria y obteniendo como mejor resultado una efectividad del 73,75% (con Naive Bayes Multinomial como clasificador y las características: tuplas de palabras, verbos y longitud de sentencias).

Posteriormente, Mochales Palau y Moens focalizaron su investigación en la detección de premisas y conclusiones en sentencias argumentativas. Para ello, clasificaron las sentencias en argumentativas o no utilizando un algoritmo de máxima entropía. Una vez determinadas cuáles eran argumentativas, aplicaron como segundo clasificador SVM

(Support Vector Machine) para identificar premisas y conclusiones. En este análisis tuvieron en cuenta características más específicas a las consideradas en trabajos anteriores, entre las que se pueden mencionar:

- Ubicación absoluta: Posición de la sentencia respecto al documento.
- Longitud de sentencia: Atributo binario que indica si la longitud de la sentencia supera un cierto umbral (12 palabras en el trabajo descrito).
- Tiempo verbal: Tiempo verbal de la oración principal de la sentencia.
- Historia: La categoría más probable, tanto de las sentencias previas como posteriores.
- Información del primer clasificador: Resultados del clasificador basado en el algoritmo de máxima entropía.
- Patrones retóricos: Tipo de patrón retórico de la sentencia actual, previa y posterior. Se tuvieron en cuenta 5 tipos (apoyo, contraposición, conclusión, otros o ninguno).
- Referencia a un artículo: Atributo binario que indica una referencia a un artículo o ley.
- Artículo: Atributo binario que indica la presencia de la definición de un artículo o ley.
- Patrones argumentativos: Tipo de patrón argumentativo detectado en la sentencia.
- Tipo de actor: El sujeto de la sentencia (demandante, demandado, juzgado, entre otros).
- Tipo de verbo principal: Tipo argumentativo del verbo principal de la sentencia (premisa, conclusión, decisión final o ninguno).

Para entrenar el clasificador, utilizaron como dataset ECHR (European Court of human Rights) obteniendo como mejor resultado una precisión de 77,49% en la detección de conclusiones y un 70,19% en la detección de premisas (con la combinación de características: ubicación absoluta, longitud de sentencia y tiempo verbal de la oración principal de la sentencia).

Los clasificadores estadísticos no permiten detectar los límites de cada argumento, como tampoco la relación entre cada uno de ellos. Teniendo en cuenta esta limitación, Mochales Palau y Moens diseñaron una gramática libre de contexto para analizar gramaticalmente el texto de entrada. Con este enfoque se obtuvo una precisión de alrededor del 60%.

3.3 Identificación de argumentos de editoriales chinas

Este trabajo está focalizado en la identificación de argumentos en textos de editoriales chinas. Se llevó a cabo ante la falta de herramientas que resuelvan dicha problemática.

3.3.1 Introducción

Si se logra identificar a los textos argumentativos de forma precisa, los argumentos principales de un conjunto extenso de datos pueden ser extraídos. Esta identificación ofrece una amplia variedad de aplicaciones prácticas.

Al estudiar distintas investigaciones realizadas en el campo de la minería de argumentos, Marisa Chow detectó que ninguna de ellas estaba enfocada en el idioma chino mandarín. A partir de ello, desarrolló y presentó distintos métodos para la detección de argumentos orientados a corpus compuestos por textos de editoriales chinas, asumiendo que son textos ideales, a pesar de que una única editorial puede contener tanto textos argumentativos como no. (Chow, 2016)

3.3.2 Desarrollo

A lo largo del desarrollo del trabajo Chow se encontró con diversos desafíos. Por ejemplo, detectó que algunas opiniones o ideas pueden no estar expresadas a un nivel de palabras, quedando de forma implícita. También, los métodos por los cuales intentan expresarse las ideas pueden variar enormemente de una editorial a otra. Por ejemplo, unas pueden presentar la conclusión seguida de las premisas que lo justifican y otras pueden no presentar las conclusiones hasta el final del párrafo. Dado que un argumento puede tener una longitud difícil de delimitar y ser extenso en demasía, se definió como longitud máxima 200 caracteres (aproximadamente entre 3 y 5 sentencias).

Para su trabajo, Chow hizo uso de dos fuentes de información: una compuesta por editoriales chinas y la otra por reportajes en el mismo idioma. Ambos corpus forman parte de un dataset llamado Lancaster Corpus of Mandarin Chinese. La unidad de texto utilizada es el párrafo, el cual contiene un único argumento. Como se mencionó anteriormente, los párrafos compuestos por más de 200 caracteres fueron descartados.

Para etiquetar los 719 párrafos que componen el corpus de editoriales hizo uso de empleados de Amazon Mechanical Turk. Para cada párrafo, al empleado se le cuestionó si el autor del texto expresaba un argumento o no. Como respuesta, cada persona brindó una clasificación entre 3 opciones posibles: “si es un argumento”, “no es un argumento”, “no estoy seguro”. La tarea consistió en presentarles un párrafo a 3 individuos distintos, donde cada uno realizó su categorización. En dichas categorizaciones, solo en el 26% de los párrafos hubo consenso. El resto de los párrafos resultantes contenían al menos 2 respuestas distintas. Dado que los párrafos etiquetados con 3 respuestas distintas no brindaban información relevante, no fueron tomados en cuenta. Con este pre-procesado, el dataset final quedó compuesto por 622 párrafos. En la figura 16 se muestra la distribución obtenida del proceso de clasificación.

Paragraph Consensus	Count	Percentage
Makes an argument	484	67.32%
Makes NO argument	138	19.19%
Unsure	43	5.98%
No consensus	54	7.51%
<i>total</i>	719	

Figura 16. Clasificación de párrafos realizado por AMT

Inicialmente se puntuaron los párrafos mediante diversos métodos que se explicarán a continuación. Esto se realizó con el objetivo de identificar el mejor umbral para cada uno de ellos y, de esa manera, clasificar en textos argumentativos o no a cada una de las sentencias.

3.3.2.1 Comparación de frecuencias de palabras

Este método de evaluación está basado en el proceso presentado por Kim y Hovy en su paper “*identifying opinion-bearing words*” (Kim & Hovy, 2005). Inicialmente se construyó una lista de pares palabra-score, los cuales fueron utilizados posteriormente para evaluar los textos de las editoriales. Se basó en la idea de que las palabras que aparecen más frecuentemente en textos de editoriales que en textos de otros ámbitos pueden ser palabras que soportan argumentos. Por cada palabra se calcula un score basado en la siguiente fórmula:

$$Score(W) = \frac{EditorialProb(W)}{ReportageProb(W)}$$

Una vez obtenida la puntuación individual por palabra, se puntúa cada párrafo teniendo en cuenta las palabras que lo componen. Si un párrafo P contiene n palabras de opiniones con sus correspondientes frecuencias $f_1, f_2, \dots, f_n$ y scores asignados $s_1, s_2, \dots, s_n$, entonces el score correspondiente a un párrafo se calcula mediante la ecuación:

$$Score(P) = f_1s_1 + f_2s_2 + \dots f_ns_n$$

Con esta información, se determinó que el mejor umbral de puntuación es de 40.0.

3.3.2.2 Palabras argumentativas conocidas

Este segundo método consiste en la creación de una lista de palabras argumentativas presentes en el corpus de editoriales y la puntuación de los párrafos, basándose en la cantidad de ocurrencias de las mismas. Para llevarlo a cabo, Chow

seleccionó manualmente las frases argumentativas más frecuentes, obteniendo una lista de palabras con su frecuencia asociada.

De esta manera, se le asignó un punto al párrafo por cada ocurrencia de un elemento de la lista de palabras construida. Con esto se intentó determinar si la presencia de algunas palabras argumentativas podría indicar que el párrafo también lo era. Finalmente, llegó a la conclusión de que la mejor lista estaba compuesta por 15 palabras y el umbral era de 1, es decir, que si al menos contenía una palabra de la lista era etiquetada como positiva.

3.3.2.3 Método combinado

Este método consiste en la combinación de los dos anteriores. Al igual que en el segundo método, se puntuó cada párrafo considerando una lista de palabras argumentativas. Sin embargo, en lugar de generar manualmente dicha lista, esta fue creada a partir de las palabras con más alta puntuación obtenidas a partir del primer método. Este método se basó en la idea de que las palabras con más alta puntuación son aquellas que soportan un argumento, dado que estas aparecen con mayor frecuencia en los textos de editoriales por sobre los de reportajes.

De igual manera que en el segundo método, se puntuó el párrafo a través de las ocurrencias de dichas palabras, determinando que el mejor umbral era 1.

3.3.2.4 Resultados

Finalmente, el mejor resultado se obtuvo con el método de comparación de frecuencia de palabras (con un 84% de eficacia y un F1 de 0.91). La tabla completa de resultados se muestra en la figura 17.

Method	Accuracy	Precision	Recall	F ₁ score
1	0.841	0.847	0.971	0.905
2	0.801	0.826	0.942	0.880
3	0.371	0.850	0.233	0.366
1 = Relative Word Frequency Method (T=40)				
2 = Targeting Argument Words (T=1,W=15)				
3 = Combined Method (T=1,W=20)				
<i>T = threshold, W = word list size</i>				

Figura 17. Resultados de la tres variantes propuestas por Chow

3.4 Procesamiento semántico automático, enfocado en la coherencia textual, para apoyar la producción escrita de noticias

El siguiente trabajo presenta el diseño e implementación de un módulo de análisis semántico, enfocado en la predicción de la coherencia textual con un dominio temático definido, programado en Python 3. El método que se utilizó para la implementación fue el análisis semántico latente.

3.4.1 Introducción

Para el caso específico que se presenta, el analizador automático de coherencia textual se enfocó en noticias sobre política, ya que por exigencia del Análisis Semántico Latente, los textos incluidos en el corpus debían tener dos características fundamentales: contener una gran cantidad de textos y pertenecer a un dominio temático restringido.

El corpus con que se trabajó se compuso de 7165 noticias obtenidas desde el sitio en Internet del diario La Tercera de Chile, desde agosto de 2012 a febrero de 2014. En este trabajo se detalla el proceso realizado, partiendo del desarrollo de un método para la recopilación automática de los textos, pasando por la creación del script que realiza la tarea de prepararlos para ser utilizados por la aplicación de Análisis Semántico Latente, y finalizando con la construcción de un prototipo de analizador semántico, capaz de realizar el procesamiento también en forma automática.

3.4.2 Concepto fundamental: Análisis Semántico Latente

El Análisis Semántico Latente (o LSA, según sus iniciales en inglés, Latent Semantic Analysis) es una técnica que utiliza procesos estadísticos para extraer y representar los significados de las palabras. No es un procesamiento del lenguaje natural tradicional o programa de inteligencia artificial; no utiliza diccionarios contruidos por humanos, ni bases de conocimiento, ni redes semánticas, ni gramáticas, ni analizadores sintácticos ni morfologías o similares; y toma como única entrada el texto segmentado en palabras, definidas como cadenas de caracteres únicos y separadas en pasajes significativos o muestras como frases o párrafos. La ventaja práctica del LSA sobre otros tipos de mediciones (como, por ejemplo, shallow word overlap) es que va más allá de las similitudes léxicas tales como silla/sillas o correr/corrió; el LSA es capaz de evaluar la similitud semántica entre términos tales como silla/mesa, mesa/madera y madera/bosque. De este modo, el Análisis Semántico Latente no solo indica que dos términos están relacionados, sino que también señala cuán similares son (McCarthy et al. 2007: 111).

Para realizar la predicción de coherencia, el LSA representa cada palabra del texto procesado, así como unidades textuales mayores, en el espacio semántico a través de vectores. Las unidades textuales mayores, como una oración, se representan en este espacio semántico multidimensional como el promedio ponderado de los vectores de los términos que contienen. La relación semántica entre dos unidades de texto puede ser comparada determinando el coseno entre los vectores de ambas unidades. Por lo tanto,

para evaluar la coherencia entre la primera y la segunda oración de un texto, el coseno entre los vectores de las dos oraciones debe ser determinado. Por ejemplo, dos oraciones que utilicen exactamente los mismos términos y con la misma frecuencia tendrán un coseno de 1; mientras que dos oraciones que usen términos que no sean semánticamente relacionados, tenderán a tener cosenos cercanos a 0 ó menores aún, resultando un valor negativo (Foltz et al., 1998).

3.4.3 Metodología de investigación

La metodología de trabajo empleada incluyó cuatro etapas. Las tres primeras no son parte del prototipo construido, sino que se enfocan en (1) la recopilación automática del corpus, (2) la preparación automática del mismo y (3) la construcción del espacio semántico. Finalmente, la última etapa es (4) la construcción del prototipo evaluador automático de coherencia textual.

3.4.3.1 Recopilación automática del corpus

El objeto de estudio de esta investigación lo constituyó el género discursivo “noticias políticas”. Se eligieron noticias de este tópico en particular, por dos razones: La primera es por el gran volumen de material que se produce diariamente sobre el tema, lo que determinó finalmente en el tamaño del corpus; y la segunda razón es por el mayor esmero que hay en la redacción de las noticias sobre este tema. Si bien esto último puede considerarse algo subjetivo, en los medios de prensa se escoge con mayor cuidado a los periodistas y editores que se aboquen a este tema, por la importancia mediática del mismo. Las noticias sobre política fueron extraídas del sitio web del diario La Tercera, un medio de circulación nacional en Chile. La razón para elegir este periódico es que en su sitio en Internet cuenta con un canal dedicado a las noticias sobre política, las que se encuentran ubicadas dentro del directorio localizado en <http://www.latercera.com/noticia/politica>. Para recopilar las noticias, se utilizó la aplicación wget. Esta permite explorar un directorio en particular y descargar todo el contenido que haya en él al momento de la revisión.

3.4.3.2 Preparación automática del corpus

El funcionamiento de esta etapa se basó en un script programado en Python. Dicho script recibió los textos recopilados y los preparó para incorporarlos al prototipo propiamente. El script permitió realizar consultas sobre relaciones semánticas y coherencia textual.

El objetivo fue transformar la información obtenida en un archivo de texto plano, eliminando marcas, caracteres especiales y toda aquella información que, si bien no corresponde a un lenguaje de marcas para la web, era considerada inútil desde el punto de vista del analizador semántico. Para esto se optó por implementar un script completo que se adaptara a la medida de los archivos recopilados y eliminara todo lo irrelevante de dichos archivos, dejándolos listos para su utilización.

Adicionalmente, el script construido resolvió un problema particular de los textos: los elementos de titulación. Normalmente, en periodismo se aceptan de tres tipos: uno

imprescindible como el titular y dos optativos como el epígrafe o antetítulo (que va sobre el titular) y la bajada o subtítulo (que como su nombre indica, va debajo del titular).

La última operación de relevancia propia del script fue agregar a cada archivo procesado las marcas exigidas por la aplicación de Análisis Semántico Latente, Infomap-NLP, utilizadas para la construcción del espacio semántico en base al corpus textual. Estas marcas son <DOC> y <TEXT> (en ese orden) antes del inicio de cada texto, y </TEXT> y </DOC> al final de cada una de las noticias.

3.4.3.3 Construcción del espacio semántico

La creación del espacio semántico se realizó a través de un script encargado únicamente de tomar el archivo con los textos entregados por la etapa de preparación de los mismos y, en base a dicho fichero, previa lematización de los textos que lo conformaban, construir el espacio semántico multidimensional con el que se trabaja posteriormente.

El script se basó en las instrucciones para realizar esta tarea en forma manual entregadas en el sitio de Infomap-NLP, y se utilizó la construcción del modelo en un directorio de trabajo y no en un directorio permanente, porque de esta forma se tuvo mayor control sobre el mismo.

3.4.3.4 Evaluador automático de coherencia textual

Para el desarrollo del prototipo del evaluador automático de coherencia textual, al igual que para el script que construyó un espacio semántico, se utilizó Python y NLTK. Si bien su funcionamiento es más complejo que el de los tres scripts anteriores, expresándolo en términos simples, se encarga de las diferentes tareas necesarias para realizar la comparación de las unidades textuales de cada noticia procesada en el prototipo.

3.4.3.4.1 Lematización de los textos

Como se mencionó antes, fue necesario realizar un proceso de lematización. Para ello, se empleó el Snowball Stemmer, incluido en NLTK, ya que el mismo trabaja con lengua española y, además, con un grado de precisión bastante aceptable. Para poder utilizar Snowball, los textos debieron ser preparados previamente, eliminando los caracteres que no eran admitidos (tildes, comas, signos de interrogación, signos de exclamación, etc.).

3.4.3.4.2 Comparación de unidades textuales adyacentes

En el caso del script construido, no se buscaba averiguar las palabras que mayor relación semántica tienen con una palabra en particular, sino que el objetivo era comparar todas las unidades textuales adyacentes, párrafos para el caso, de una noticia en particular.

El prototipo, en esta etapa, toma cada uno de los textos del módulo de entrada por separado, los segmenta en párrafos y calcula el vector que representa a cada uno de estos párrafos. Luego, entrega los valores de cada vector, con el fin de poder pasar a la última tarea que realiza este script del prototipo: la comparación de todas las unidades textuales adyacentes, a través del cálculo del coseno del ángulo que forman los vectores que

representan a cada unidad. Este coseno es igual al producto escalar entre los vectores, dividido por el producto de sus módulos. Se puede representar como sigue:

$$\cos(A, B) = \frac{\sum_{i=1}^n a_i b_i}{|A| * |B|}$$

Una vez realizadas todas estas operaciones, el módulo de salida arroja los resultados en un archivo de texto plano que permite revisarlos y someterlos a algún tipo de procesamiento posterior.

3.4.4 Resultados de las pruebas realizadas al prototipo

El prototipo se sometió a una serie de pruebas en las que se alteraba intencionadamente la coherencia textual con el fin de determinar si era sensible a estos cambios: manipular la coherencia de textos, crear textos sin sentido uniendo párrafos extraídos desde diferentes noticias (cuya única relación era pertenecer al dominio en que se enfocó el trabajo) y comparar textos que presentaban la misma noticia, pero extraída de medios de prensa diferentes (para ello se trató a cada noticia como una única unidad textual).

Entre las pruebas realizadas se compararon oraciones construidas, alusivas a temas pertenecientes al dominio. Por ejemplo, al cotejar la secuencia “el diputado votó en contra de la ley” con “el senador rechazó el proyecto” el puntaje de coherencia que arrojó el prototipo fue de 0.81, dicha cifra refleja la similitud semántica de ambas oraciones y también da cuenta de los matices que las diferencian. Al cambiar la segunda oración por “el ministro fue a ver a la presidenta” el puntaje de la comparación descendió a 0.13, dando cuenta de la poca relación entre ambas secuencias.

3.4.5 Conclusiones

La construcción de este módulo de procesamiento semántico, se centró en la tarea mayor de diseñar un sistema tutorial inteligente destinado a apoyar la producción escrita de noticias en estudiantes de periodismo. La importancia de este trabajo radica en que la noticia es un texto complejo desde el punto de vista de su elaboración para los estudiantes de periodismo: exige que el autor comprenda el hecho sobre el que escribe y tenga la capacidad de resumirlo sin restarle nada de la información fundamental que involucra.

Pese al objetivo particular con que se desarrolló la herramienta aquí descrita, un prototipo como el diseñado, al basarse en scripts de texto, permite su adaptación a múltiples proyectos en que se requiera realizar análisis semánticos. De hecho, bastaría con cambiar el corpus a partir del cual se construye el espacio semántico y adaptar algunas líneas de código para tener una herramienta personalizada para trabajar en otro ámbito que se desee.

3.5 Resumen

Al considerar los trabajos citados, se puede observar que todos ellos presentan diferentes enfoques para la resolución de problemas de argumentación en distintos contextos o dominios teniendo en común la misma problemática. A partir de la carencia de

herramientas que resuelven esta problemática, se realizó el presente trabajo con el objetivo de ampliar el enfoque y realizar nuevos estudios para obtener nuevos resultados.

Capítulo 4

4. Enfoque propuesto

Este capítulo tiene como objetivo la explicación detallada de cada una de las etapas del proceso KDD aplicado. Como primer paso, se definen las fuentes de datos y los pre-procesamientos y transformaciones aplicados. Luego se detallan los dos modelos generados a partir de los distintos enfoques considerados, haciendo principal énfasis en su implementación.

4.1 Selección de datos

El objeto de estudio de esta investigación fue constituido por argumentos elaborados por alumnos de la cursada 2016 de la materia Inteligencia Artificial de la Universidad Nacional del Centro de la provincia de Buenos Aires (UNICEN). El trabajo de la cátedra fue orientado con la temática “Reglas de asociación y algoritmos de clasificación” en el cual los alumnos, a partir de una base de datos propuesta, debían realizar un análisis para luego evaluar las diferentes técnicas utilizadas con el objetivo de argumentar sobre cuál recomendaría.

La elección de estos datos se debió a que la herramienta, para lograr obtener resultados satisfactorios, debe analizar argumentos bien estructurados. Existen otras fuentes (como chat online o foros de discusión) las cuales no fueron consideradas dado que manejan un lenguaje informal y su estructura argumentativa además de ser poco clara, no provee las características necesarias para el análisis propuesto.

Se analizaron un total de 608 argumentos candidatos para los enfoques propuestos. Durante la selección de los mismos se identificaron diferentes características:

- Algunos de los argumentos no tenían de manera explícita los conectores argumentativos para lograr identificar si los mismos eran correctos o no, y de no serlo, poder determinar cuál era el conector faltante.

“En los clasificadores no se pudo tener en cuenta los valores indefinidos, esto nos quita una posible parte de la clasificación, a diferencia de las reglas de asociación.”

El argumento del ejemplo no contiene los indicadores de premisa y conclusión pero se consideró válido ya que al comenzar, expresa un resultado y luego las consecuencias del mismo.

- Argumentos que estaban estructuralmente incompletos ya que carecían, en la mayoría de los casos, del indicador de conclusión. En estas sentencias en la premisa se asumía el resultado de la misma sin expresarlo de manera conclusiva.

“Las reglas de asociación son más precisas y no es necesario insertar ruido.”

Este argumento no contiene el conector de conclusión, por lo tanto, se consideró como un argumento mal estructurado que carece de dicho indicador.

- Argumentos que comenzaban con el conector de conclusión y carecían de premisa. Estos casos fueron menos frecuentes en el análisis.

“En definitiva podemos concluir que no solo es necesario una buen dataset, sino un buen pre-procesamiento del mismo para poder generar buenas reglas y clasificaciones.”

Este argumento concluye un análisis el cual es implícito. Es por eso que se lo trata como un argumento que no contiene la premisa.

- Argumentos en los cuales identificar que estructuralmente eran válidos resultó una tarea difícil ya que la posición en la cual se encontraban los conectores no era la más frecuente de encontrar en un argumento dado que comenzaban con la conclusión seguido de la premisa.

“En conclusión, la evaluación de los algoritmos fue más simple que en reglas de asociación dado que hay menos parámetros que considerar.”

Este argumento comienza con la conclusión del análisis, en donde el autor expresa qué algoritmo fue más simple de utilizar y luego anuncia el porqué del mismo dentro de la premisa.

Todas estas consideraciones fueron de suma importancia al momento de separar el dataset para los dos enfoques propuestos en el trabajo. De esta forma, los argumentos que presentaron la característica de no poseer estructuralmente alguno de los dos conectores fueron utilizados para el enfoque estructural y los argumentos que eran correctos en cuanto a su estructura fueron utilizados para el segundo enfoque en el cual se realiza un análisis semántico de los mismos.

4.2 Pre-procesamiento y transformación

4.2.1 Tratamiento preliminar del dataset

De acuerdo a los trabajos realizados por los alumnos de la cátedra de Inteligencia Artificial, se obtuvieron 608 argumentos candidatos, los cuales fueron considerados para ser analizados y procesados individualmente. Se utilizaron diferentes resoluciones para obtener una estructura de datos útil para su posterior transformación. Dichas resoluciones se detallan a continuación:

- Para facilitar el trabajo en el procesamiento de las sentencias, se consideró que cada uno de ellas debía estar contenidas en un único párrafo. Se eliminó el salto de línea presente en algunos argumentos candidatos.
- Se realizó un análisis individual de cada uno de los argumentos candidatos con el objetivo de identificar si los mismos contenían las características necesarias para ser utilizado en los enfoques propuestos. En total se analizaron 606 sentencias de las cuales 506 eran no argumentativas y se utilizaron para realizar el primer análisis del

proyecto, el cual se basa en la detección de fallas argumentativas a nivel estructural. De los 506 no argumentos analizados, 258 carecían de premisa y los 248 restantes de conclusión.

- Del análisis general, las 100 sentencias restantes (las cuales a nivel estructural estaban bien constituidas), si bien se utilizaron para el segundo enfoque del trabajo, en el cual se analiza la construcción semántica de las mismas, a modo de no descartarlas e incrementar la cantidad de sentencias no argumentativas a analizar se extrajeron por un lado parte de la sentencia de premisa y parte de la conclusión para mantener balanceado el dataset.

“Considerando que para el caso del clasificador no se encontró una buena relación entre el resultado y los datos ya preprocesados, como consecuencia, no se logró un buen porcentaje de aciertos.”

En el ejemplo anterior se puede apreciar que la sentencia argumentativa es válida. A modo de utilizarla para el enfoque estructural se realizó la siguiente extracción:

- *Considerando que para el caso del clasificador no se encontró una buena relación entre el resultado y los datos ya preprocesados.*

Se considera sentencia no argumentativa por falta de conclusión.

- *como consecuencia, no se logró un buen porcentaje de aciertos.*

Se considera sentencia no argumentativa por falta de premisa.

De esta forma no fueron descartadas las sentencias sintácticamente válidas y se logró incrementar el volumen de los datos de análisis, obteniendo de esta forma un total de 710 sentencias no argumentativas.

- A la hora de analizar los conectores de conclusión se debió tener en cuenta que algunos de ellos podrían tener diferente significado según el contexto argumentativo. Si observamos el siguiente ejemplo:

“Hoy habrá paro de colectivos, por lo que tendré que ir al trabajo caminando.”

En este contexto, es correcto decir que este conector cumple la función de indicador de conclusión.

Ahora, si observamos la siguiente sentencia:

“Te compré un regalo, si no te gusta, podés cambiarlo por lo que quieras.”

Vemos que “por lo que” ya no se comporta como un indicador de conclusión.

- Debido a que el dataset contiene sentencias construidas por alumnos las cuales son propensas a contener errores ortográficos, se utilizó un corrector antes de someter las sentencias al procesamiento de lenguaje natural a modo de no generar ruido en la salida del archivo ARFF.

4.2.2 Procesamiento de lenguaje natural

A partir de las sentencias analizadas anteriormente, se requirió manipular individualmente a cada uno de ellas a través del uso de herramientas de procesamiento de lenguaje natural con el objetivo de mejorar la tasa de aciertos. Para esta práctica, no se tuvo en cuenta el aspecto semántico sino que se hizo foco en la función que cumplía cada palabra de la sentencia. De esta manera, se logró reducir considerablemente la cantidad de atributos a procesar.

El procesamiento de lenguaje natural es un proceso en modo pipeline, compuesto por diversos módulos con funciones específicas, que tienen una entrada y una salida previamente definidas. Cada módulo tiene como entrada la información procesada por un módulo anterior y como resultado se obtiene información enriquecida útil para los módulos que continúan en el pipeline.

Como se mencionó en el capítulo 2, para esta etapa se utilizó la herramienta de procesamiento Freeling. En la figura 18 se detalla el pipeline y a continuación se describen cada uno de los módulos utilizados y los detalles o consideraciones que se tuvieron en cuenta al momento del procesamiento.

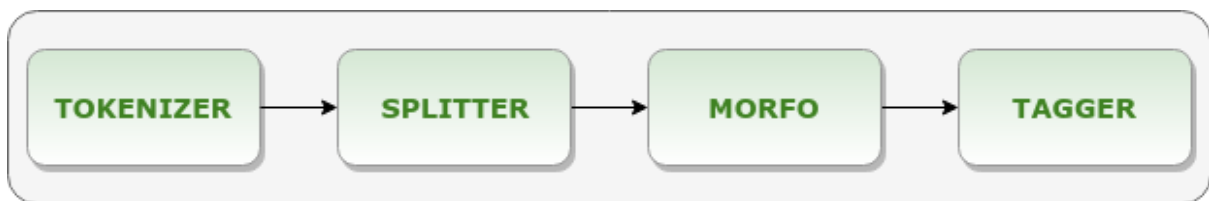


Figura 18. Pipeline de procesamiento de lenguaje natural

- Tokenizer: Es la primer etapa del proceso, el cual recibe como entrada un texto plano y devuelve una lista de objetos de tipo Word.
- Splitter: Recibe una lista de objetos de tipo Word (proporcionada por el módulo tokenizer) y retorna como salida una lista de objetos Sentence. En otras palabras, es el módulo que delimita el inicio y el fin de cada oración. Al procesar el dataset se detectaron algunas limitaciones:
 - Si la sentencia no finaliza con un punto, Freeling no es capaz de identificarla de forma correcta. Para solucionar este inconveniente se completó con un punto en los casos correspondientes.
 - Se debió corroborar dentro de cada sentencia el correcto uso de caracteres especiales tales como, “(”, “[”, “{” con el objetivo de evitar inconsistencias en el análisis. Para solucionar este problema se eliminaron los caracteres que presentaban inconsistencias.
- Morfo: Recibe una lista de objetos Sentence y analiza morfológicamente cada Word de cada Sentence de la lista. Este módulo aplica una cascada de analizadores especializados entre los que se puede mencionar la detección de números, fechas y multipalabras, entre otros.

- **Tagger:** Este módulo recibe una lista de objetos Sentence y desambigua la categoría morfosintáctica de cada palabra en las frases de la lista. Está basado en módulos ocultos de Markov y como resultado agrega una categoría a cada token generado por las etapas anteriores. En caso de que a un token se le haya asignado una etiqueta especial en la etapa anterior, esta es mantenida.

4.2.3 Generación de archivo de procesamiento

Luego de aplicar las diferentes transformaciones generadas por la herramienta PLN sobre las sentencias, se procedió a la generación de un archivo que fuese aceptado por la herramienta Weka. Como se mencionó en el capítulo 2, este archivo debe ser del tipo ARFF, y estar compuesto por un nombre de relación, una lista de atributos y un apartado de datos. Para la realización del mismo se utilizó la API de Java provista por WEKA. El archivo generado cuenta con las siguientes secciones:

- **@relation:** Se detallan los filtros y sus configuraciones. Para poder aplicar los algoritmos de clasificación se debió utilizar el filtro *stringtowordvector*, el cual transforma los textos en vectores de palabras.
- **@attribute FALTANTE{Premisa, Conclusión}:** Este atributo se instancia con el valor obtenido por la clasificación realizada por un experto, el cual determina si a una sentencia de entrada le falta la premisa o la conclusión.
- **@attribute TAG tipo:** Por cada tipo de etiqueta dentro del conjunto de sentencias se genera una entrada de este tipo. Por el filtro aplicado, todos los atributos son del tipo numérico.
- **@data:** Indica el comienzo de las sentencias de entrenamiento. Por cada sentencia se tiene una clase y un conjunto de atributos.

A continuación, de forma acotada, se muestra en la figura 19 el archivo ARFF para el caso “Sin etiquetar” el cual no fue sometido al procesamiento de lenguaje natural.

```

@relation 'original-weka.filters.unsupervised.attribute.StringToWordVector-R2-W1000-prune-rate-1.0-N0
-stemmerweka.core.stemmers.NullStemmer-M1-tokenizerweka.core.tokenizers.WordTokenizer
-delimiters \" \\r\\n\\t.,;:\\\\'\\\\\\\\()?!\\\"'

@attribute FALTANTE {Premisa,Conclusión}
@attribute 000 numeric
@attribute 2 numeric
@attribute 3 numeric
@attribute 5 numeric
@attribute '80\\%' numeric
@attribute A numeric
@attribute AD numeric
@attribute Además numeric
@attribute Apriori numeric
@attribute Asociación numeric
@attribute Asociación\" numeric
@attribute Así numeric
@attribute Bayes numeric
@attribute Bayesiano numeric
@attribute Clasificación numeric
@attribute Como numeric
@attribute Con numeric
@attribute Concluimos numeric
@attribute Concluyo numeric
@attribute Consecuentemente numeric
@attribute DataSet numeric

@data
{3 1,8 1,12 1,14 1,15 1,16 1,28 1,35 1,39 1,40 1,41 1,42 1,48 1,49 1,51 1,52 1,59 1,78
1,79 1,80 1,94 1,95 1,100 1,110 1,115 1,123 1}
{4 1,8 1,11 1,12 1,14 1,16 1,25 1,26 1,35 1,39 1,44 1,47 1,48 1,49 1,51 1,52 1,54 1,59
1,61 1,66 1,73 1,79 1,80 1,93 1,98 1,99 1,100 1,115 1,117 1}
{4 1,8 1,11 1,12 1,13 1,14 1,16 1,35 1,39 1,45 1,47 1,48 1,49 1,51 1,52 1,64 1,68 1,73
1,78 1,79 1,80 1,88 1,93 1,100 1,110 1,117 1}
{13 1,14 1,16 1,17 1,35 1,39 1,48 1,49 1,51 1,52 1,54 1,73 1,78 1,79 1,80 1,88 1,95 1,100 1}
{12 1,14 1,16 1,35 1,39 1,46 1,48 1,49 1,51 1,53 1,73 1,80 1,93 1,100 1}
{4 1,11 1,12 1,14 1,16 1,35 1,39 1,47 1,49 1,52 1,53 1,55 1,58 1,78 1,80 1,93 1,100 1,
104 1,111 1}
{5 1,12 1,13 1,15 1,16 1,27 1,35 1,39 1,47 1,49 1,51 1,52 1,53 1,62 1,73 1,78 1,80 1,94
1,95 1,100 1}
{12 1,13 1,16 1,19 1,28 1,34 1,35 1,38 1,39 1,40 1,41 1,44 1,48 1,49 1,51 1,52 1,53 1,
65 1,78 1,80 1,94 1,95 1,100 1}
{2 1,3 1,8 1,12 1,13 1,16 1,19 1,23 1,26 1,28 1,35 1,37 1,39 1,46 1,48 1,49 1,52
1,54 1,78 1,79 1,80 1,95 1,99 1,100 1,115 1,117 1}
{8 1,11 1,13 1,16 1,30 1,35 1,39 1,40 1,41 1,48 1,49 1,52 1,53 1,78 1,79 1,80 1,94 1,
100 1,101 1,118 1}

```

Figura 19. Fragmento de archivo de entrenamiento de Weka sin etiquetar

A modo de observar las diferencias se muestra en la figura 20 el archivo ARFF que fue sometido al procesamiento de lenguaje natural.

```
@relation 'original-weka.filters.unsupervised.attribute.StringToWordVector-R2-W1000-prune-rate-1.0-N0
-stemmerweka.core.stemmers.NullStemmer-M1-tokenizerweka.core.tokenizers.WordTokenizer
-delimiters \" \\r\\n\\t.,;:\\\\'\\\"()?!\\\"'

@attribute FALTANTE {Premisa,conclusión}
@attribute AO0MS0 numeric
@attribute AQ0CP0 numeric
@attribute AQ0CS0 numeric
@attribute AQ0FP0 numeric
@attribute AQ0FS0 numeric
@attribute AQ0MP0 numeric
@attribute AQ0MS0 numeric
@attribute AQSMS0 numeric
@attribute CC numeric
@attribute CS numeric
@attribute DA0FP0 numeric
@attribute DA0FS0 numeric
@attribute DA0MP0 numeric
@attribute DA0MS0 numeric
@attribute DA0NS0 numeric
@attribute DD0FP0 numeric
@attribute DD0FS0 numeric
@attribute DD0MP0 numeric

@data
{3 1,8 1,12 1,14 1,15 1,16 1,28 1,35 1,39 1,40 1,41 1,42 1,48 1,49 1,51 1,52 1,59 1,78 1,
79 1,80 1,94 1,95 1,100 1,110 1,115 1,123 1}
{4 1,8 1,11 1,12 1,14 1,16 1,25 1,26 1,35 1,39 1,44 1,47 1,48 1,49 1,51 1,52 1,54 1,59 1,
61 1,66 1,73 1,79 1,80 1,93 1,98 1,99 1,100 1,115 1,117 1}
{4 1,8 1,11 1,12 1,13 1,14 1,16 1,35 1,39 1,45 1,47 1,48 1,49 1,51 1,52 1,64 1,68 1,73 1,
78 1,79 1,80 1,88 1,93 1,100 1,110 1,117 1}
{13 1,14 1,16 1,17 1,35 1,39 1,48 1,49 1,51 1,52 1,54 1,73 1,78 1,79 1,80 1,88 1,95 1,100 1}
{12 1,14 1,16 1,35 1,39 1,46 1,48 1,49 1,51 1,53 1,73 1,80 1,93 1,100 1}
{4 1,11 1,12 1,14 1,16 1,35 1,39 1,47 1,49 1,52 1,53 1,55 1,58 1,78 1,80 1,93 1,100 1,104 1,
111 1}
{5 1,12 1,13 1,15 1,16 1,27 1,35 1,39 1,47 1,49 1,51 1,52 1,53 1,62 1,73 1,78 1,80 1,94 1,95 1,
100 1}
{12 1,13 1,16 1,19 1,28 1,34 1,35 1,38 1,39 1,40 1,41 1,44 1,48 1,49 1,51 1,52 1,53 1,65 1,78 1,
80 1,94 1,95 1,100 1}
{2 1,3 1,8 1,12 1,13 1,16 1,19 1,23 1,26 1,28 1,35 1,37 1,39 1,46 1,48 1,49 1,52 1,54 1,78 1,79
1,80 1,95 1,99 1,100 1,115 1,117 1}
{8 1,11 1,13 1,16 1,30 1,35 1,39 1,40 1,41 1,48 1,49 1,52 1,53 1,78 1,79 1,80 1,94 1,100 1,101
1,118 1}
```

Figura 20. Fragmento de archivo de entrenamiento de Weka etiquetado

4.3 Data mining

El objetivo de esta etapa es hallar el modelo óptimo, es decir, aquel que maximice la tasa de detección de sentencias no argumentativas candidatas inválidas, mediante la detección de errores sintácticos o semánticos.

Se utilizaron diversos algoritmos con el objetivo de generar distintas salidas y así, poder analizar cuál de ellas produce mejores resultados para, de esta manera, encontrar el modelo más adecuado. Dichos algoritmos se listan a continuación:

- SVM
- Naive Bayes
- J48
- IBK

Considerando que cada algoritmo tiene sus propios parámetros configurables, se generaron distintos modelos configurando dichos parámetros con diferentes valores para, de esta manera, comparar los resultados de cada uno de ellos. Cada una de las configuraciones se ejecutó con diferentes variantes obteniendo así un total de 16 modelos. A continuación se explican los diferentes enfoques considerados y sus variantes.

4.3.1 Enfoque basado en longitud de etiquetas

Este enfoque está basado en la salida obtenida de la herramienta Freeling, la cual consiste en la generación de etiquetas a partir de un texto de entrada. Dichas etiquetas están conformadas por un conjunto de caracteres, donde cada uno determina un atributo de la palabra analizada. Específicamente, el primer carácter de la etiqueta define la categoría, el segundo determina el tipo, y los caracteres siguientes brindan información relacionada con diversas características. En la figura 21 se muestra en detalle el significado de cada uno de los caracteres de una etiqueta.

Pos.	Atributo	Valor	Código
1	Categoría	Nombre	N
2	Tipo	Común	C
		Propio	P
3	Género	Masculino	M
		Femenino	F
		Común	C
4	Número	Singular	S
		Plural	P
		Invariable	N
5-6	Clasificación semántica	Persona	SP
		Lugar	G0
		Organización	O0
		Otros	V0
7	Grado	Aumentativo	A
		Diminutivo	D

Figura 21. Etiqueta de Freeling

Analizando la forma en que están diseñadas las etiquetas utilizadas por Freeling, se observa que los primeros caracteres tienen mayor importancia y los últimos son los menos relevantes para el análisis que se lleva a cabo en este trabajo de tesis. Utilizando esto como base, se decidió realizar el estudio variando la longitud de las etiquetas con el objetivo de explorar diversas alternativas y sus resultados. Se generaron distintos archivos de entrenamiento con etiquetas de longitud N (como máximo). Las alternativas consideradas fueron:

- N=2: Primeramente se consideraron sólo los primeros dos caracteres de cada etiqueta para realizar el análisis. De este modo, se consideró únicamente la categoría y tipo de cada palabra de la sentencia.
- N=3: Como segunda variante se agregó el tercer carácter de cada etiqueta para diferenciar así las palabras por su género.
- N=6: Finalmente, se utilizó como longitud máxima de etiqueta el valor N=6, incrementando así la cantidad de atributos a analizar.

4.3.2 Enfoque basado en conectores argumentativos

Como se explicó anteriormente, al analizar las sentencias de entrada se detectó que no poseen una estructura definida, es decir, no existe un orden en el cual se encuentren las premisas y las conclusiones. Por tal motivo, con el objetivo de poder detectar las partes de dichas sentencias independientemente de su lugar de aparición, se consideró en el análisis como una variante realizar un etiquetado distinguiendo entre los antecesores a premisas y los que indican una conclusión. Este enfoque encapsula todas las posibles formas de escritura de un indicador y las trata de igual manera, considerando la función que cumplen dichas palabras en la sentencia, independientemente de cómo esté redactado. Dichas frases o palabras se denominan indicadores de premisa y su estructura es la siguiente:

CONCLUSIÓN → INDICADOR DE PREMISA → PREMISA

Algunos posibles indicadores de premisa son: “En cuanto a”, “Si bien”, “En el caso de”, “En caso de”, “Considerando que”, “Puede que”, “Dado que”, “Como se”, “Como lo”, “Debido que”, “Debido a”, “Como el caso de”, entre otros.

Para configurar dichos indicadores, se agregó un registro por cada uno de ellos en el archivo *locucions.dat* propio de Freeling, como se muestra a continuación.

```
de_ballesta de_ballesta RG I
de_barra_a_barra de_barra_a_barra RG I
debido_a debido_a TAG-PRE I
debido_a_que debido_a_que TAG-PRE I
debido_que debido_que TAG-PRE I
```

Figura 22. Ejemplos de configuración de indicadores de premisa

De igual manera se detectaron indicadores de conclusión, los cuales respetan la siguiente estructura:

PREMISA → INDICADOR DE CONCLUSIÓN → CONCLUSIÓN

Algunos ejemplos de indicadores de conclusión son: “Lo que conlleva a”, “Se puede deducir que”, “Consecuentemente”, “En conclusión”, “Llegamos a la conclusión que”, “Lo cual muestra que”, “Lo cual prueba que”, “Por ende”, “Por estas razones”, “Por lo anterior”, “Por lo cual”, “Puede deducirse que”, entre otros.

Repitiendo el proceso seguido con las premisas, se agregó un registro por cada indicador de conclusión en el archivo *locucions.dat*.

```
salga_lo_que_salga salga_lo_que_salga RG I
<salir> <airoso> salir_airoso $1:VM I
se_podría_decir_que se_podría_decir_que TAG-CON I
se_puede_concluir_que se_puede_concluir_que TAG-CON I
se_puede_deducir_que se_puede_deducir_que TAG-CON I
se_puede_inferir_que se_puede_inferir_que TAG-CON I
se_sigue_que se_sigue_que TAG-CON I
```

Figura 23. Ejemplos de configuración de indicadores de conclusión

Al considerar ambos indicadores, las posibles estructuras completas de una sentencia son:

INDICADOR DE PREMISA → PREMISA → INDICADOR DE CONCLUSIÓN → CONCLUSIÓN

INDICADOR DE CONCLUSIÓN → CONCLUSIÓN → INDICADOR DE PREMISA → PREMISA

Con el objetivo de obtener mayor claridad en cada uno de los procesos, separar las etapas de análisis y lograr individualizarlas, la implementación se dividió en análisis sintáctico y semántico.

Este enfoque se tuvo en cuenta en ambas partes del análisis, a continuación se detalla el modo en que se aplicó en cada uno de ellos.

4.3.2.1 Análisis sintáctico

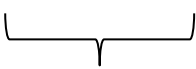
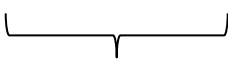
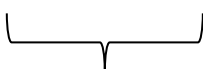
Este análisis, como se explicó anteriormente, consiste en la detección de errores estructurales en sentencias no argumentativas.

Los conectores argumentativos se utilizaron en este análisis para generar los archivos de entrenamiento y los que luego se clasificaron mediante el uso de la herramienta Weka.

4.3.2.2 Análisis semántico

En el análisis semántico se utilizaron los tags de indicadores de premisa y conclusión para separar las partes de cada sentencia de la siguiente manera:

- Dado una sentencia etiquetada, se buscó el tag correspondiente al inicio de la premisa (tag-pre).
- Se extrajo de la oración lo siguiente al tag de inicio de premisa, hasta la detección del tag de inicio de conclusión (tag-con) ó el final de la oración.
- Se realizó el mismo proceso con la conclusión, obteniendo lo siguiente:

Dado que	hace frío	entonces	me pongo el abrigo
			
TAG-PRE	PREMISA	TAG-CON	CONCLUSIÓN

Luego, se realizó un análisis de la premisa y la conclusión independientemente. Dicho análisis consistió en la extracción de todos los sustantivos de ambas partes de la sentencia analizada, para ello se utilizó Freeling.

En el ejemplo, los resultados obtenidos fueron:

Premisa: frío.

Conclusión: abrigo.

Es importante mencionar que las longitudes de las listas resultado dependen del contenido de la premisa y conclusión de la sentencia siendo posible, de esta manera, que se compongan de más de un sustantivo o incluso estar vacía.

Por ejemplo, para la sentencia “*Dado que hace calor, hay sol y no hay nubes entonces me saco el abrigo, la corbata y las botas*” las listas serán:

Premisa: calor sol nubes.

Conclusión: abrigo corbata botas.

Una vez obtenidas las listas de sustantivos, se utilizó la base de datos de la ontología Wordnet (citada anteriormente) para determinar la existencia de relaciones entre las palabras. Para el caso más simple, con un único sustantivo en la lista de premisas y conclusiones, el algoritmo realiza la búsqueda de registros en la base de datos (específicamente, en la tabla *wei_spa_30_relation*) en los cuales se vincule la palabra *frío* con *abrigo*.

Si, en cambio, las listas contienen varios sustantivos se realiza la búsqueda de todas las combinaciones de palabras, finalizando el algoritmo con la detección del primer positivo lo cual indicaría que las palabras analizadas están vinculadas semánticamente, es decir, la sentencia es un argumento semánticamente correcto. En el ejemplo, considerando las listas de tres sustantivos, se busca en la base de datos los pares:

- *calor - abrigo.*
- *calor - corbata.*
- *calor - botas.*
- *sol - abrigo.*
- *sol - corbata.*
- *sol - botas.*
- *nubes - abrigo.*
- *nubes - corbata.*
- *nubes - botas.*

La existencia de una relación se determina de la siguiente manera:

- Se buscan los valores de *offset* de ambas palabras en la tabla *wei_spa_30_variant*. En el ejemplo, los valores de *offset* de la palabra **frío** son *offset-pal1 001* y *offset-pal1 002* y el correspondiente a **abrigo** es *offset-pal2 001*.
- Se recorre la tabla *wei_spa_30_relation* en búsqueda de registros en los cuales estén presentes ambos valores de *offset* (uno correspondiente a cada palabra). Dichos valores están en los campos *source_synset* y *target_synset*.

En el ejemplo, se buscan las combinaciones:

- offset-pal1 001 y offset-pal2 001.
- offset-pal1 002 y offset-pal2 001.
- offset-pal2 001 y offset-pal1 001.
- offset-pal2 001 y offset-pal1 002.

La detección de un registro indica la existencia de la relación y la finalización del algoritmo de búsqueda para el par de palabras analizadas.

En la figura 24 se visualiza el ejemplo explicado.

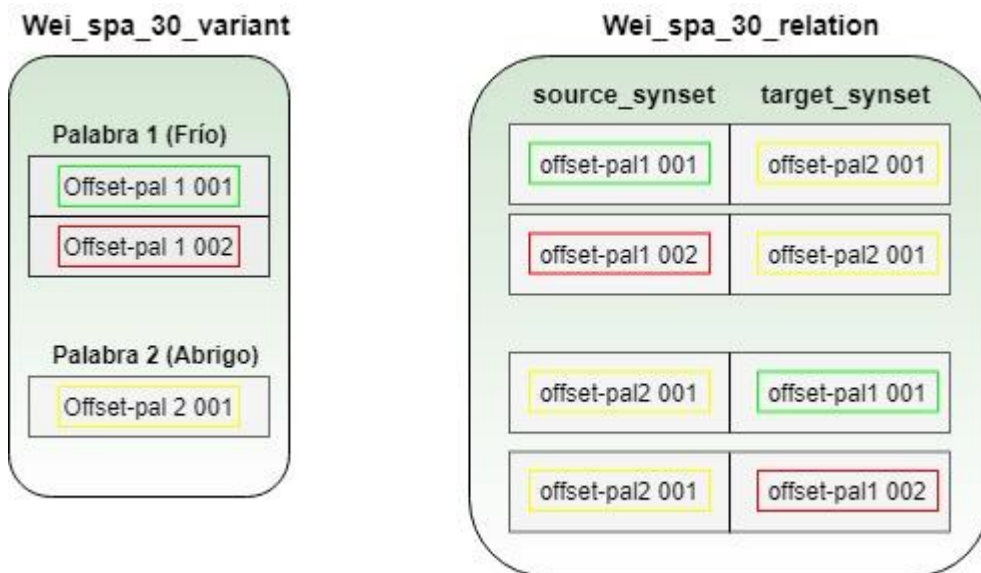


Figura 24. Ejemplo de búsqueda de relaciones semánticas

Teniendo en cuenta el modo en el que se analizan los datos y considerando el gran incremento en el tiempo de procesamiento que dicho análisis requiere, se presentan tres alternativas que tienen como objetivo mejorar la performance del procesamiento. La sentencia utilizada para la explicación de dichas alternativas es la siguiente:

***“Considerando que** el dataset brindado por la cátedra, había una cantidad mucho mayor de instancias con desempeño bajo en comparación con las demás. **Se puede deducir que** esta preponderancia, genera que el clasificador tienda a asignar las nuevas instancias a la clase “bajo” debido a que en datos de entrenamiento la cantidad de tuplas con dicho desempeño duplica al resto.”*

Al analizar la sentencia, se detecta como un indicador de premisa *“Considerando que”* y, *“Se puede deducir que”* se percibe como un indicador de conclusión. De esta manera, se guardan los sustantivos correspondientes a la primera oración en el archivo de premisa y los correspondientes a la segunda oración en el archivo de la conclusión.

4.3.2.2.1 Alternativa 1: Salida original

Esta alternativa consiste en realizar el análisis con los archivos generados originalmente por las herramientas de PLN. De esta manera, se tienen los sustantivos en el orden en que fueron encontrados en las oraciones.

En las figuras 25 y 26 se visualiza el contenido de los archivos de premisas y conclusiones.

```
1 dataset cátedra cantidad instancias desempeño comparación
  datos entrenamiento cantidad tuplas desempeño resto
```

Figura 25. Contenido de archivo Premisas.txt

```
1 preponderancia tienda instancias clase bajo
```

Figura 26. Contenido de archivo Conclusiones.txt

4.3.2.2.2 Alternativa 2: Salida ordenada

Este enfoque propone ordenar alfabéticamente la lista de sustantivos antes de realizar la búsqueda de relaciones. En las figuras 27 y 28 se visualiza el contenido de dichos archivos.

```
1 cantidad cantidad cátedra comparación dataset datos
2 desempeño desempeño entrenamiento instancias resto tuplas
```

Figura 27. Contenido de archivo Premisas.txt

```
1 bajo clase instancias preponderancia tienda
```

Figura 28. Contenido de archivo Conclusiones.txt

4.3.2.2.3 Alternativa 3: Salida ordenada y sin repeticiones

Esta propuesta consiste en, además de ordenar los sustantivos de cada archivo, eliminar las palabras repetidas en cada uno de ellos. En las figuras 29 y 30 se pueden observar los archivos de premisas y conclusiones.

```
1 cantidad cátedra comparación dataset datos
2 desempeño entrenamiento instancias resto tuplas
```

Figura 29. Contenido de archivo Premisas.txt

```
1 bajo clase instancias preponderancia tienda
```

Figura 30. Contenido de archivo Conclusiones.txt

4.4 Implementación

La implementación se dividió en dos proyectos que hacen uso de la misma etapa de procesamiento de lenguaje natural para luego centrarse en cada uno de los enfoques propuestos.

4.4.1 Enfoque basado en la estructura sintáctica del argumento

Luego de realizar los distintos pre-procesamientos y transformaciones detallados en la etapa anterior, se continuó con la implementación de una herramienta la cual es capaz de recibir el dataset de entrada, aplicar los diferentes procesos mencionados y generar un archivo de clasificación válido para luego ser analizados por la herramienta Weka.

Se generó una herramienta en la cual se aplicó el procesamiento de extracción de conocimiento KDD en donde, se obtuvo una única salida que será luego analizada por el modelo Weka con el objetivo de determinar la causa por la cual una sentencia es considerada no argumentativa. A continuación, en la figura 31 se muestra la herramienta generada para el enfoque sintáctico.

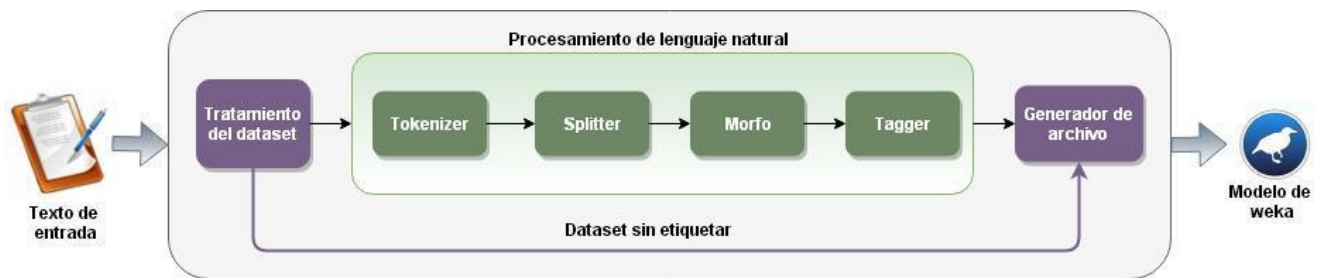


Figura 31. Herramienta resultante para el enfoque sintáctico

Como se observa en la figura, la herramienta resultante consta de una sub-herramienta la cual es utilizada para la generación del archivo.

Para la generación del archivo se tiene como entrada un archivo de extensión .xls que está conformado por dos columnas. La primera contiene la clasificación otorgada por un usuario experto y la segunda almacena la sentencia a analizar. La clase AbrirArchivo es la encargada de interpretar este archivo y generar una lista de Registros, en donde cada uno de estos está conformado por la información contenida en cada fila, bajo los nombres de “clasificación” y “texto”.

Luego se inicializan los recursos necesarios por cada una de las herramientas utilizadas. Esto incluye los módulos de análisis lingüísticos de Freeling necesarios para el procesamiento de lenguaje natural, y el clasificador implementado basado en la API de Weka.

Una vez inicializados cada uno de los recursos, se itera sobre la lista de registros cargados previamente, realizando por cada uno de ellos el pipeline definido previamente.

Luego, cada argumento candidato es procesado por el módulo de PLN, conformado por la clase Splitter, Tokenizer, Maco y Tagger, dispuestos en forma secuencial generando como salida el texto etiquetado. Esta salida dependerá de la variante de pre-procesamiento que se haya elegido, es decir, del tipo de enfoque con respecto a la longitud de etiquetas.

Para poder incorporar el argumento candidato al modelo de clasificador se deberá crear un objeto de la clase Instance de Weka que contendrá dos atributos: Clasificación, que almacena la clasificación contenida en la clase Registro, y Texto, encargado de guardar el argumento candidato etiquetado. Luego cada Instance es agregada a la colección de Instances.

Finalmente, y una vez realizado el procesamiento de cada registro de la lista, se procede al entrenamiento del modelo. Para ello será necesario aplicar previamente un filtro *StringToWordVector*, para convertir el texto en vectores de palabras.

Por medio del método *volcarDataset* de la clase Clasificador se obtiene el ARFF resultante a partir de la variante seleccionada.

La interacción entre los módulos previamente mencionado se visualiza en el diagrama de secuencia presentado en la figura 32.

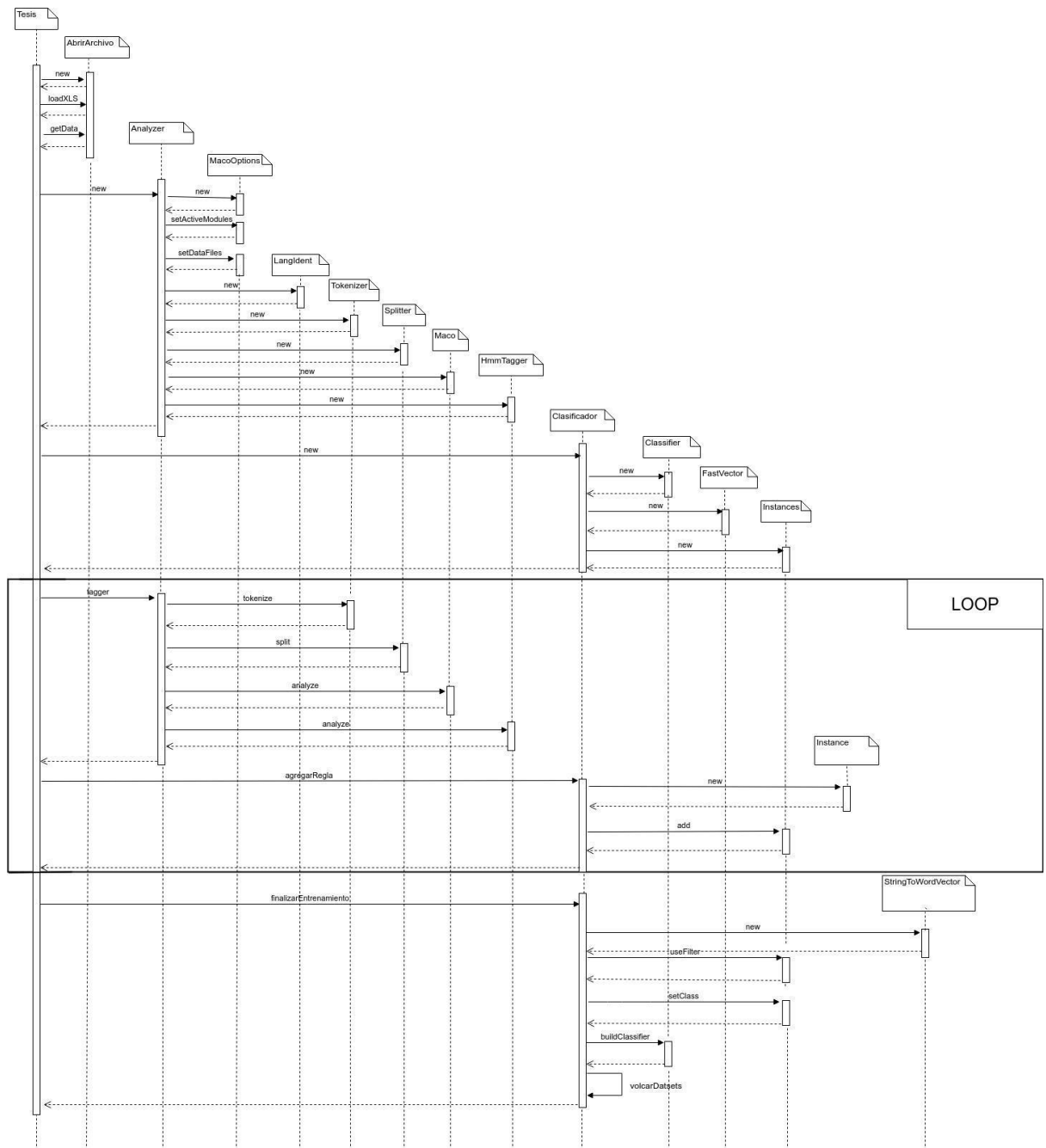


Figura 32. Diagrama de secuencia

4.4.2 Enfoque basado en el análisis semántico del argumento

Se desarrolló una segunda herramienta, la cual genera como salida un archivo de extensión txt con la lista de argumentos válidos semánticamente.

El objetivo de esta herramienta es poder determinar, dada una lista de argumentos estructuralmente correctos, cuáles de ellos poseen coherencia semántica. Para ello, se realiza el análisis detallado a continuación.

Inicialmente, se toma como entrada el dataset con las sentencias a evaluar. Dicha entrada es un archivo de extensión xls que posee una estructura definida, la cual se detalla en la figura 33.

	A	B	C	D	E	F	G	H	I	J
1	Si	Dado que hace frío entonces me pongo el abrigo.								
2	Si	Debido a que hace mucho calor se puede deducir que voy a salir a correr								
3	Si	Por lo tanto, saldré a caminar por la playa a las 7 dado que el día está espléndido								
4	Si	Dado que hace calor, hay sol y no hay nubes entonces me saco el abrigo, la corbata y las botas								

Figura 33. Estructura del dataset de entrada

El archivo se compone de dos columnas, la primera contiene la clasificación sintáctica de la sentencia, es decir, indica si estructuralmente es válida. Se marcó con “Si” al detectar la presencia de premisa y conclusión en dicha sentencia. Para evitar demoras innecesarias en el procesamiento, se eliminaron del dataset los registros cuya clasificación fue negativa, los cuales fueron analizados por la herramienta explicada anteriormente. En la segunda columna se almacena la sentencia propiamente dicha.

La clase AbrirArchivo es la encargada de leer el archivo xls y generar una lista de String, donde cada elemento es una sentencia.

En esta implementación se utilizan algunas de las herramientas mencionadas en el enfoque anterior. La secuencia que se lleva a cabo durante el proceso se observa en detalle en la figura 34.



Figura 34. Herramienta resultante para el enfoque semántico

Luego de la utilización de las herramientas de PLN, se obtienen como resultado dos archivos de extensión txt en los cuales se almacenan los sustantivos de las premisas y las conclusiones de cada sentencia, según corresponda.

Seguidamente, se establece la conexión con la base de datos de Wordnet, explicada en capítulos anteriores.

Se recorren los archivos de premisas y conclusiones, tomando pares de palabra y realizando consultas a la base de datos con dicho par. Es importante destacar que, en el

caso de detectar un vacío (como el registro 2 del ejemplo) se saltea la sentencia tomándola como inválida (dado que una de sus secciones no tiene ningún sustantivo para comparar). Adicionalmente, para el cuarto registro, se realiza el proceso con todas las combinaciones de palabras posibles, finalizando al encontrar el primer positivo.

A medida que se detectan relaciones, se almacena en un archivo de extensión txt el número del argumento válido, obteniendo la lista de los argumento semánticamente correctos al finalizar el proceso. La interacción previamente detallada puede verse reflejada en el diagrama de secuencia presentado en la figura 35.

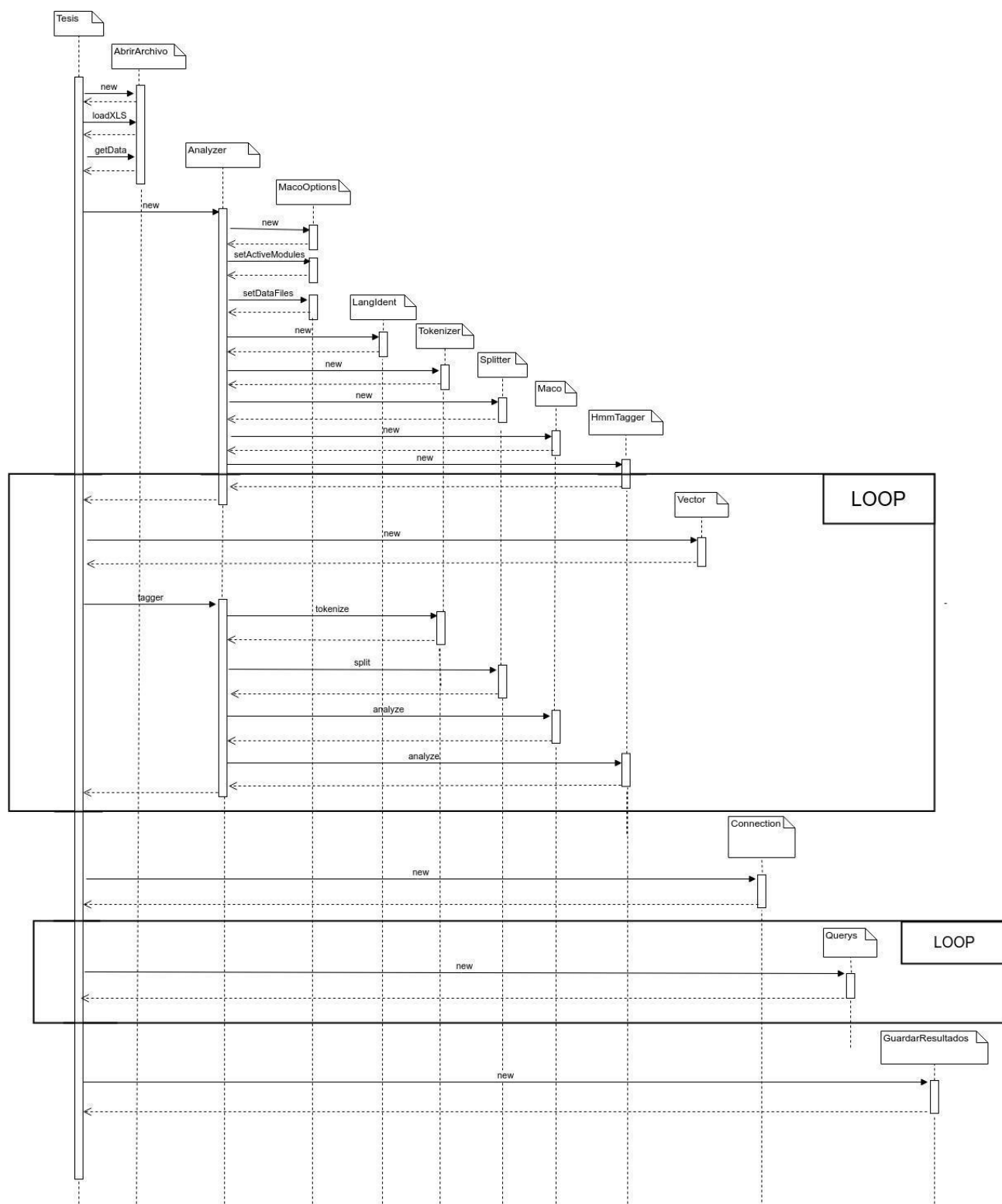


Figura 35. Diagrama de secuencia

4.5 Resumen

En este capítulo se detallaron los pasos para la obtención de una herramienta que fuera capaz de detectar errores en la construcción sintáctica de un argumento y el nivel de relación semántico entre las partes del mismo.

El trabajo fue dividido en dos enfoques bien definidos: el primero estuvo orientado a la estructura sintáctica del argumento y el segundo al análisis semántico.

Inicialmente, se detalló el origen de los datos y se identificaron diferentes características que fueron analizadas para detectar las sentencias candidatas, entre las cuales se pueden mencionar la ausencia de alguno de los conectores sintácticos, conectores que no se hallaban explícitamente en la sentencia, entre otros.

Luego, como primer instancia de la etapa KDD, se detallaron los pre-procesamientos y transformaciones que se aplicaron al dataset origen, con el objetivo de estructurar de forma óptima la información para que luego pueda ser reconocida y procesada por la herramienta de procesamiento de lenguaje natural Freeling cuya ejecución otorgó como salida resultante una secuencia de etiquetas.

Finalmente, se explicaron las consideraciones que se tuvieron en cuenta a la hora de la generación del archivo de entrenamiento. En esta etapa se detalló la composición del archivo ARFF y los filtros que fueron indispensables al momento de aplicar los distintos algoritmos de clasificación.

En la etapa de data mining del KDD se generaron modelos a partir de la combinación de distintos algoritmos y algunas variantes de los pre-procesamientos y transformaciones explicadas anteriormente. Si bien todas las variantes estaban enfocadas en la transformación de las etiquetas, se tuvieron en cuenta los diferentes análisis basados en longitudes de etiquetas, presencia de conectores argumentativos y ausencia de etiquetas en el dataset.

En el contexto mencionado anteriormente, para el enfoque basado en el análisis sintáctico, se hizo uso de estas combinaciones de las cuales se obtuvieron diferentes modelos que fueron analizados con el objetivo de encontrar la combinación que maximice la tasa de acierto al momento de clasificar.

Por último, para llevar a cabo el enfoque semántico del proyecto, sólo se consideraron los registros del dataset cuya estructura sintáctica era correcta y la etapa de data mining era basada en conectores argumentativos. Como resultado de esta etapa se obtuvieron 2 salidas “premisa” y “conclusión” las cuales fueron sometidas a análisis de relaciones, utilizando la base datos Wordnet.

Los análisis y evaluaciones de ambos enfoques se presentan en detalle en el siguiente capítulo.

Capítulo 5

5. Pruebas realizadas

En este capítulo se detallarán, para cada enfoque propuesto, los resultados obtenidos luego de realizadas las diferentes pruebas.

Con el objetivo de visualizar de forma clara dichos resultados y observar sus diferencias se utilizarán herramientas gráficas a lo largo del capítulo.

5.1 Enfoque basado en la estructura sintáctica del argumento

En este enfoque se mostrarán las métricas obtenidas para cada uno de los algoritmos seleccionados (Naive Bayes, SMO, IBK y J48). Inicialmente se presentarán los resultados que se obtuvieron al analizar el dataset original sin incluir la etapa de procesamiento de lenguaje natural, es decir, sin etiquetar las sentencias no argumentativas. Luego, se mostrarán los resultados de los diferentes enfoques propuestos analizados por la herramienta Weka.

5.1.1 Métricas utilizadas

Para cada modelo generado se presenta la matriz de confusión obtenida a partir de la ejecución del cross-validation con folds = 10. Esta técnica consiste en dividir un conjunto de datos, en este caso en n partes y, por cada una de ellas, se construye un clasificador con las $n-1$ partes restantes y se realizan las pruebas. Luego de realizar n veces la validación cruzada y calcular los resultados de la evaluación, Weka invoca el algoritmo de aprendizaje seleccionado por última vez para obtener el modelo que se imprime.

Para realizar las diferentes comparaciones entre los modelos generados se utilizaron cuatro métricas que se obtienen a partir de la matriz de confusión resultante.

1. Efectividad: Mide el número de términos correctamente clasificados respecto al total de términos del dataset.

$$ef = \frac{TP + FP}{TP + FP + FN + TN}$$

2. Precisión: Mide el número de términos correctamente reconocidos respecto al total de términos predichos, sean estos ciertos o no.

$$p = \frac{TP}{TP + FP}$$

3. Recall: Mide la proporción de términos correctamente reconocidos respecto al total de términos reales. Cuando este valor se acerca al 1, indica que el modelo se encuentra más cerca de detectar la totalidad de los términos a clasificar.

$$r = \frac{TP}{TP + TN}$$

4. F-Measure: es una métrica que permite caracterizar con un único valor la bondad de un clasificador o algoritmo.

$$f = 2 * \frac{precision * recall}{precision + recall}$$

Es importante aclarar que si bien la efectividad es una métrica que nos da una percepción de la calidad del modelo generado, no permite identificar otras variantes importantes a la hora de comparar entre un algoritmo y otro. Por este motivo, se utilizaron las métricas de precisión, recall y F-Measure las cuales proporcionan otra visión sobre los resultados obtenidos. Teniendo en cuenta que el dataset se encuentra balanceado, es decir, existe la misma cantidad de sentencias no argumentativas con faltante de premisas y faltante de conclusión, se buscará maximizar cada una de estas tres métricas sin que una perjudique en demasía a las otras.

5.1.2 Dataset inicial sin etiquetar

Con el objetivo de tener una métrica que pueda ser utilizada como base para evaluar los resultados obtenidos de los diferentes enfoques, se decidió utilizar como entrada los datos sin ser sometidos al procesamiento de lenguaje natural. De esta forma, se realizó un único pre-procesamiento de los datos de entrada con el objetivo de generar un archivo de entrenamiento que fuera aceptado por Weka.

A continuación se observan las diferentes matrices de confusión obtenidas de los algoritmos utilizados para este análisis, tales como Naive Bayes, SMO, J48 e IBK.

<pre> === Confusion Matrix === a b <-- classified as 169 87 a = Premisa 40 203 b = Conclusión Naive Bayes </pre>	<pre> === Confusion Matrix === a b <-- classified as 205 51 a = Premisa 68 175 b = Conclusión SMO </pre>	<pre> === Confusion Matrix === a b <-- classified as 183 73 a = Premisa 65 178 b = Conclusión J48 </pre>
<pre> === Confusion Matrix === a b <-- classified as 237 19 a = Premisa 196 47 b = Conclusión IBK </pre>		

Figura 36. Matriz de confusión para dataset sin etiquetar

- Modelo 1: Sin etiquetar

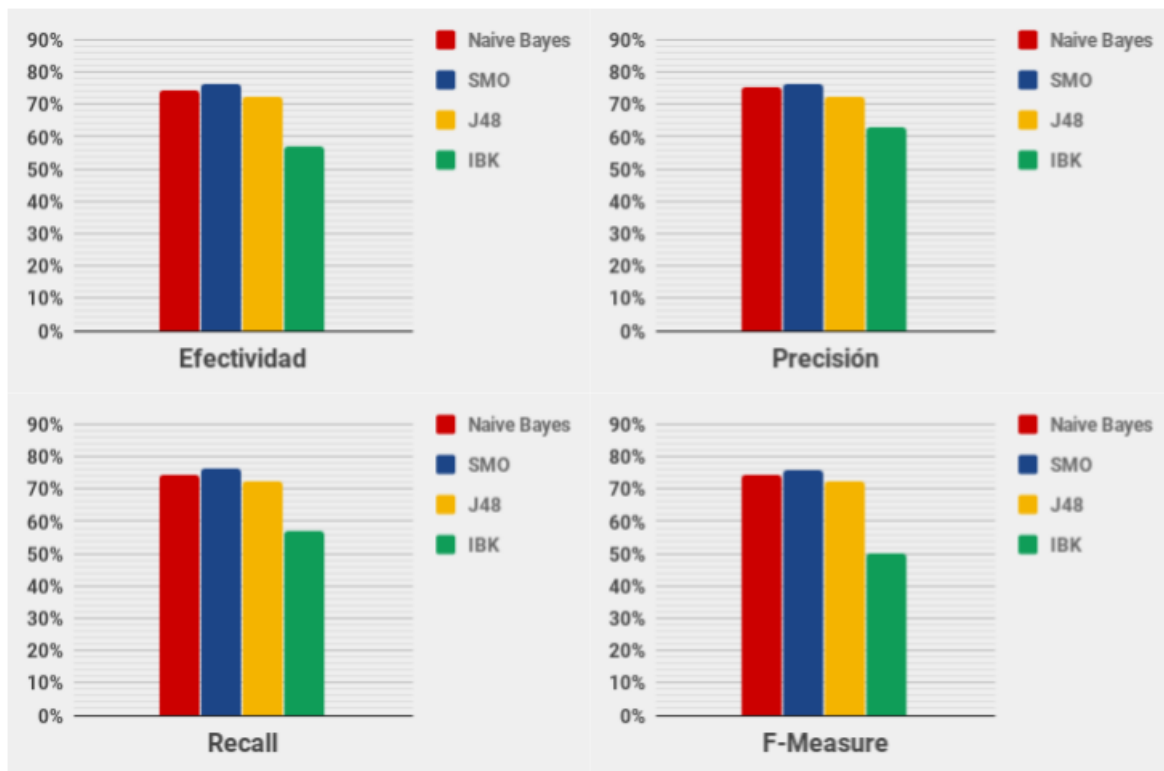


Figura 37. Gráficos de variables para el dataset sin etiquetar

Los porcentajes resultantes de cada algoritmo sobre cada una de las variables analizadas por la herramienta Weka graficados en la figura 37, se observan en la siguiente tabla:

	Efectividad	Precisión	Recall	F-Measure
 Naive Bayes	74,54%	75,6%	74,5%	74,4%
 SMO	76,15%	76,2%	76,2%	76,1%
 J48	72,34%	72,4%	72,3%	72,3%
 IBK	56,91%	62,8%	56,9%	50,1%

Como se visualiza en la figura 37, entre los resultados obtenidos el algoritmo SMO y Naive Bayes fueron los que tuvieron mejor rendimiento. Al analizar la tabla de porcentajes, se observa que ambos arrojan valores similares en cada métrica, SMO presenta una leve mejoría en cada métrica que Naive Bayes. Además, se obtuvo la misma probabilidad de precisión y recuperación lo cual es favorable para el cálculo de la medida F (F-Measure). También, se observa que en ambos algoritmos la precisión es mejor que su efectividad lo cual indica que el reconocimiento de los datos que se consideraron con falta de premisa y/o conclusión fueron reconocidos en su mayoría de manera efectiva.

Como conclusión del análisis, el mejor resultado fue obtenido por el algoritmo SMO con una efectividad de 76,15% y un F-measure de 76,1%. Estas variables, por su significado a la hora de realizar los distintos análisis, se utilizarán como parámetros para mejorar con los diferentes enfoques propuestos.

5.1.3 Enfoque basado en conectores argumentativos

Este enfoque se basa en el tratamiento de conectores argumentativos en el cual se incluye el módulo de procesamiento de lenguaje natural y se encapsulan todas las posibles formas de escritura de un indicador tratandolas de igual manera distinguiendo entre los que indican premisas y conclusión.

A continuación se presenta cada una de las matrices de confusión que generó cada algoritmo utilizado para este análisis, los cuales son Naive Bayes, SMO, J48 e IBK.

<pre> === Confusion Matrix === a b <-- classified as 194 62 a = Premisa 54 189 b = ConclusiÃ³n </pre> Naive Bayes	<pre> === Confusion Matrix === a b <-- classified as 203 53 a = Premisa 64 179 b = ConclusiÃ³n </pre> SMO	<pre> === Confusion Matrix === a b <-- classified as 205 51 a = Premisa 73 170 b = ConclusiÃ³n </pre> J48
<pre> === Confusion Matrix === a b <-- classified as 222 34 a = Premisa 123 120 b = ConclusiÃ³n </pre> IBK		

Figura 38. Matriz de confusión el dataset con etiqueta diferenciada entre indicadores

- Modelo 2: Etiqueta premisa y conclusión

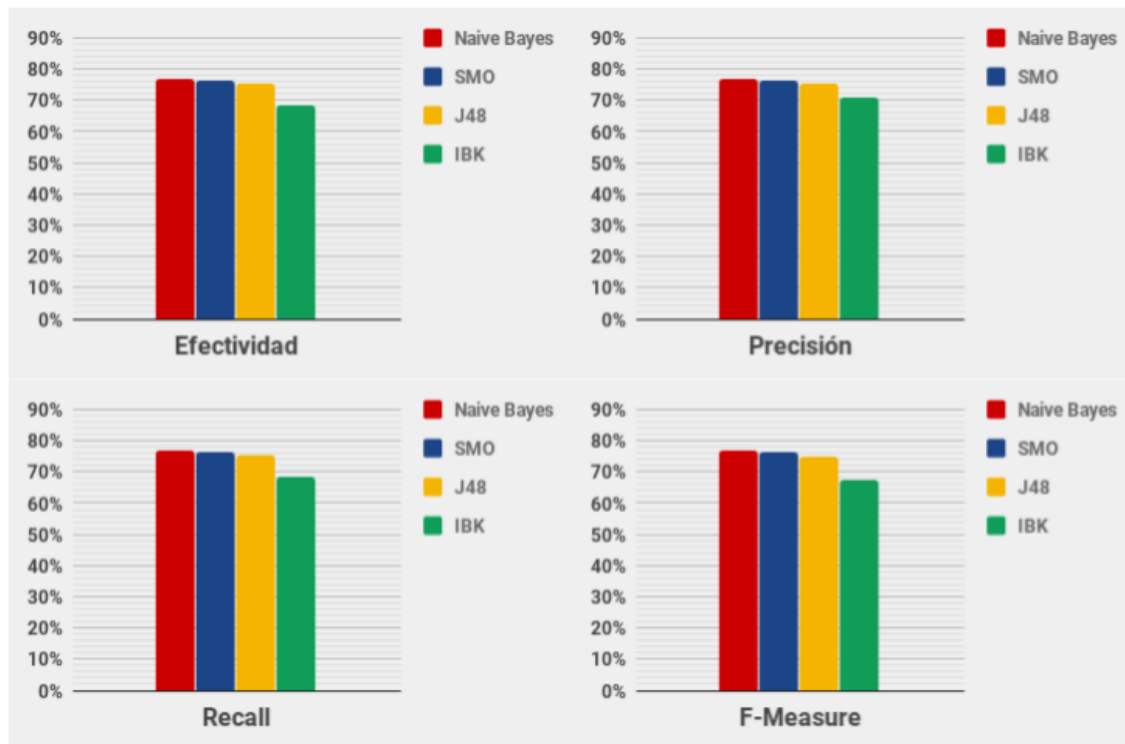


Figura 39. Gráficos de variables para el dataset sin etiquetar

Los porcentajes resultantes de cada algoritmo sobre cada una de las variables analizadas por la herramienta Weka graficados en la figura 39 se observan en la siguiente tabla:

	Efectividad	Precisión	Recall	F-Measure
 Naive Bayes	76,75%	76,8%	76,8%	76,8%
 SMO	76,55%	76,6%	76,6%	76,5%
 J48	75,15%	75,3%	75,2%	75,1%
 IBK	68,53%	71%	68,5%	67,3%

Como se puede visualizar en la figura 39, entre los resultados obtenidos el algoritmo SMO y Naive Bayes, al igual que el modelo sin etiquetar, fueron los que tuvieron mejor rendimiento. Al analizar la tabla de porcentajes, ambos arrojan valores similares en las métricas precisión y recall, lo cual indica que a la hora de analizar los datos, ambos clasificaron de forma correcta aproximadamente las mismas instancias.

Como conclusión de este análisis, el mejor resultado fue obtenido por el algoritmo Naive Bayes con una efectividad de 76,75% y un F-measure de 76,8%.

Se puede apreciar que los resultados detallados anteriormente superan los valores obtenidos en el primer modelo, en el cual el dataset fue analizado sin ser sometido al módulo de procesamiento de lenguaje natural.

5.1.4 Enfoque basado en longitud de etiquetas

Para este enfoque se incluyó el módulo de procesamiento de lenguaje natural y se analizaron los resultados teniendo en cuenta:

1. Las diferentes longitudes de las etiquetas almacenadas.
2. El etiquetado de los indicadores de premisa y conclusión.

Como se explicó en el capítulo anterior, las variantes que fueron consideradas son las de longitud de etiqueta 6, 2 y 3.

5.1.4.1 Longitud de etiqueta = 6

A continuación se observan las diferentes matrices de confusión obtenidas de los algoritmos utilizados para este análisis, tales como Naive Bayes, SMO, J48 e IBK.

<pre> === Confusion Matrix === a b <-- classified as 198 58 a = Premisa 57 186 b = Conclusión Naive Bayes </pre>	<pre> === Confusion Matrix === a b <-- classified as 203 53 a = Premisa 58 185 b = Conclusión SMO </pre>	<pre> === Confusion Matrix === a b <-- classified as 203 53 a = Premisa 69 174 b = Conclusión J48 </pre>
<pre> === Confusion Matrix === a b <-- classified as 223 33 a = Premisa 119 124 b = Conclusión IBK </pre>		

Figura 40. Matriz de confusión para longitud de etiqueta n=6

- Modelo 3: Longitud de etiqueta = 6

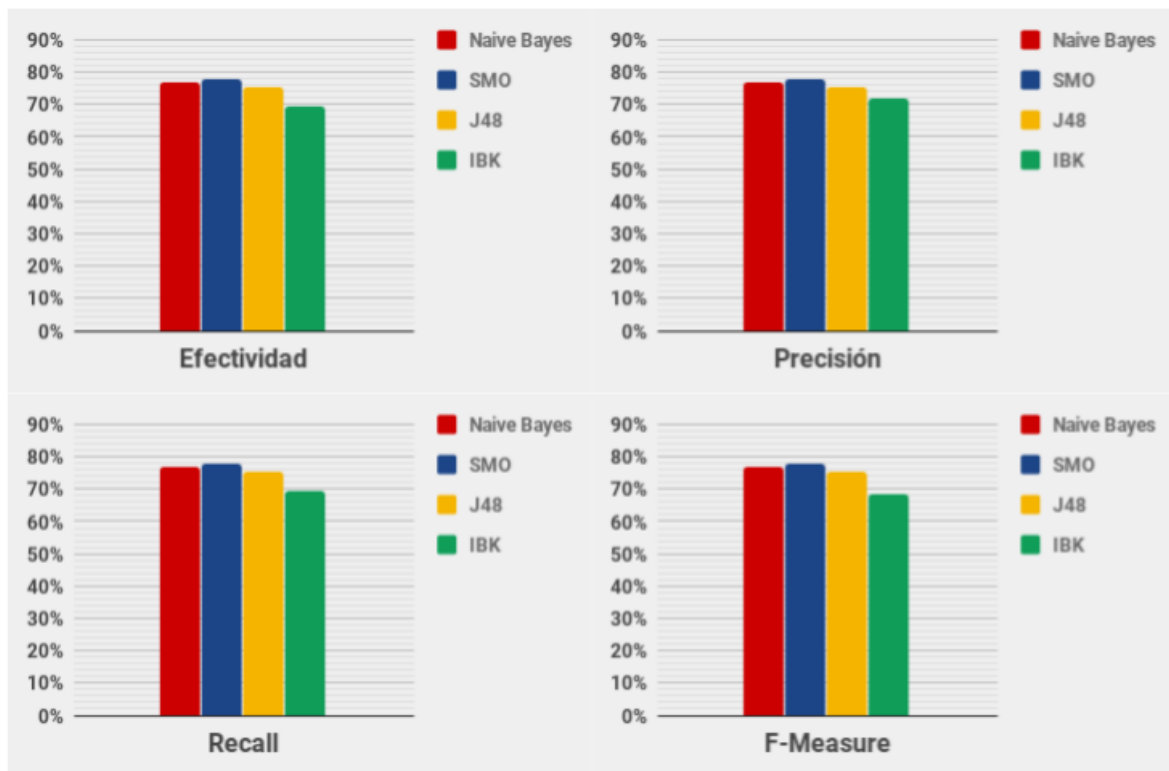


Figura 41. Gráficos de variables para el dataset de longitud de etiqueta n=6

Los porcentajes resultantes de cada algoritmo sobre cada una de las variables analizadas por la herramienta Weka graficados en la figura 41 se observan en la siguiente tabla:

	Efectividad	Precisión	Recall	F-Measure
Naive Bayes	76,95%	77%	77%	77%
SMO	77,75%	77,8%	77,8%	77,7%
J48	75,55%	75,6%	75,6%	75,5%
IBK	69,53%	71,9%	69,5%	68,5%

Como se puede visualizar en la figura 41, entre los resultados obtenidos el algoritmo SMO y Naive Bayes nuevamente fueron los que tuvieron mejor rendimiento. Al analizar la tabla de porcentajes se puede apreciar que ambos vuelven a arrojar valores muy similares en cuanto a la precisión y recall, por lo que se puede deducir que las instancias de clasificación correctas fueron prácticamente las mismas.

Como conclusión final para este análisis, el mejor resultado fue obtenido por el algoritmo SMO con una efectividad de 77,75% y un F-measure de 77,7%.

Finalmente, se observa que los resultados detallados anteriormente superan a los valores obtenidos en el segundo modelo. Considerando que este último utiliza el módulo de

procesamiento natural, este genera en su salida el dataset con longitud de etiqueta completa, es decir, de 2 a 7 caracteres. Por consiguiente, se deduce que hay caracteres que son redundantes en la etiqueta y causan mayor ruido a la hora del análisis provocando que los resultados obtenidos sean menos eficientes. De esta manera, al realizar un recorte óptimo en la etiqueta como se analizó en el presente modelo, los resultados obtenidos serán los deseables.

5.1.4.2 Longitud de etiqueta = 3

A continuación se observan las diferentes matrices de confusión obtenidas de los algoritmos utilizados para este análisis, tales como Naive Bayes, SMO, J48 e IBK.

<pre> === Confusion Matrix === a b <-- classified as 192 64 a = Premisa 118 125 b = Conclusión Naive Bayes </pre>	<pre> === Confusion Matrix === a b <-- classified as 188 68 a = Premisa 111 132 b = Conclusión SMO </pre>	<pre> === Confusion Matrix === a b <-- classified as 174 82 a = Premisa 104 139 b = Conclusión J48 </pre>
<pre> === Confusion Matrix === a b <-- classified as 199 57 a = Premisa 143 100 b = Conclusión IBK </pre>		

Figura 42. Resultados para longitud de etiqueta n=3

- Modelo 4: Longitud de etiqueta = 3

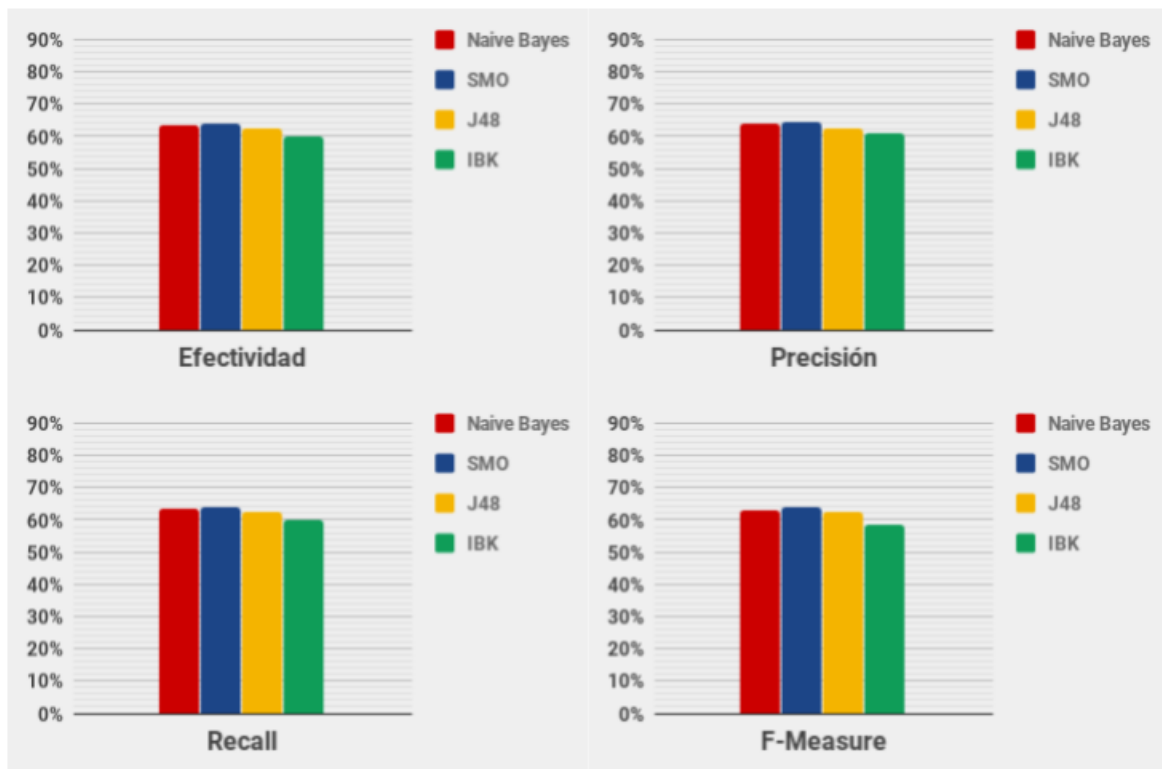


Figura 43. Gráficos de variables para el dataset de longitud de etiqueta n=3

Los porcentajes resultantes de cada algoritmo sobre cada una de las variables analizadas por la herramienta Weka graficados en la figura 43 se observan en la siguiente tabla:

	Efectividad	Precisión	Recall	F-Measure
Naive Bayes	63,52%	64%	63,5%	63%
SMO	64,12%	64,4%	64,1%	63,8%
J48	62,72%	62,7%	62,7%	62,6%
IBK	59,91%	60,9%	59,9%	58,5

Como se visualiza en la figura 43, entre los resultados obtenidos nuevamente los algoritmos SMO y Naive Bayes fueron los que tuvieron mejor rendimiento. Al observar la tabla de porcentajes se detecta una similitud en el resultado de la variante F-Measure en ambos algoritmos, lo que permite deducir que ambos presentan una semejanza en cuanto al equilibrio entre la precisión y el recall.

Como conclusión final para este análisis, el mejor resultado fue obtenido nuevamente por el algoritmo SMO que, a pesar de presentar una repetida similitud con Naive Bayes, tanto la eficiencia (64,12%) como el F-measure (63,8%) fueron mejores que este último.

Es importante destacar que esta alternativa no mejora los resultados obtenidos en el modelo 3, donde la longitud de etiqueta es 6. Esto permite deducir que al reducir significativamente la cantidad de caracteres se está disminuyendo considerablemente la cantidad de atributos que componen el dataset de entrenamiento provocando así la pérdida de características esenciales a la hora de realizar el análisis.

5.1.4.3 Longitud de etiqueta = 2

A continuación se observan las diferentes matrices de confusión obtenidas de los algoritmos utilizados para este análisis, tales como Naive Bayes, SMO, J48 e IBK.

<pre> === Confusion Matrix === a b <-- classified as 192 64 a = Premisa 119 124 b = Conclusión Naive Bayes </pre>	<pre> === Confusion Matrix === a b <-- classified as 173 83 a = Premisa 102 141 b = Conclusión SMO </pre>	<pre> === Confusion Matrix === a b <-- classified as 187 69 a = Premisa 117 126 b = Conclusión J48 </pre>
<pre> === Confusion Matrix === a b <-- classified as 192 64 a = Premisa 136 107 b = Conclusión IBK </pre>		

Figura 44. Matriz de confusión para longitud de etiqueta n=2

- Modelo 5: Longitud de etiqueta = 2

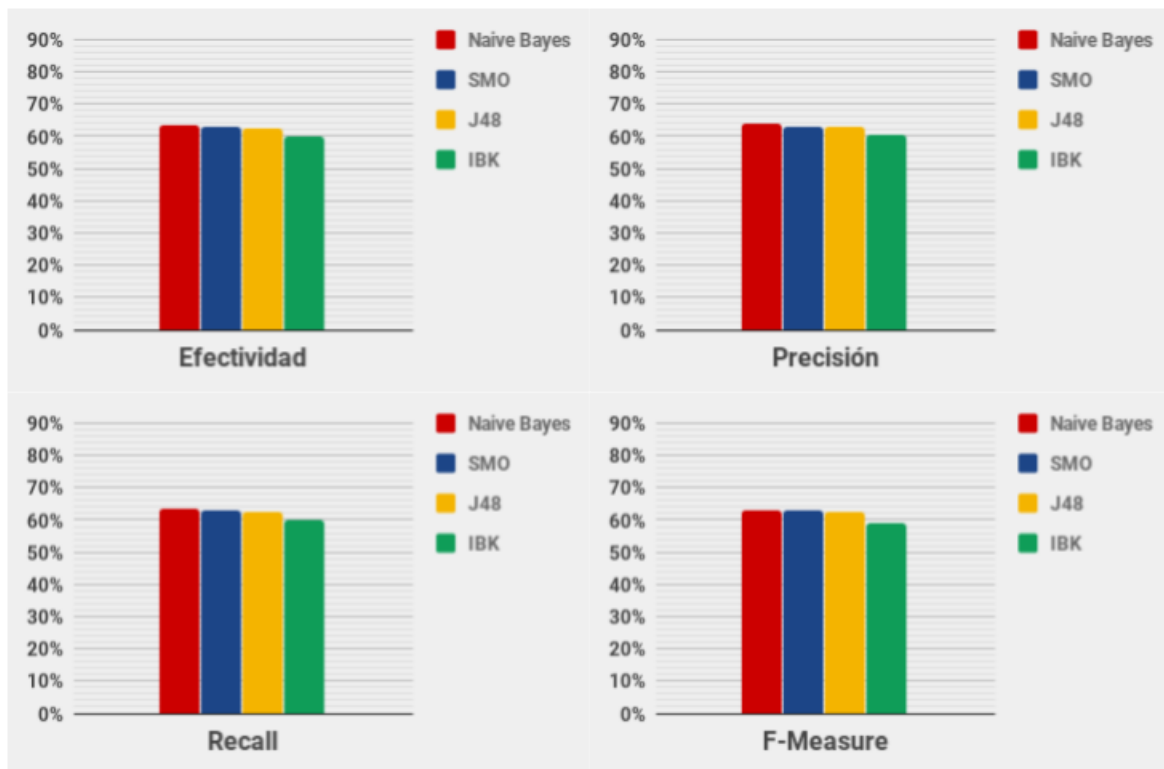


Figura 45. Gráficos de variables para el dataset de longitud de etiqueta n=2

Los porcentajes resultantes de cada algoritmo sobre cada una de las variables analizadas por la herramienta Weka graficados en la figura 45 se visualizan en la siguiente tabla:

	Efectividad	Precisión	Recall	F-Measure
Naive Bayes	63,32%	63,8%	63,3%	62,8%
SMO	62,92%	62,9%	62,9%	62,8%
J48	62,72%	63%	62,7%	62,3%
IBK	59,91%	60,5%	59,9%	58,9%

Como se observa en la figura 45, entre los resultados obtenidos el algoritmo Naive Bayes fue el que tuvo mejor rendimiento. Al visualizar la tabla de porcentajes se detecta que a la hora de analizar los datos fue más preciso que efectivo, lo que indica que los valores que se consideraron correctos fueron analizados como tales.

Como conclusión de este análisis, si bien SMO y Naive Bayes obtuvieron el mismo F-Measure (62,8%), este último fue más efectivo (63,32%) por lo que Naive Bayes fue el que obtuvo mejores resultados.

Es importante destacar que esta alternativa no mejora los resultados obtenidos en el modelo anterior, donde la longitud de etiqueta es 3. Esto permite deducir nuevamente que al

continuar acotando la cantidad de caracteres se sigue disminuyendo considerablemente la cantidad de atributos que componen el dataset lo cual provoca que, en vez de producir una mejora en cuanto a los resultados, ocasiona mayor ruido en el resultado final.

finalmente, para concluir con los resultados de las pruebas realizadas para el enfoque basado en la estructura sintáctica del argumento, se puede deducir que el modelo 3, basado en el enfoque de longitud de etiqueta igual a 6, presenta la mejor tasa de acierto al momento de detectar el problema por el cual una sentencia argumentativa no es considerada como tal. Es importante destacar que todas las variantes evidencian una mejora en cada una de las métricas analizadas al momento de compararlas con el modelo inicial. Esto se debe a que, sí se logra realizar una reducción óptima en la longitud de las etiquetas, eliminando aquellos caracteres innecesarios y se destacan de algún modo los conectores argumentativos que resaltan por sobre el conjunto de etiquetas, se obtiene como resultado la correcta clasificación de la sentencia argumentativa.

5.2 Enfoque basado en el análisis semántico del argumento

Inicialmente, se detallarán los resultados obtenidos del estudio semántico de las sentencias sintácticamente correctas.

Considerando que el dataset está compuesto por argumentos semánticamente correctos, determinados en la etapa de pre-procesamiento, se denomina “Éxito” a los identificados como tales y “Fracaso” a los que la herramienta implementada no clasifica como semánticamente correctos.

En la figura que se presenta a continuación, se detallan los porcentajes de aciertos logrados con distintas cantidades de argumentos de entrada.

Resultados	Cantidad de Argumentos Analizados					
	10	15	20	30	50	100
Éxito	80%	73,33%	70%	76%	82%	86%
Fracaso	20%	26,67%	30%	24%	18%	14%

Figura 46. Resultados del enfoque semántico

Al analizar el gráfico se observa que, en todos los casos, el porcentaje de éxito es un valor superior al 70% lo cual refleja la calidad de la herramienta desarrollada.

Es de suma importancia resaltar que los porcentajes resultantes no se ven afectados por las distintas alternativas propuestas, mencionadas anteriormente. Esto se debe a que el objetivo de dichas alternativas apunta a mejorar la performance con respecto al tiempo de procesamiento.

A continuación se mostrarán los resultados obtenidos para cada una de las propuestas analizadas.

5.2.1 Alternativa 1: Salida original

Como primera alternativa, se consideraron como entrada los archivos de premisas y conclusiones originales, es decir, los generados por la herramienta de PLN, explicado en el capítulo anterior.

Los resultados obtenidos se visualizan en la figura 47.



Figura 47. Gráfico de performance para la alternativa 1

Al observar el gráfico se puede determinar que los tiempos de ejecución aumentan considerablemente al incrementar la cantidad de sentencias analizadas.

Para esta alternativa, el tiempo total de ejecución con un dataset de entrada compuesto por 100 sentencias fue de 615 minutos.

5.2.2 Alternativa 2: Salida ordenada

Con el objetivo de mejorar los resultados obtenidos en la alternativa anterior, se propone realizar un ordenamiento en los datos de entrada antes de su procesamiento.

Los resultados obtenidos se visualizan en la figura 48.

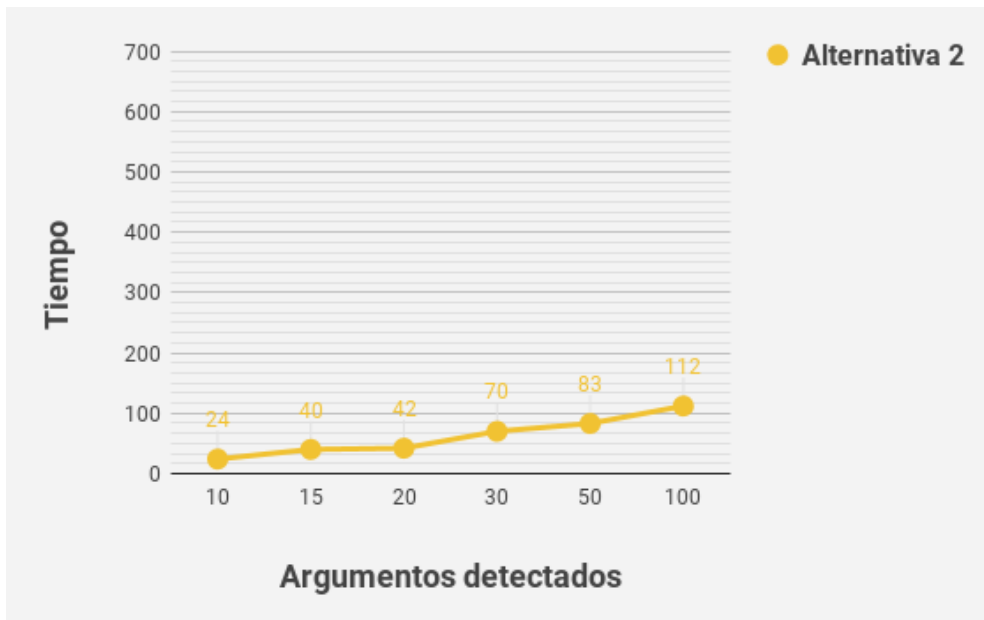


Figura 48. Gráfico de performance para la alternativa 2

Como se observa en este gráfico los tiempos de ejecución mejoran considerablemente respecto a la alternativa anterior, obteniendo como resultado del procesamiento de 100 sentencias un tiempo total de 112 minutos.

5.2.3 Alternativa 3: Salida ordenada y sin repeticiones

Esta alternativa tiene como objetivo mejorar aún más la performance del procesamiento.

En la figura 49 se visualizan los resultados obtenidos.



Figura 49. Gráfico de performance para la alternativa 3

Como se observa, nuevamente se logró una mejora con respecto a la performance obtenida en la alternativa anterior, disminuyendo el tiempo total de procesamiento a 105 minutos.

5.2.4 Comparación de resultados

Finalmente, con el objetivo de visualizar los resultados obtenidos de las alternativas estudiadas se presenta la figura 50.

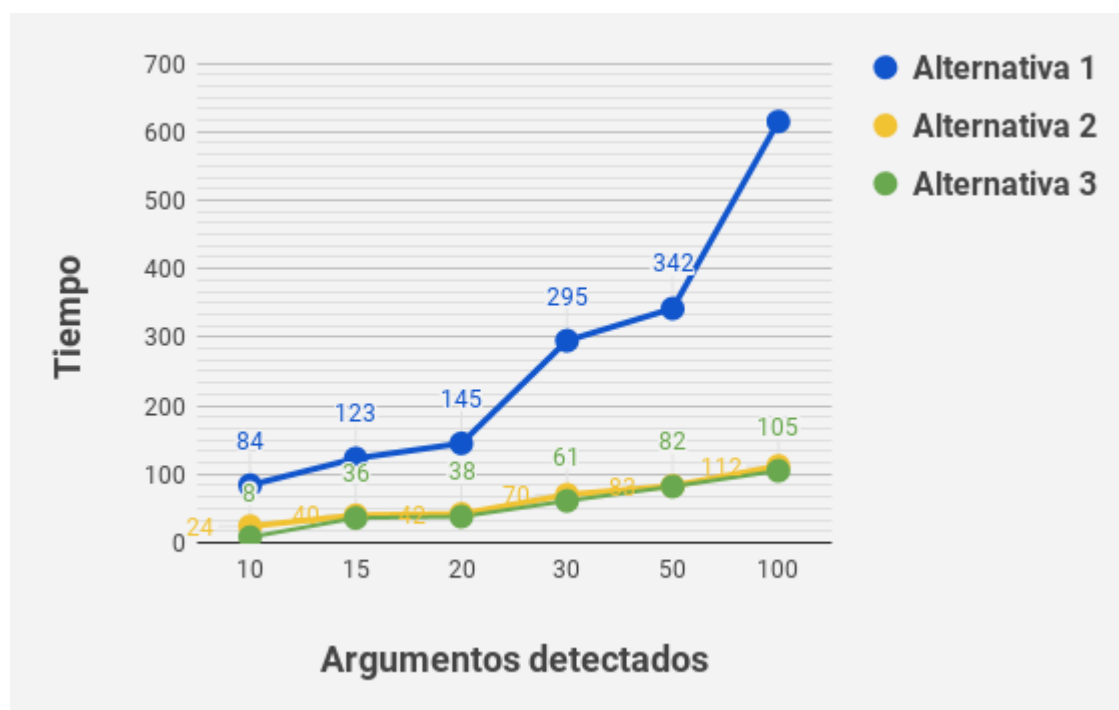


Figura 50. Gráfico comparativo

Al analizar el gráfico se observa claramente que, si bien se mantiene la precisión de los resultados, se obtiene una mejora en cuanto a performance con las alternativas 2 y 3 respecto de la 1. Esto demuestra el gran impacto que genera el ordenamiento previo de los sustantivos en los archivos de entrada (premisas y conclusiones) del procesamiento, lo cual genera una notoria disminución en la cantidad de consultas a la base de datos que se realizan.

Con respecto a la comparación de las alternativas 2 y 3, se observa que en algunos puntos del gráfico la mejora es más notoria que en otros, esto se debe a que dicha mejora está fuertemente ligada al contenido de los argumentos y la forma en que están redactados. Es decir, los argumentos en los cuales se repiten los sustantivos afectan considerablemente a la performance del procesamiento. En cambio, si la redacción del argumento no repite sustantivos, la performance de su análisis no se ve afectado.

5.3 Resumen

En este capítulo se presentaron los resultados obtenidos en cada uno de los enfoques propuestos.

Para el caso del enfoque basado en la estructura sintáctica, a la hora de realizar la evaluación de los resultados se tuvieron en cuenta la efectividad, precisión, Recall y F-Measure. Estas cuatro métricas se utilizaron para analizar la calidad del modelo generado.

Inicialmente se generó un modelo a partir de un dataset que no fue procesado por la herramienta de procesamiento de lenguaje natural con el objetivo de obtener una métrica que pueda ser utilizada como punto de partida para evaluar los resultados obtenidos en los diferentes enfoques. Para dicho dataset se obtuvo como mejor resultado una efectividad del 76,15% utilizando SMO como algoritmo de clasificación.

Luego, se obtuvieron los resultados de los distintos modelos generados con el objetivo de mejorar el modelo inicial. En primer lugar se analizó el modelo basado en el etiquetado de los conectores de premisa y conclusión, logrando como resultado una efectividad del 76,75% utilizando Naive Bayes como algoritmo de clasificación.

Finalmente, para este enfoque, se generaron los modelos basados en la longitud de etiquetas. Los resultados de los mismos fueron:

1. Longitud de etiqueta 6: efectividad del 77,75% utilizando SMO.
2. Longitud de etiqueta 3: efectividad del 64,12% utilizando SMO.
3. Longitud de etiqueta 2: efectividad del 63,32% utilizando Naive Bayes.

De este modo se concluye que, el enfoque basado en longitud de etiqueta 6 fué el modelo que mejores resultados obtuvo al momento de identificar la problemática de sentencias no argumentativas obteniendo una mejora del 1,6% sobre el modelo inicial.

Al analizar el enfoque basado en el análisis semántico de los argumentos, se consideraron en primera instancia, los resultados obtenidos del procesamiento de las entradas originales, es decir, en un primer análisis se tomaron las entradas tal cual fueron generadas por la herramienta. Este estudio generó resultados de muy baja performance dado que los tiempos de procesamiento fueron considerablemente altos. Al evaluar el porcentaje de éxito del análisis (con 10, 15, 20, 30, 50 y 100 argumentos) se obtuvieron buenos resultados, dado que dicho valor nunca fue inferior al 70%.

Posteriormente, se analizaron dos alternativas diferentes con el objetivo de mejorar la performance de la herramienta. La primera propone ordenar alfabéticamente los sustantivos de los archivos antes de realizar la comparación y la segunda adiciona a esta última la eliminación de palabras repetidas para, de este modo, disminuir la cantidad de combinaciones entre palabras a analizar.

Los resultados obtenidos con dichas alternativas lograron disminuir considerablemente los tiempos de ejecución, mejorando así la performance. Dichos resultados fueron:

1. Archivo original: tiempo de procesamiento de 615 minutos.
2. Ordenamiento alfabético: tiempo de procesamiento de 112 minutos.
3. Ordenamiento alfabético y eliminación de duplicados: tiempo de procesamiento de 105 minutos.

En conclusión, la propuesta que mejoró los resultados fue la que incluye un mayor preprocesamiento de los archivos de entrada, disminuyendo de esa manera la cantidad de combinaciones de palabras a analizar. Dicha mejora logró disminuir el tiempo de procesamiento para una entrada de 100 sentencias de 615 a 105 minutos.

Capítulo 6

6. Conclusiones

El enfoque basado en la estructura sintáctica de sentencias permitió determinar la causa por la cual una sentencia no es un argumento. Para llevar a cabo este objetivo, se consideraron diferentes aspectos los cuales fueron de suma importancia para obtener una respuesta efectiva y una mayor tasa de acierto en el resultado final.

El primer aspecto relevante fue la recolección de los datos de entrada. Fue fundamental utilizar un buen dataset para lograr resultados aceptables. Se considera que un buen dataset es aquel que contiene un balance entre la cantidad de sentencias con cada tipo de error sintáctico y sin falta de ortografías.

Otro de los aspectos de gran importancia para este enfoque, fue la incorporación de la herramienta de procesamiento de lenguaje natural Freeling. A partir de ella se logró hacer foco en la función semántica que cumple cada palabra en la sentencia no argumentativa logrando la abstracción de su significado. De esta manera, se logró un mayor encapsulamiento en las sentencias que posteriormente fueron analizadas por la herramienta Weka.

Se tuvieron en cuenta diferentes modelos con el objetivo de hallar la combinación que maximizará la tasa de acierto al momento de realizar la clasificación. De todos ellos, a partir de los diferentes algoritmos de clasificación que se utilizaron, SMO fue el que presentó mejor comportamiento utilizando el enfoque basado en longitud de etiqueta igual a 6, el cual obtuvo una efectividad del 77,75%.

Finalmente, como análisis de los resultados obtenidos se destaca la importancia del pre-procesamiento del dataset, ya que al considerar diferentes modelos con sus respectivas variantes se logró mejorar los resultados iniciales, lo que generó un incremento en la tasa de aciertos y, de esta manera, la herramienta desarrollada en este enfoque, logró ser eficiente y confiable.

La segunda herramienta desarrollada tuvo como objetivo la detección de argumentos semánticamente incorrectos, es decir, aquellos que aunque sean estructuralmente válidos, tienen inconsistencias semánticas.

Al igual que la herramienta detallada anteriormente, el dataset de entrada fue esencial para la obtención de buenos resultados. Por tal motivo, fue imprescindible una revisión y corrección de errores ortográficos antes del procesamiento. De dicho dataset se consideraron como entrada únicamente los argumentos sintácticamente válidos, con el objetivo de mejorar los resultados.

Al comienzo del procesamiento se utilizó la herramienta Freeling para el pre-procesamiento del dataset para la delimitación de las secciones de cada sentencia. Luego, se utilizó nuevamente Freeling para detectar los sustantivos presentes en cada una de dichas secciones.

Posteriormente, se utilizó la base de datos de Wordnet, la cual fue indispensable para la búsqueda y obtención de relaciones semánticas entre sustantivos presentes en premisas y conclusiones. La detección de una relación entre una palabra de la premisa y una de la conclusión de una sentencia, determinó la vinculación semántica de ambas partes y, de esta manera, su validez semántica.

Al realizar el procesamiento considerando los aspectos anteriores, los resultados fueron satisfactorios en cuanto a la detección de argumentos semánticamente correctos. Con el objetivo de obtener una mejora en la performance, se agregó una etapa previa al procesamiento en la cual se realizó un ordenamiento y eliminación de palabras repetidas con el objetivo de minimizar las consultas a la base de datos, dado que estas eran recursivas y tenían un gran impacto en el tiempo de procesamiento.

Los resultados obtenidos de la herramienta implementada (analizados en el capítulo anterior) fueron muy satisfactorios ya que el porcentaje de aciertos de argumentos semánticamente correctos en ninguna instancia fue inferior a un 70%. Adicionalmente, los tiempos de procesamiento disminuyeron notoriamente con las mejoras realizadas, lo cual indica la efectividad de las decisiones tomadas.

6.1 Limitaciones

En cuanto a las limitaciones de las herramientas generadas se pueden mencionar las siguientes:

Relacionadas al dataset inicial, se detectó que la presencia de abreviaciones, errores ortográficos y el uso de caracteres especiales (por ejemplo \$, %,], }) generan ruido a la hora de analizar las sentencias no argumentativas que componen dicho dataset. Adicionalmente, la gran variedad de formas de redacción de una sentencia incrementó la complejidad del análisis debiendo, de esta manera, contemplar las distintas formas de estructurar un argumento. Otro punto a considerar fue la omisión de alguna de las partes de la sentencia por el sentido común del redactor, por ejemplo, “El día está lindo, voy a salir a correr” lo cual disminuye la efectividad de la herramienta.

En cuanto a la determinación de los conectores argumentativos, el contexto al cual pertenecen las sentencias no argumentativas del dataset permitió detectar los que, a nuestro parecer, eran considerados como tal. De este modo, es probable que existan conectores argumentativos en otros contextos que no fueron detectados como tales. Por tal motivo, la herramienta puede disminuir su eficiencia en otro contexto o con otro dataset de entrada.

Adicionalmente, ante la ausencia de un dataset con sentencias clasificadas se tuvo la necesidad de realizar dicha clasificación. Como consecuencia, la subjetividad de cada persona provocó un gran impacto en los resultados. Por ejemplo, una sentencia no argumentativa para una persona no era válida porque no tenía premisa y para otra no lo era porque no tenía conclusión. Esto es consecuencia de la inexperiencia en cuanto a la clasificación de argumentos.

Al momento de utilizar la segunda herramienta implementada con grandes volúmenes de datos, el tiempo de respuesta incrementa considerablemente lo cual

representa una gran limitación en el uso de dicha herramienta. No obstante, se analizaron alternativas y, al aplicarlas, se logró reducir considerablemente el tiempo de procesamiento.

Respecto a la base de datos que se utilizó para la búsqueda y detección de relaciones semánticas, se percibió la ausencia de algunas relaciones consideradas, a nuestro entender, esenciales. Por ejemplo, se detectó que las palabras “frío” y “abrigo” están relacionadas y, sin embargo, “calor” y “abrigo” no lo están. Esto tiene un alto impacto en la calidad de los resultados.

6.2 Trabajos futuros

Con el objetivo de mejorar el primer enfoque propuesto, una de las alternativas será desarrollar una herramienta para la automatización de la clasificación del dataset de entrada. Dicha herramienta debería ser capaz de detectar las causas de error de cada sentencia no argumentativa y clasificarla.

Considerando el segundo enfoque, se podría profundizar con respecto al análisis que determina las formas de vinculación de palabras. Podrían considerarse relaciones entre verbos o adjetivos y se podrían estudiar otras formas de detectar relaciones. Por ejemplo, analizar otras tablas, para luego analizar cómo esto impacta en los resultados.

También podría realizarse el estudio semántico reemplazando el uso de una base de datos por otra estructura, por ejemplo, mediante la utilización de librerías específicas.

Adicionalmente, en el enfoque semántico, se podría agregar un módulo de traducción de texto a sentencias lógicas con el objetivo de validar dichas sentencias lógicamente. Esto tendría como objetivo ampliar el análisis semántico determinando si una sentencia es lógicamente válida.

Referencias

- [1] Sierra C., Jennings N. Noriega P., and Parsons S. A framework for argumentation based negotiation, in Proc. 4th Int. Workshop on ATAL, Rode Island, USA. 1998. 177-192.
- [2] Palau, R. M. and Moens, M.-F. (2009). Argumentation mining: The detection, classification and structure of arguments in text. In Proc. of the 12th Int. Conf. on Artificial Intelligence and Law, ICAIL '09, pages 98–107, New York, NY, USA. ACM
- [3] Arana Roa Juan Bruno, Lake Moctezuma Franz Ludwig, Vergara Albarrán Enrique Alberto. "Implementación de un módulo de análisis semántico sobre el analizador sintáctico Freeling".
- [4] Sameer Pradhan, Kadri Hacioglu, Valerie Krugler, Wayne Ward, James Martin and Daniel Jurafsky. "Support Vector Learning for Semantic Argument Classification". Machine Learning, 60, 11–39, 2005.
- [5] Alessandro Moschitti and Cosmin Adrian Bejan. "A Semantic Kernel for Predicate Argument Classification".
- [6] Timarán, R. (2009). Una mirada al descubrimiento de conocimiento en bases de datos. Revista Ventana Informática, 20, 39-58.
- [7] Agrawal, R. y Srikant, R. (1994). Fast Algorithms for Mining Association Rules. vldb Conference, Santiago de Chile.
- [8] Chen, M., Han, J. y Yu, P. (1996). Data Mining: An Overview from Database Perspective. iee Transactions on Knowledge and Data Engineering
- [9] Han, J. y Kamber, M. (2001). Data Mining Concepts and Techniques. San Francisco: Morgan Kaufmann Publishers
- [10] Fayyad, U., Piatestky-Shapiro, G. y Smyth, P. (1996). The kdd Process for Extracting Useful Knowledge from Volumes of Data. Communications of the acm, 39(11), 27-34.
- [11] Jeria, V. H. (2007). Minería web de uso y perfiles de usuario: aplicaciones con lógica difusa. Granada.
- [12] Dubiau, L. (2013). Procesamiento de Lenguaje Natural en Sistemas de Análisis de Sentimientos.
- [13] Baeza-Yates, R. & Ribeiro-Neto, B. (1996). *Modern information retrieval: The concepts and technology behind search*. Reading, M.A.: Addison-Wesley.
- [14] Sebastiani, F. (2002). Machine learning in automated text categorization. *ACM Computing Surveys*, 34(1), 1-47.
- [15] Mitchell, T. (1997). *Machine learning*. Nueva York: WCB/Mcgraw-Hill.
- [16] K.A. Vidhya y G. Aghila, "Hybrid Text Mining Model for Document Classification," IEEE 1 (2010): 210.

- [17] C. D. Manning, P. Raghavan, y H. Schütze, *Introduction to Information Retrieval* (Cambridge: Cambridge University Press, 2008), 297; Torunoglu et al., "Analysis of Preprocessing Methods...", 114.
- [18] Wanjun y Xiaoguang, "Research on Text Categorization Based on Machine Learning", 254.
- [19] Wanjun y Xiaoguang, "Research on Text Categorization Based on Machine Learning", 52-53.
- [20] Ibid., 8.
- [21] Daniel Jurafsky y James H. Martin. *Speech and Language Processing*. 2nd Edition. Prentice Hall, 1999.
- [22] Padró, L. (2011). Analizadores Multilingües en FreeLing. *Linguamatica*.
- [23] Stanilovsky, E., & Padró, L. (2012). FreeLing 3.0. Towards Wider Multilinguality. *Proceedings of the Language Resources and Evaluation Conference*.
- [24] Chandrasekaran B., Josephson J.R., Benjamins V.: What Are Ontologies, and Why Do We Need Them?. *IEEE Intelligent Systems*, v.14 n.1, (1999) pp 20-26.
- [25] Pisanelli D.M., Gangemi A., Steve G.: Ontologies and Information Systems: the Marriage of the Century?. In *Proceedings of Lyes Workshop*, Paris, (2002)
- [26] Wang X., Chan C., Hamilton H.: Design of knowledge-based systems with the ontology-domain-system approach. In *Proceedings of SEKE 2002*, (2002) pp 233-236.
- [27] Miller, G. A. (1986) "Dictionaries in the Mind", *Language and Cognitive Processes*, Vol. 1, No. 3: 171-185.
- [28] Rodríguez, Horacio, Salvador Climent, Piek Vossen, Laura Bloksma, Wim Peters, Antonietta Alonge, Francesca Bertagna, e Adriana Roventini. 1998. The top-down strategy for building eurowordnet: Vocabulary coverage, base concepts and top ontology. *Computers and the Humanities*, 32(2-3):117–152.
- [29] Magnini, Bernardo, C. Satrapparava, G. Pezzulo, e A. Gliozzo. 2002. The role of domains informations. Em *In Word Sense Disambiguation*, Treto, Cambridge, July, 2002.
- [30] Magnini, Bernardo e G. Cavagliá. 2000. Integrating subject field codes into wordnet. Em *Proceedings of the Second International Conference on Language Resources and Evaluation (LREC)*, Athens. Greece.
- [31] Niles, I. e Adam Pease. 2001. Towards a standard upper ontology. Em Chris Welty e Barry Smith, editores, *Proceedings of the 2nd International Conference on Formal Ontology in Information Systems (FOIS-2001)*, Ogunquit, Maine.
- [32] Álvarez, Javier, Paqui Lucio, e G. Rigau. 2012. Adimen-sumo: Reengineering an ontology for first-order reasoning. *International Journal on Semantic Web and Information Systems*, 8(4):80–116.

[33] Benítez, Laura, Sergi Cervell, Gerard Escudero, Mónica López, German Rigau, e Mariona Taulé. 1998. Methods and tools for building the catalan wordnet. Em Proceedings of ELRA Workshop on Language Resources for European Minority Languages, Granada, Spain.

[34] Capaldi, N. (2000). La identificación de los argumentos. Cómo ganar una discusión, 27-37.

[35] Corso, L. C. (2009). Aplicación de algoritmos de clasificación supervisada usando weka.